

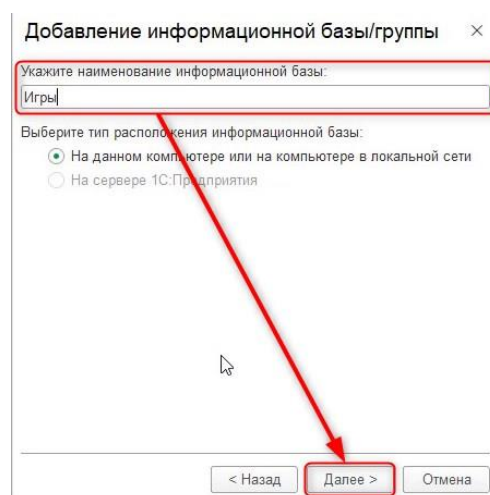


Рис. 1.2.18. Наименование информационной базы

После этого укажем каталог информационной базы, в котором она будет располагаться. Папку можно оставить по умолчанию и нажать кнопку «Далее» (рис. 1.2.19).

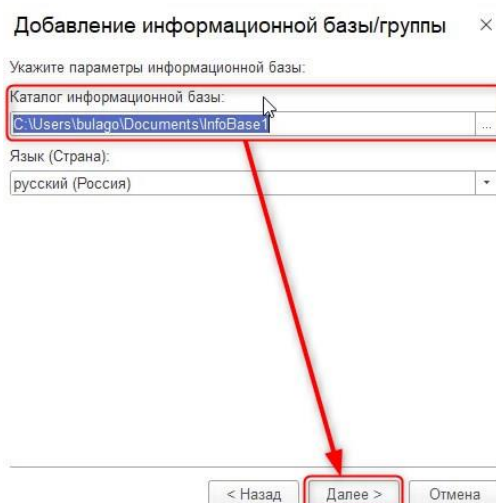


Рис. 1.2.19. Каталог информационной базы

В следующем окне нет необходимости менять никакие настройки, можно нажать «Готово» (рис. 1.2.20).

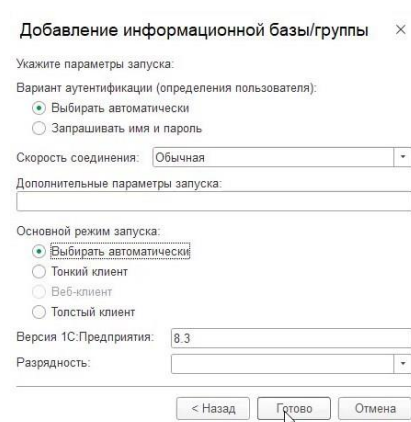


Рис. 1.2.20. Завершение добавления информационной базы

В результате в списке появится новая база «Игры», которая является нашей программой, в которой будут находиться игры. Если запустить её, нажав дважды левой кнопкой мыши по названию, то откроется совершенно пустое окно программы (рис. 1.2.21 – 1.2.22).

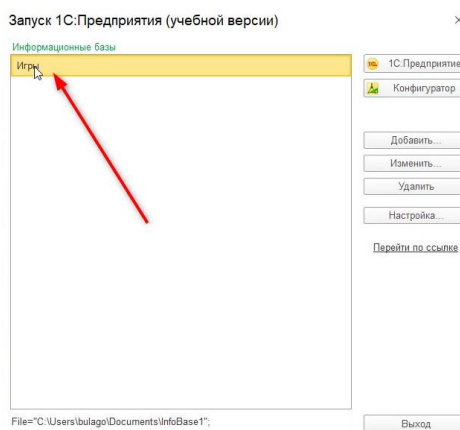


Рис. 1.2.21. Открытие программы

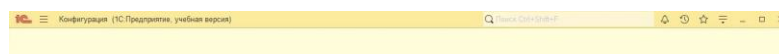


Рис. 1.2.22. Пустое окно программы

Приступим к знакомству с платформой. Для этого закроем пустое окно программы и заново запустим 1С. После перейдём в режим конфигулятора (рис. 1.2.23).

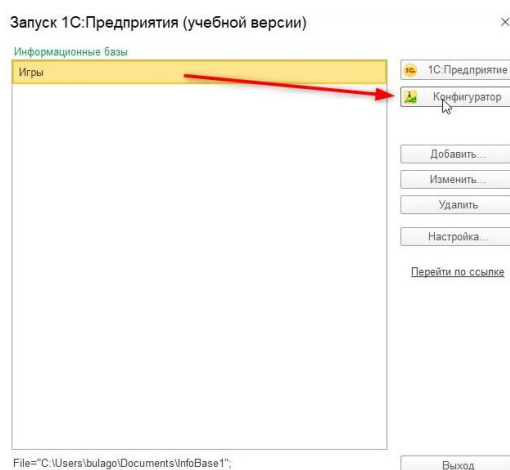


Рис. 1.2.23. Открытие режима конфигулятора

Важно!

Есть несколько режимов запуска информационной базы. Одни из них – «1С:Предприятие» и «Конфигуратор». Первый является пользовательским режимом, а второй – режимом для разработчика.

В результате открывается конфигуратор. Конфигуратор – это среда разработки, в которой мы будем создавать игры.

В каждой среде разработки создание нового функционала происходит по-разному. В случае конфигулятора, разработка происходит на основании готовых объектов. Нам не нужно заново создавать детали конструктора, их необходимо только грамотно использовать. Для того чтобы увидеть эти «детали», нужно открыть конфигурацию с помощью пункта меню «Конфигурация» – «Открыть конфигурацию» (рис. 1.2.24).

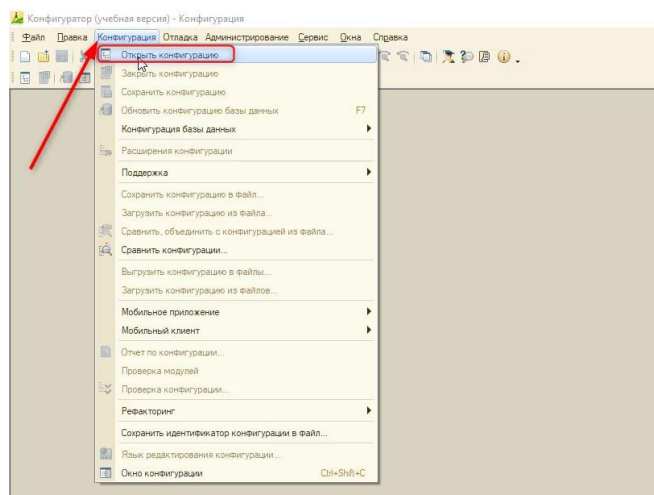


Рис. 1.2.24. Открытие дерева конфигурации

Открывшийся набор «деталей» называется деревом конфигурации (рис. 1.2.25). А сами «детали» – это прикладные объекты.

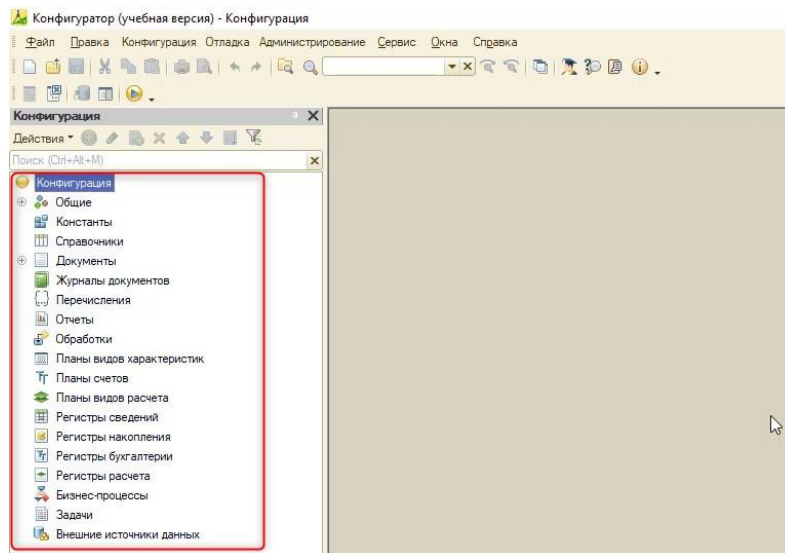


Рис. 1.2.25. Дерево конфигурации

Для игры понадобится форма с кнопками, с выводом картинок и текста. Такой функционал есть у объекта «Обработки». Для того чтобы создать новую обработку, необходимо нажать по ней правой кнопкой мыши и выбрать «Добавить» (рис. 1.2.26).

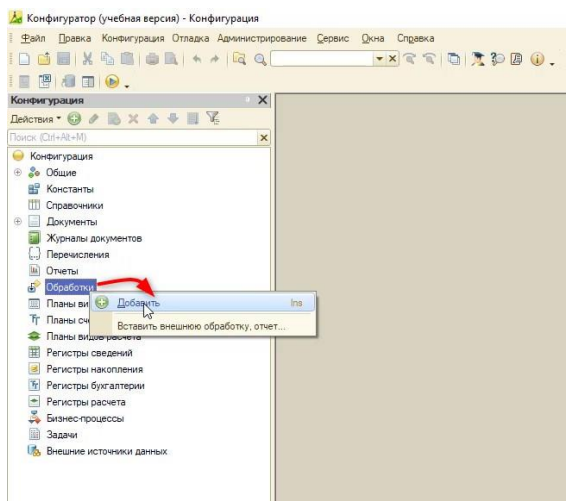


Рис. 1.2.26. Создание новой обработки

Важно!

Обработка – это прикладной объект конфигурации, который может производить различные действия с данными. Обработки являются дополнениями к программе, которые позволяют решать разные задачи – например, с их помощью можно делать небольшие игры.

Дадим имя новой обработке – «Кубики» и нажмём кнопку «Enter» на клавиатуре. После этого автоматически заполнится синоним обработки (рис. 1.2.27).

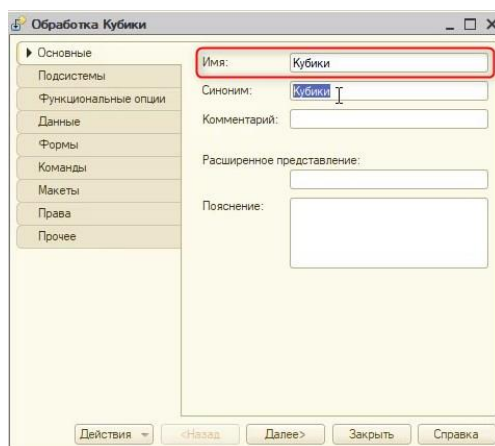


Рис. 1.2.27. Добавление имени новой обработке

Синоним – это отображение названия объекта в пользовательском режиме, а имя – это название объекта в самой программе.

Есть ряд правил при заполнении имени объекта, а именно:

- 1) Нельзя использовать пробелы;
- 2) Имя не может начинаться с цифры;
- 3) Имя не может состоять из одного символа;
- 4) Нельзя использовать специальные символы, кроме «_».

Обычно имена в 1С заполняются верблюжьим регистром. Это стиль написания, при котором несколько слов пишутся слитно без пробелов, при этом каждое слово внутри фразы пишется с прописной буквы. К примеру, «Игра с кубиками» в верблюжьем регистре будет выглядеть как «ИграСКубиками».

На наименование синонима такие правила не распространяются. Кроме того, синоним в 1С автоматически заполнится с нужными пробелами на основе имени в верблюжьем регистре (рис. 1.2.27).

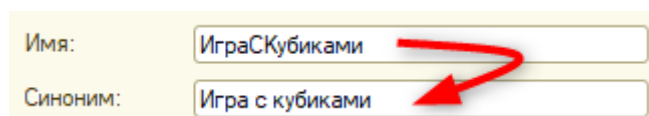


Рис. 1.2.28. Автоматическое преобразование имени в синоним

Если в данный момент запустить пользовательский режим, то он будет по-прежнему пуст. Необходимо перенести функционал, а именно сохранить его с помощью соответствующей кнопки на панели (см. рис. 1.2.29).

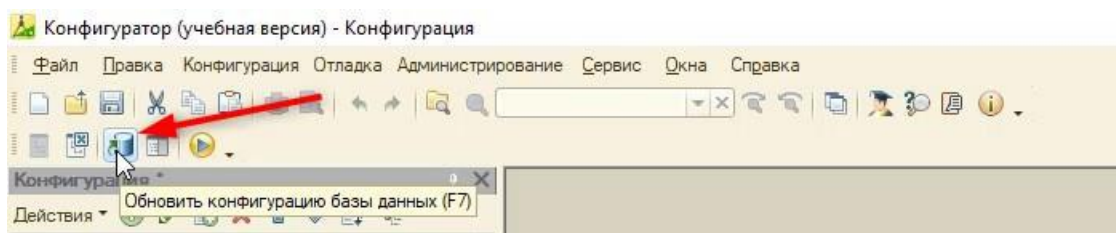


Рис. 1.2.29. Кнопка сохранения

Созданный функционал автоматически не сохраняется, потому что это может повлечь за собой ошибки и потерю данных. К примеру, в играх всегда есть «Технические работы», когда нельзя взаимодействовать с приложением. В это время происходит обновление, чтобы у игрока во время игры не произошёл сбой.

Для того чтобы пользоваться каким-либо функционалом, нужен интерфейс.

Важно!

Интерфейс пользователя – это все окна, меню, кнопки и прочее, с чем пользователь работает непосредственно в программе.

На данный момент такого интерфейса нет, поэтому в пользовательском режиме обработка отсутствует. Проверить это можно, запустив пользовательский режим. Для этого не нужно запускать заново программу, а достаточно нажать на пункт главного меню «Сервис» – «1С:Предприятие» (рис. 1.2.30).

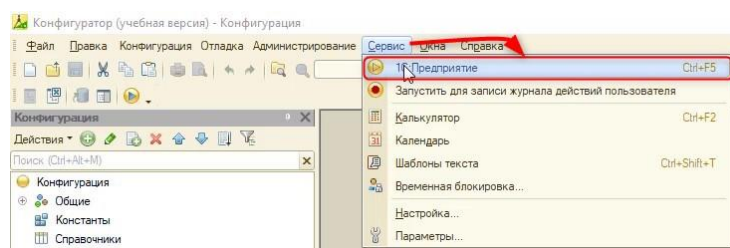


Рис. 1.2.30. Открытие пользовательского режима в конфигураторе

Программа запустится, но обработки в ней ожидаемо нет.

У обработки, в отличие от других прикладных объектов (справочник, документы), нет стандартной, заданной программой формы. Её необходимо создать самостоятельно, чтобы обработка отобразилась в интерфейсе.

Все формы обработки отображены в одноименной вкладке – «Формы». На данной вкладке можно как создать новую, так и открыть и редактировать существующие формы. Для создания новой формы нажмём кнопку «Добавить» (рис. 1.2.31).

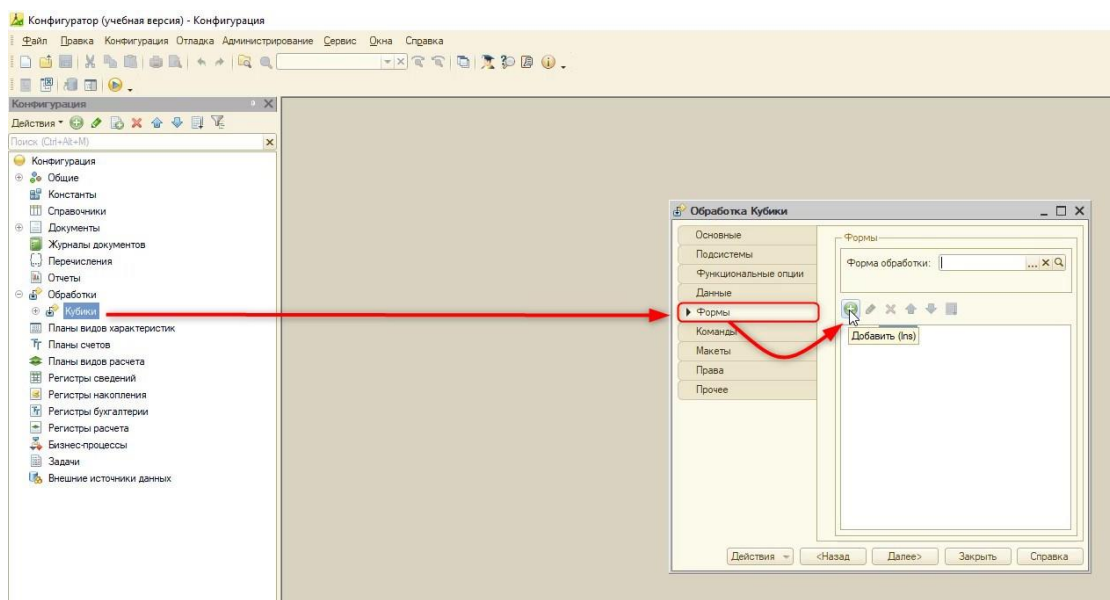


Рис. 1.2.31. Создание формы для обработки

В открывшемся окне должен быть обязательно активен флаг (галочка) рядом с «Назначить форму основной» (рис. 1.2.32). Это нужно, чтобы наша форма отображалась пользователю в интерфейсе.

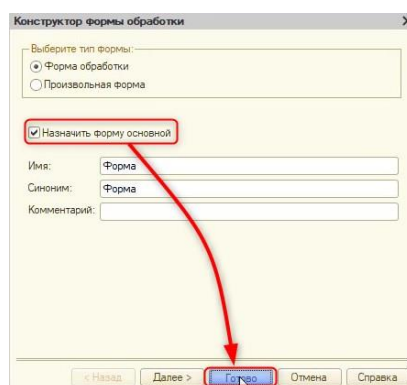


Рис. 1.2.32. Создание основной формы

После этого нажмем «Готово», и откроется окно конструктора формы (рис. 1.2.33).

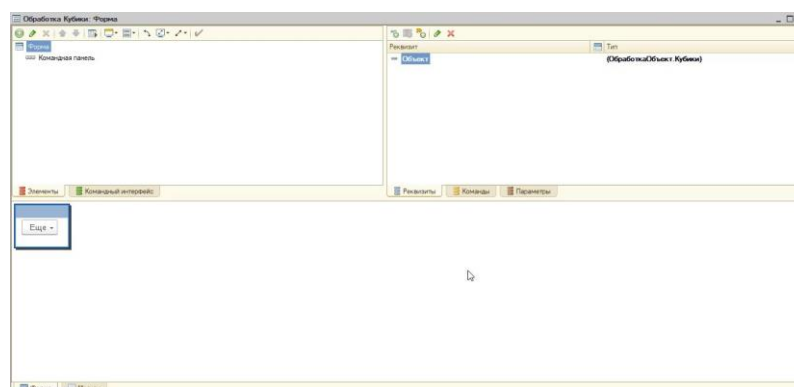


Рис. 1.2.33. Конструктор форм

Обновим конфигурацию базы данных, как делали ранее, и откроем пользовательский режим.

Теперь в режиме «1С:Предприятие» появится кнопка «Сервис», с помощью которой можно открыть будущую игру (рис. 1.2.34).

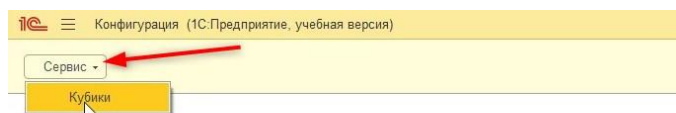


Рис. 1.2.34. Пользовательский режим

На форме по-прежнему пусто, поэтому закроем пользовательский режим и начнём разработку.

Конструктор форм состоит из нескольких областей (рис. 1.2.35):

- 1) Область элементов формы – это всё, что касается внешнего вида формы (кнопки, поля ввода, картинки и т. д.);
- 2) Область реквизитов – это часть формы, которая хранит данные. К примеру, количество кубиков, которые выбрасывает игрок, значения этих кубиков и т. д.;
- 3) Предварительный внешний вид формы.

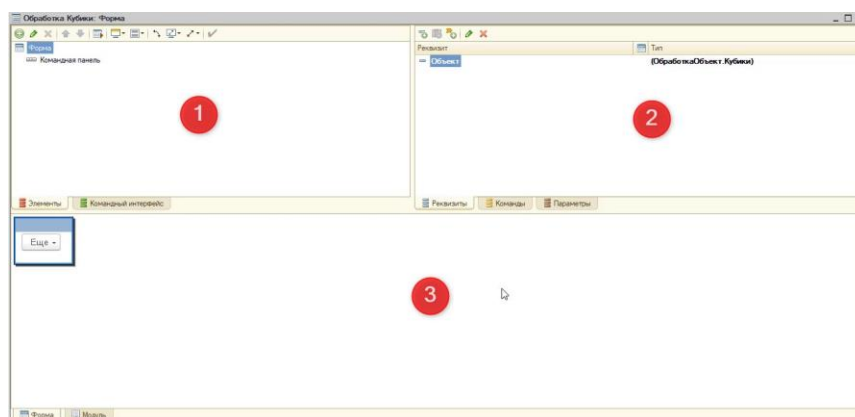


Рис. 1.2.35. Области конструктора форм

Научим нашу форму хранить числовое значение кубика, который выпал после броска. Так как в наших играх будет максимум три кубика, понадобится три значения.

Чтобы добавить их, можно (рис. 1.2.36):

- 1) Нажать кнопку «Добавить» в области реквизитов;
- 2) С помощью правой кнопки мыши щелкнем по нужной области и выберем «Добавить реквизит».

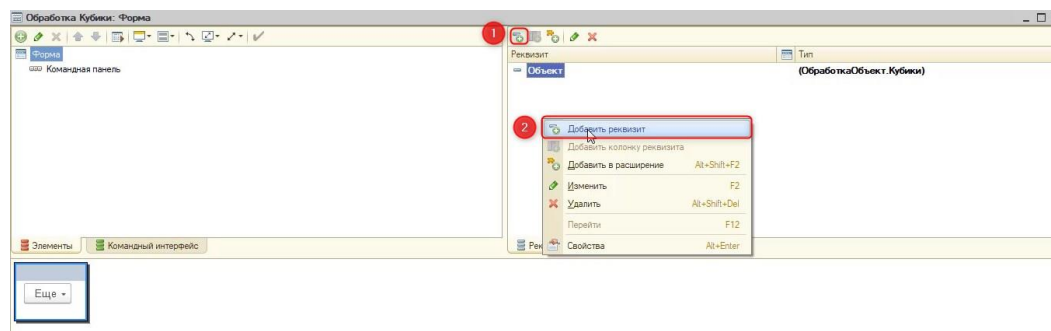


Рис. 1.2.36. Добавление реквизита

Для настройки реквизитов используется палитра свойств. Открыть её можно нажатием левой клавиши мыши по нужному реквизиту.

Свойства могут отображаться в виде вкладок или списком. Сменить эти два режима можно, нажав левой клавишей мыши по пустой области окна и выбрав «Списком» или «Закладками» (рис. 1.2.37).

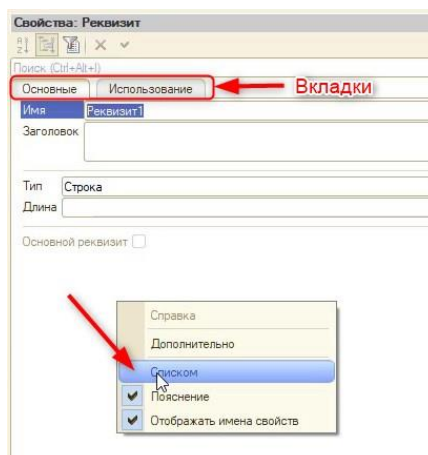


Рис. 1.2.37. Смена режима отображения

Рекомендуем оставить отображение в виде вкладок.

Частая ошибка начинающего программиста – это включенный фильтр (рис. 1.2.38). При включенном фильтре отображаются только важные свойства, поэтому важно оставлять его выключенным.

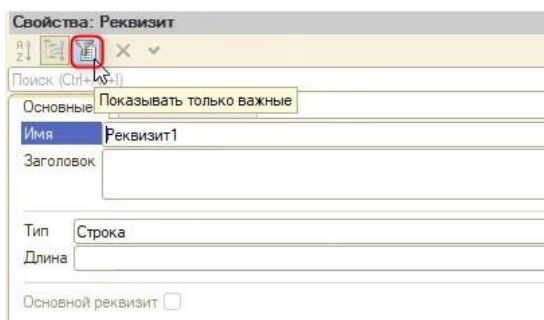


Рис. 1.2.38. Фильтр

Присвоим имя новому реквизиту в верблюжьем регистре – «ЧислоКубик1» и тип данных «Число» (рис. 1.2.39).

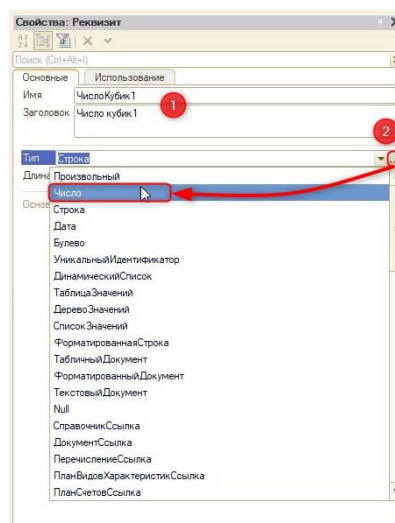


Рис. 1.2.39. Настройка реквизита

Программа может хранить множество разных типов данных: число, строка, дата, булево (имеет только два значения «Истина» или «Ложь»). Так как значение будет от 0 до 6, то задается тип «Число».

У типа «Число» если несколько дополнительных настроек:

- 1) Длина (в символах). В нашем случае – 1, так как граней на кубике больше 6 не будет;
- 2) Точность (количество цифр после запятой). Так как у нас целые числа, будет 0;
- 3) Неотрицательное (запрет на отрицательные числа).

Сделаем настройку реквизита так, как представлено на рис. 1.2.40.

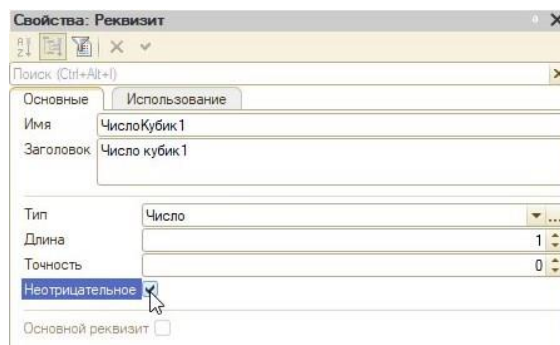


Рис. 1.2.40. Настройка реквизита

По аналогии создадим два таких же реквизита.

Для того чтобы реквизиты появились на форме, нужно создать соответствующие элементы. Сделать это можно с помощью простой операции (рис. 1.2.41):

- 1) Удерживаем кнопку «Ctrl» на клавиатуре и выбираем все три реквизита;
- 2) Зажимаем левую клавишу на реквизитах и перетаскиваем их в левую область.

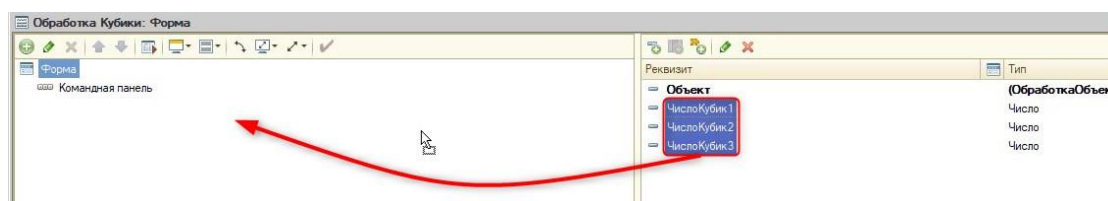


Рис. 1.2.41. Создание элементов

После этого на форме появятся элементы, связанные с реквизитами, которые хранят данные.

Для того чтобы логически отделить элементы, относящиеся к картинкам или, к примеру, к выводу чисел, пригодится механизм групп. Группы – это «папки», которые могут содержать поля, таблицы, кнопки и группы других видов. Конкретно в данной ситуации нам нужна группа «Обычная группа без отображения», она отличается тем, что её заголовок не виден пользователю.

Для создания группы нужно нажать кнопку «Добавить» в области элементов и выбрать «Обычная группа без отображения» (рис. 1.2.42).

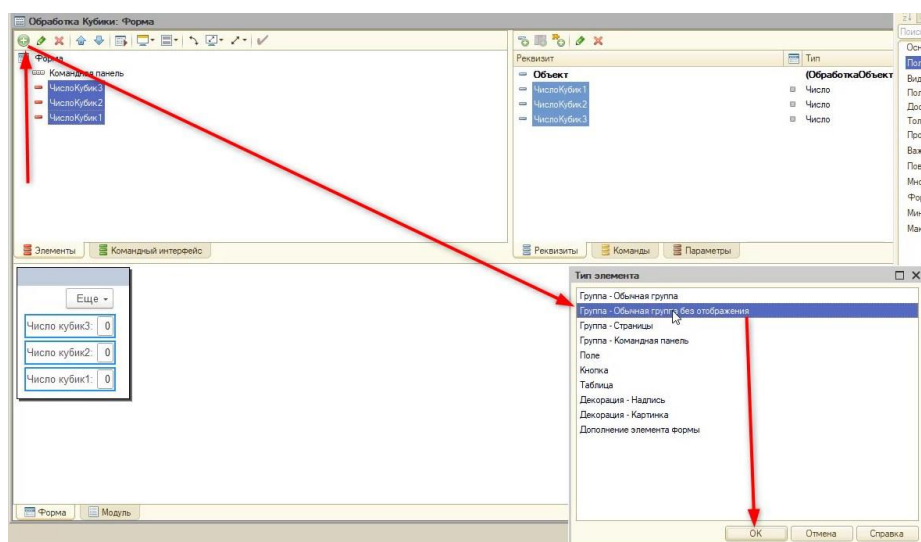


Рис. 1.2.42. Создание группы без отображения

Присвоим заголовок новой группе: «Числа кубиков» (рис. 1.2.43).

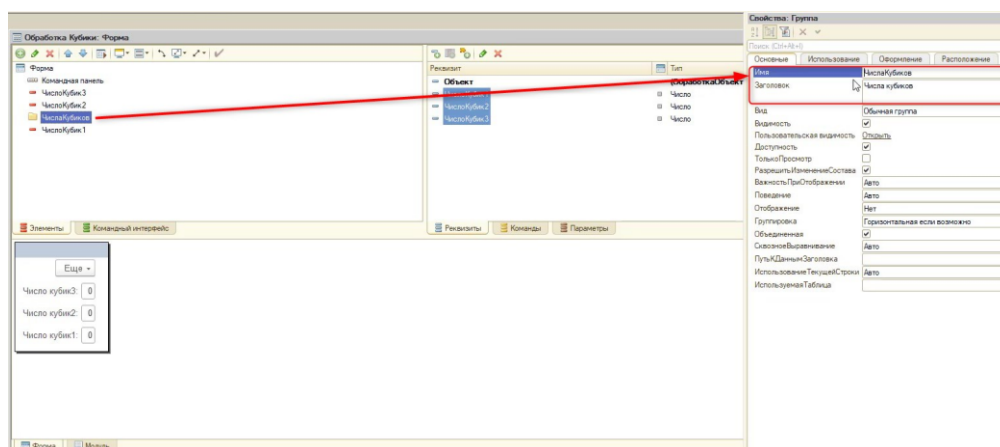


Рис. 1.2.43. Свойства группы

После этого в группу перенесем все элементы, которые создали ранее. В итоге получится следующая картина (рис. 1.2.44).

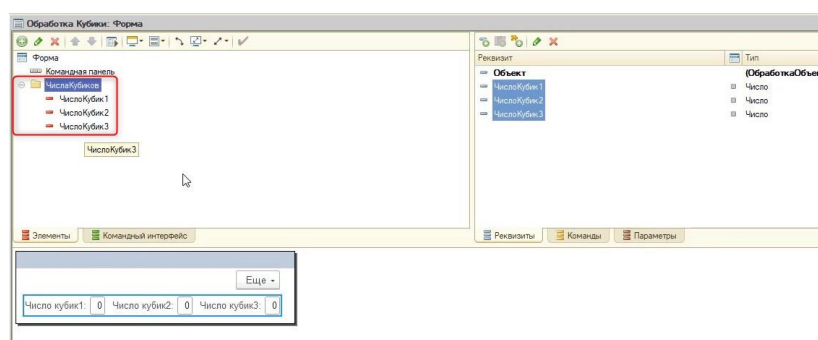


Рис. 1.2.43. Перенос элементов в группу

Если открыть в данный момент пользовательский режим, предварительно сохранив все изменения в конфигураторе, то на форме появится возможность ввести значения (рис. 1.2.44).

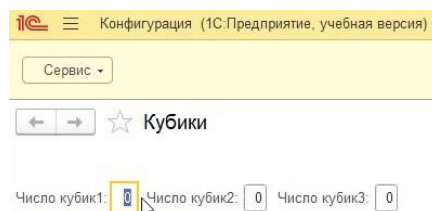


Рис. 1.2.44. Форма в пользовательском режиме

С логической точки зрения это неправильно, так как значения должны формироваться автоматически при нажатии на кнопку. Следующей нашей задачей будет создание такой кнопки.

Для начала, чтобы создать кнопку, нужно в области реквизитов нужно перейти на вкладку «Команды» (рис. 1.2.45) и нажать на «Добавить».

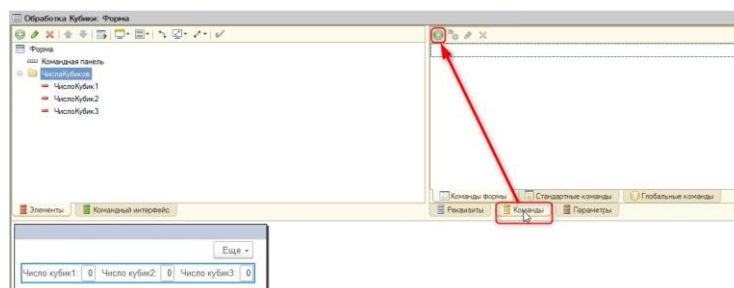


Рис. 1.2.45. Создание команды

Назовём новую команду «Бросить» и перенесём её на область элементов в командную панель, как делали ранее с реквизитами (рис. 1.2.46).

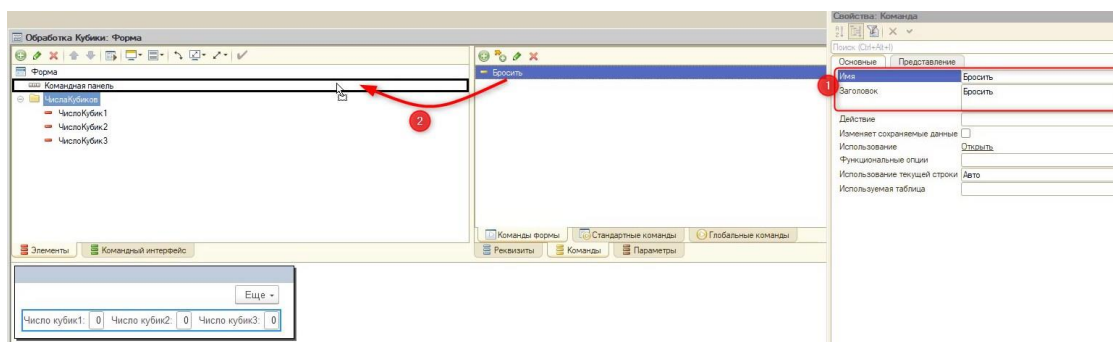


Рис. 1.2.46. Создание кнопки

Теперь у пользователя появится кнопка броска (рис. 1.2.47), но при нажатии на неё ничего происходить не будет.

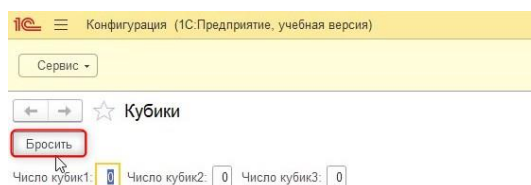


Рис. 1.2.47. Кнопка броска в пользовательском режиме

Связано это с тем, что мы программно не определили действие, которое должно происходить при нажатии. Перед тем, как его определить, обозначим, что при броске должны случайным образом появляться значения кубиков от 1 до 6 соответственно.

Для того чтобы это сделать, нужно написать код. Перейти к написанию кода команды можно несколькими способами:

- 1) С помощью нажатия правой кнопки мыши по кнопке в области элементов и выбора «Действие команды» (рис. 1.2.48);

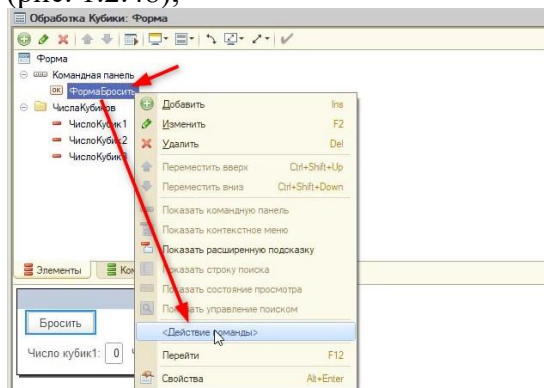


Рис. 1.2.48. Действие команды

- 2) Открыть свойства команды и рядом со свойством «Действие» нажать кнопку в виде лупы (рис. 1.2.49).

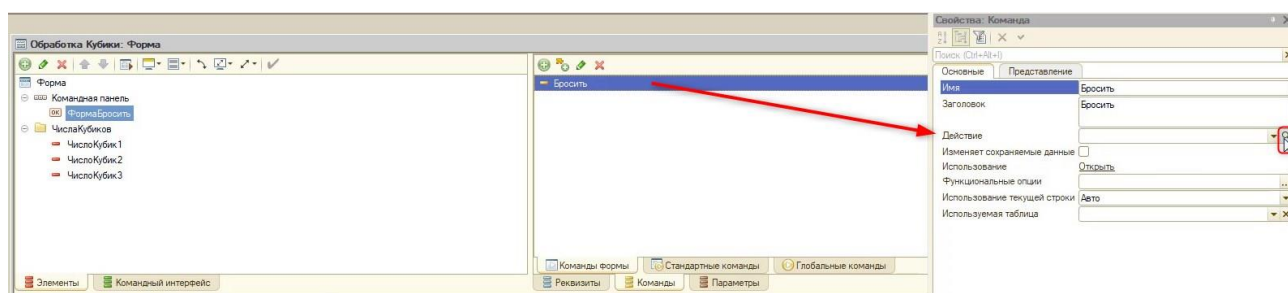


Рис. 1.2.49. Создание действия

Когда мы выбираем создать действие для команды, система спрашивает, где будет находиться обработчик команды.

Важно!

Обработчик команды – это порядок действий, который происходит при нажатии кнопки.

Поскольку взаимодействие с сервером нам не нужно, оставим стандартный выбор и нажмём кнопку «Ок» (рис. 1.2.50).

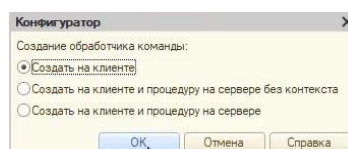


Рис. 1.2.50. Создание обработчика команды

Важно!

На клиенте – если действие происходит перед глазами, и системе не нужно обращаться в другую часть программы.

На сервере же доступно взаимодействие с другой частью программы.

В результате программа создаст процедуру в модуле формы (рис. 1.2.51).

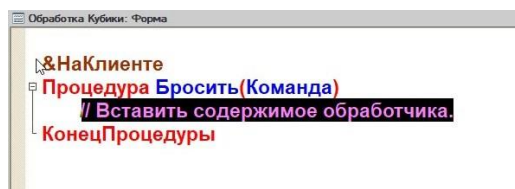


Рис. 1.2.51. Процедура

Важно!

В модуле содержится программный код.

Модуль формы предназначен для того, чтобы обработать действия пользователя. Например, описать алгоритм реакции программы при нажатии кнопки. Или, например, в момент ввода в поле значения сразу же выполнить проверку на корректность.

Если необходимо вернуться из модуля на форму, можно воспользоваться кнопками, расположенными внизу окна конструктора (рис. 1.2.52).

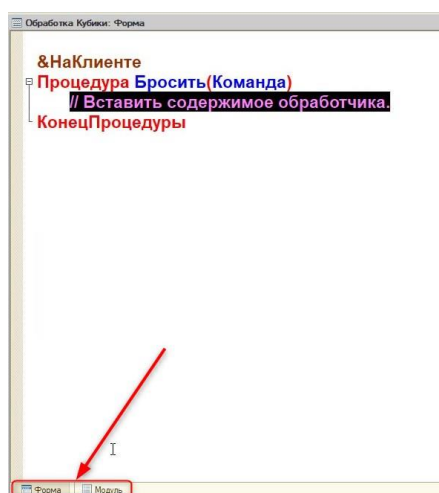


Рис. 1.2.52. Переключение окон

Процедура – это порядок действий, который должна выполнить программа.

Задача на данный момент – программно сгенерировать 3 случайных числа и подставить их в реквизиты. В данном блоке будет разрабатываться не одна игра с кубиками, а несколько, поэтому функционал броска придётся использовать много раз в коде. Поэтому более грамотно будет изначально создать под него отдельную процедуру, которую мы будем вызывать. Вызов такой процедуры упростит написание кода и сократит его количество.

Для создания процедуры нужно написать ниже в модуле «проц», нажать сочетание клавиш **Ctrl+Q** и выбрать первый вариант – «Процедура».

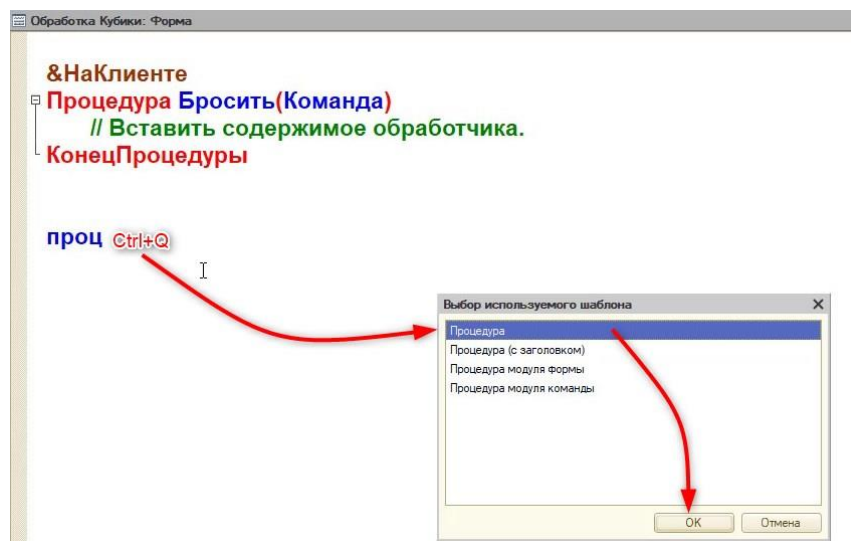


Рис. 1.2.53. Создание процедуры

Далее программа попросит указать имя процедуры.

Важно!

Правильный выбор имен процедур и функций очень важен для повышения читаемости кода. Так как за большими проектами разработки всегда стоит целая команда, код должен быть понятен всем её участникам.

Назовём нашу процедуру «БроситьКубики» и нажмём кнопку «ОК» (рис. 1.2.54).

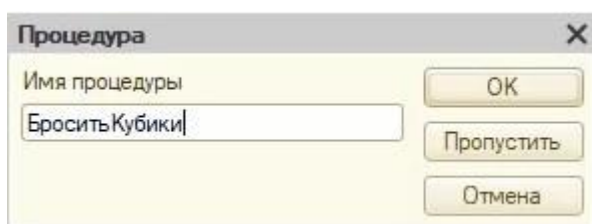


Рис. 1.2.54. Имя процедуры

В модуле появится процедура, и теперь нужно указать программе, что она выполняется на клиенте. Для этого перед процедурой нужно написать «дирек» и нажать сочетание клавиш Ctrl+Q, как делали ранее. После выбираем «НаКлиенте» (рис. 1.2.55).

Важно!

Директивы компиляции – это директивы, которые определяют, в какой среде будут исполняться процедуры и функции (на сервере или на клиенте). Перед определением директивы ставится «&».

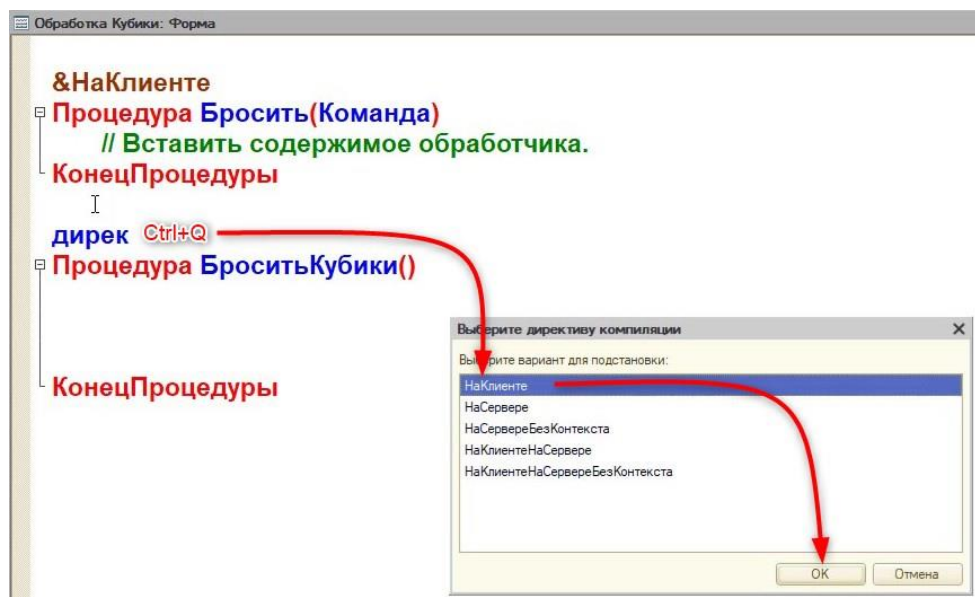


Рис. 1.2.55. Директива компиляции

Теперь можно приступить к программированию!

Важно!

Для быстрого написания кода используйте сочетание клавиш **Ctrl+Пробел**. Оно не только подскажет, что можно по синтаксису написать дальше, но и исключит орфографические ошибки (рис. 1.2.56).

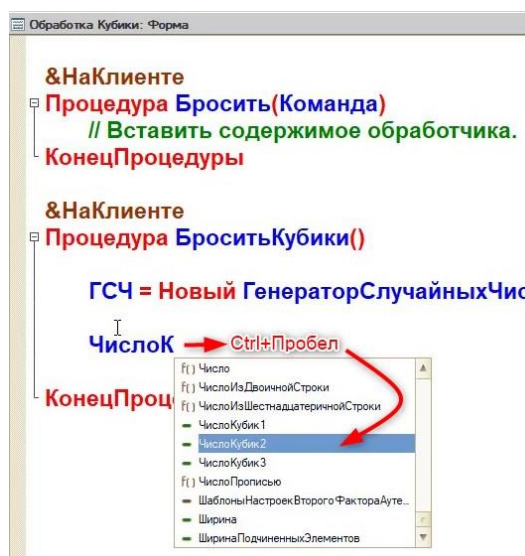


Рис. 1.2.56. Пример использования горячей клавиши «Ctrl+Пробел»

В дальнейшем в данном методическом материале описание кода будет в виде комментариев в нём. Комментирование кода – это внесение пояснений в тест модулей, которые не являются обязательными и не влияют на алгоритм (не исполняются).

Производится только с помощью последовательности «//», при этом комментарием считается все, что находится после. В 1С комментарий выделяется зеленым цветом (рис. 1.2.57).

```

&НаКлиенте
Процедура Бросить(Команда)
    // Вставить содержимое обработчика.
КонецПроцедуры

```

Рис. 1.2.57. Пример комментария

Пример комментария:

Перем ЭтоНеКомментарий;\ А это уже комментарий \Это тоже комментарий

Напишем следующий код в модуле формы:

```

&НаКлиенте
Процедура Бросить(Команда)
    БроситьКубики(); //Вызов нижней процедуры, без нее код выполняться не будет
КонецПроцедуры

&НаКлиенте
Процедура БроситьКубики()
    //Создадим новую переменную, которая будет содержать в себе генератор случайных чисел
    ГСЧ = Новый ГенераторСлучайныхЧисел;
    //Обращаемся к реквизитам формы(к данным) через ЭтотОбъект.ИмяРеквизита
    //С помощью функции СлучайноеЧисло(НижняяГраница, ВерхняяГраница) - заполним
    рандомным числом из диапазона

    ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1,6); //(1,6) - от 1 до 6
    ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1,6);
    ЭтотОбъект.ЧислоКубик3 = ГСЧ.СлучайноеЧисло(1,6);

КонецПроцедуры

```

Важно!

Все строки друг от друга в программном коде отделяются с помощью символа «;». С помощью точки с запятой программа понимает, что закончилась команда и далее должна выполняться другая.

После этого сохраним все изменения и откроем пользовательский режим. Если все сделано верно, то при нажатии кнопки «Бросить» будут выпадать случайные числа (рис. 1.2.58).

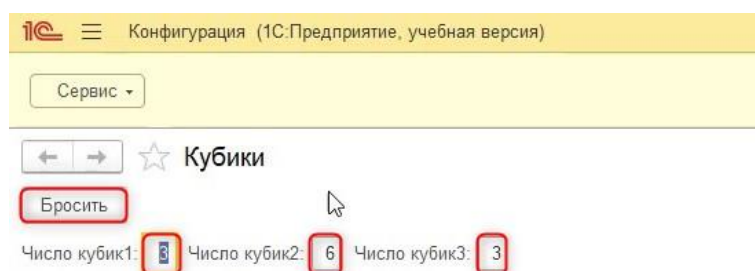


Рис. 1.2.58. Работа программы

Усложним задачу и добавим пользователю выбор количества (от 1 до 3) кубиков для броска. Для этого вернёмся обратно на форму и создадим новый реквизит (рис. 1.2.59).

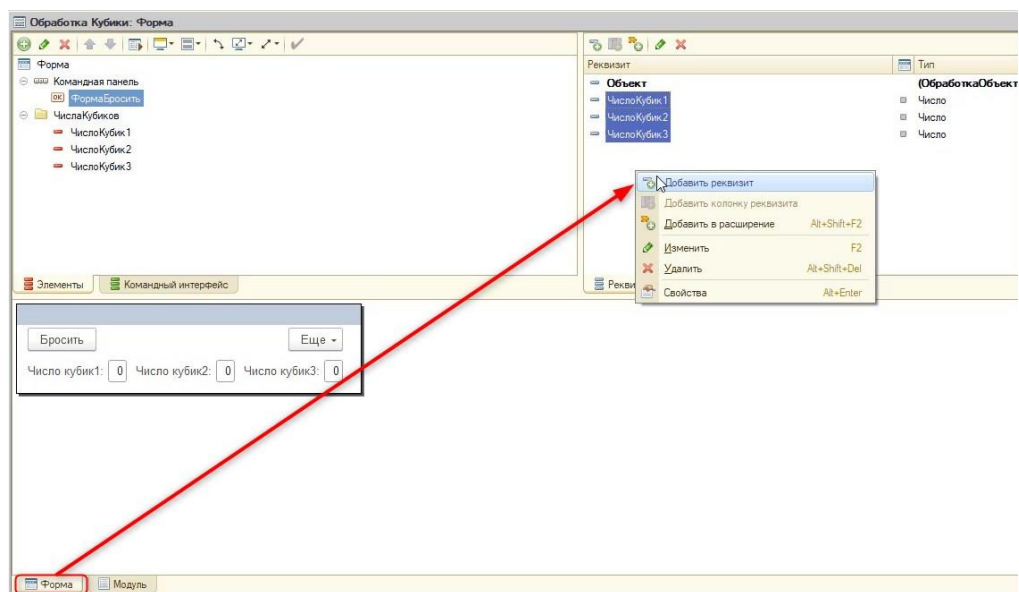


Рис. 1.2.59. Создание нового реквизита

Новому реквизиту дадим название «КоличествоКубиков». Поскольку это количество, укажем соответствующий тип данных – число с длиной в 1 символ, точностью 0 и неотрицательное (рис. 1.2.60).

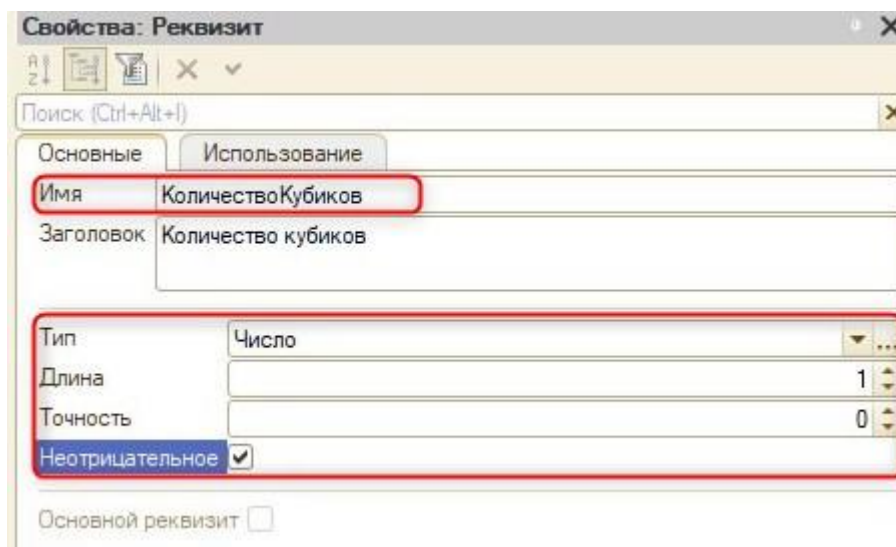


Рис. 1.2.60. Свойства нового реквизита

Перенесём данный реквизит на форму (рис. 1.2.61).

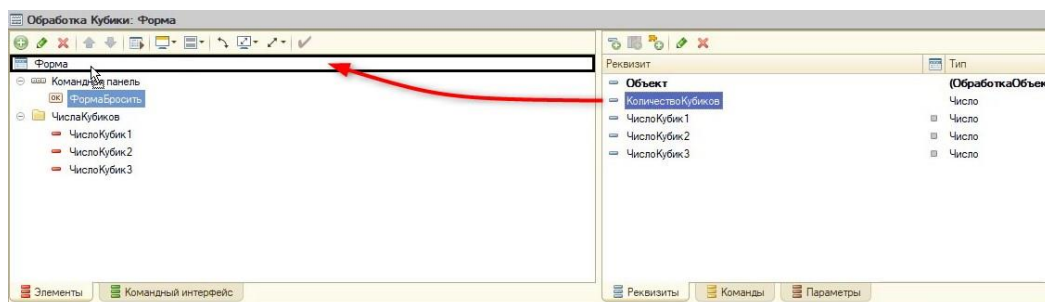


Рис. 1.2.61. Создание связанного с реквизитом элемента

С помощью стрелочек в области элементов передвинем наш элемент наверх (рис. 1.2.62).

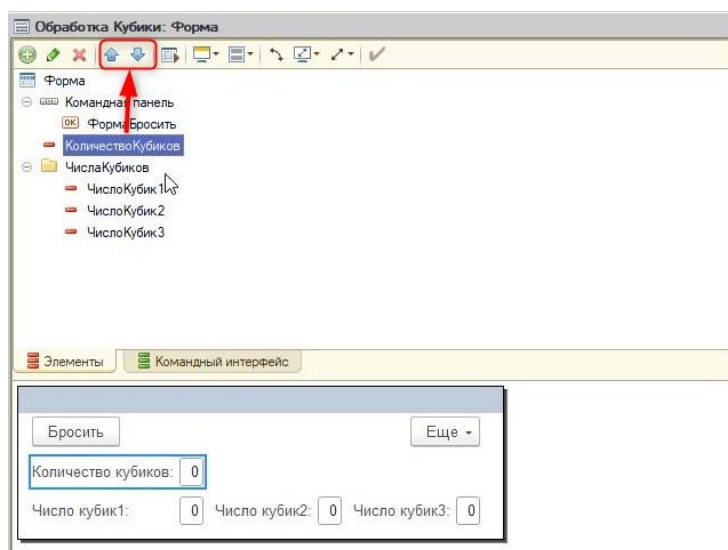


Рис. 1.2.62. Изменение местоположения элемента на форме

В данном случае количество придётся указывать пользователю вручную, но можно сделать так, чтобы у него был ограниченный выбор. Для этого откроем свойства данного элемента, нажав по нему правой кнопкой мыши (рис. 1.2.63).

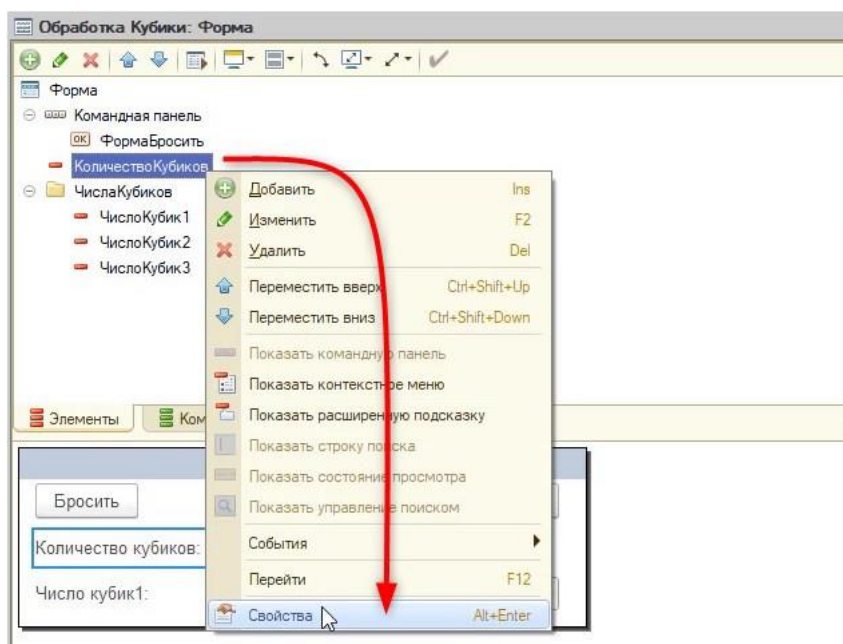


Рис. 1.2.63. Открытие свойств элемента

В палитре свойств перейдём на закладку «Использование», на которой находятся все настройки внешнего вида элемента, и включим флаг «РежимВыбораИзСписка» (рис. 1.2.64).

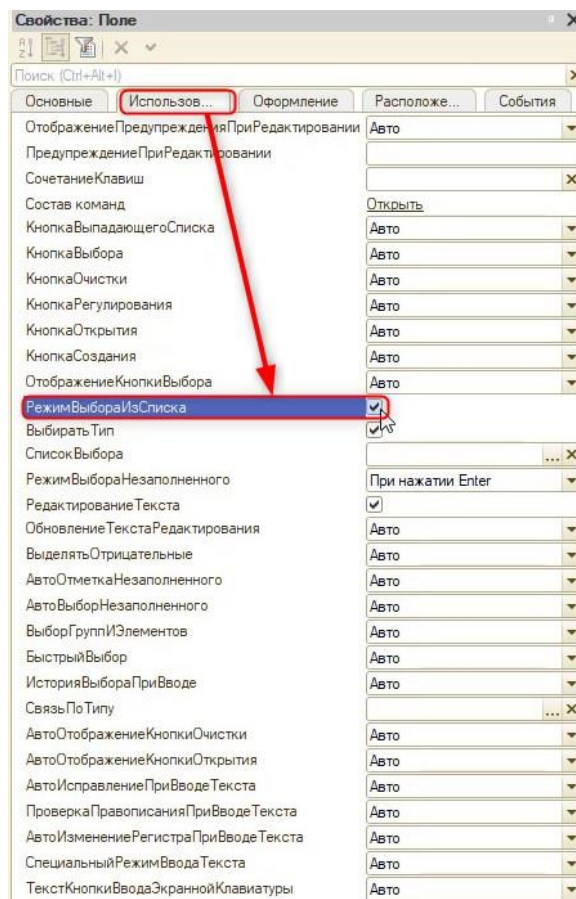


Рис. 1.2.64. Открытие свойств элемента

Так как список всегда фиксированный, его нужно определить (задать значения). Сделать это можно с помощью свойства «СписокВыбора», которое находится ниже (рис. 1.2.65).

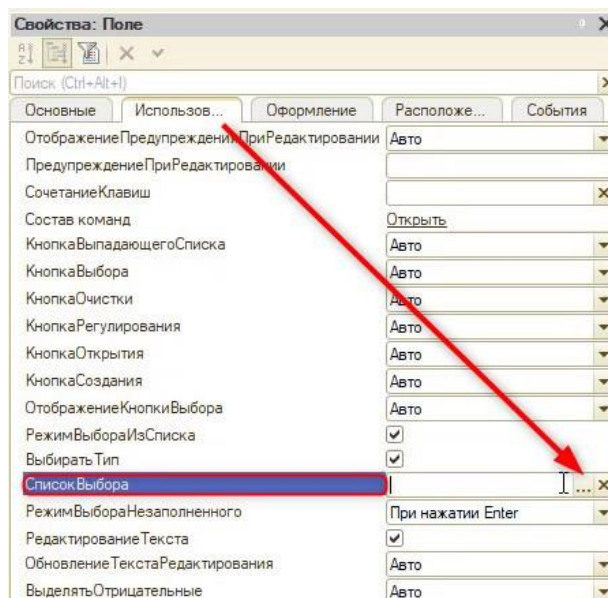


Рис. 1.2.65. Список выбора

После нажатия на троеточие откроется окно со списком выбора. Для того чтобы добавить новое значение в список, нажмём кнопку «Добавить» в виде зелёного плюса (рис. 1.2.66).

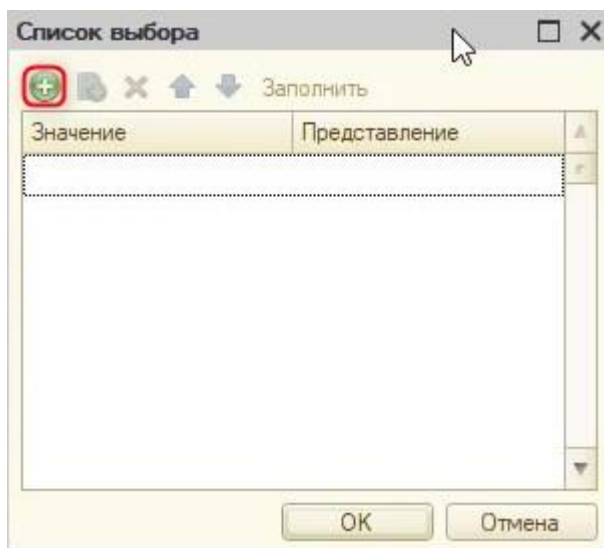


Рис. 1.2.66. Добавление значения в список выбора

Добавим значения «1», «2», «3» и нажмем кнопку «ОК» (рис. 1.2.67).

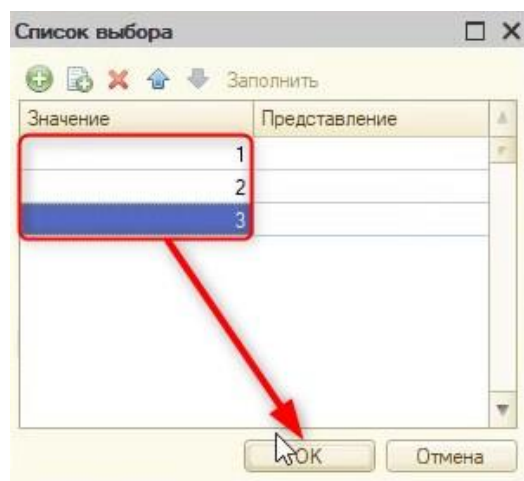


Рис. 1.2.67. Создание значений

В режиме пользователя появится новое поле и выбор из трех значений (рис. 1.2.68). Но такая настройка не будет учитываться при нажатии кнопки «Бросить», так как нужно прописать это дополнительно в коде.

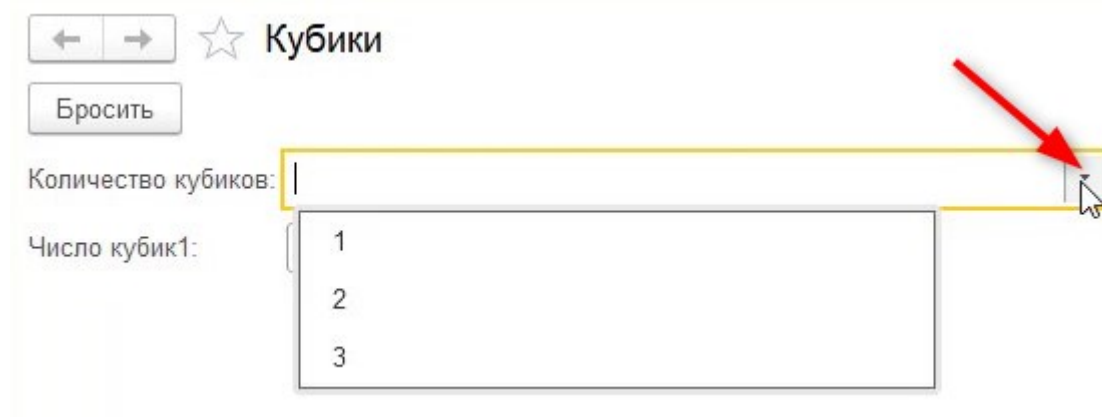


Рис. 1.2.68. Кнопка выбора в пользовательском режиме

Нужно добавить вариативность броску с помощью анализа выбора пользователя. Для этого откроем модуль формы.

Можно модернизировать процедуру, указав, сколько кубиков хочет выбросить пользователь до начала выполнения кода с броском. Для этого у процедуры есть параметры.

Процедура или функция начинается со слова Процедура (Функция). Далее следует Имя процедуры (функции). После имени обязательно указываются круглые скобки. Внутри скобок могут находиться описываемые параметры. Параметры процедуры – это значения, записываемые в специальные локальные переменные процедуры при ее вызове.

Данные параметры нужно будет передавать при вызове (при обращении к процедуре) и получать (указывать сам параметр у процедуры).

Добавим в вызов процедуры и в саму процедуру новый параметр, который будет содержать количество выбранных пользователем кубиков (рис. 1.2.69).

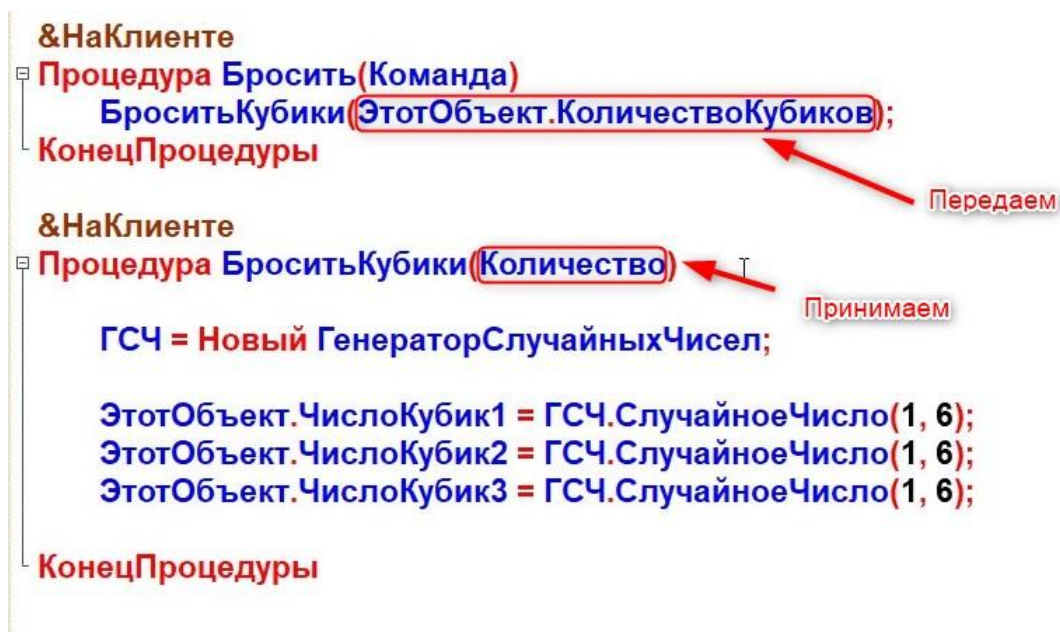


Рис. 1.2.69. Параметр в процедуре

Перед тем как приступить к написанию кода, стоит познакомиться с функцией проверки модуля на ошибки. Она расположена в панели программы (рис. 1.2.70).

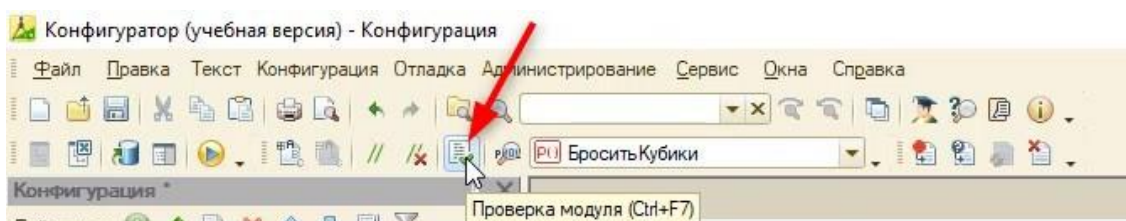


Рис. 1.2.70. Проверка модуля

После проверки программа выдаст надпись «Синтаксических ошибок не обнаружено!» (рис. 1.2.71), либо текст ошибки.

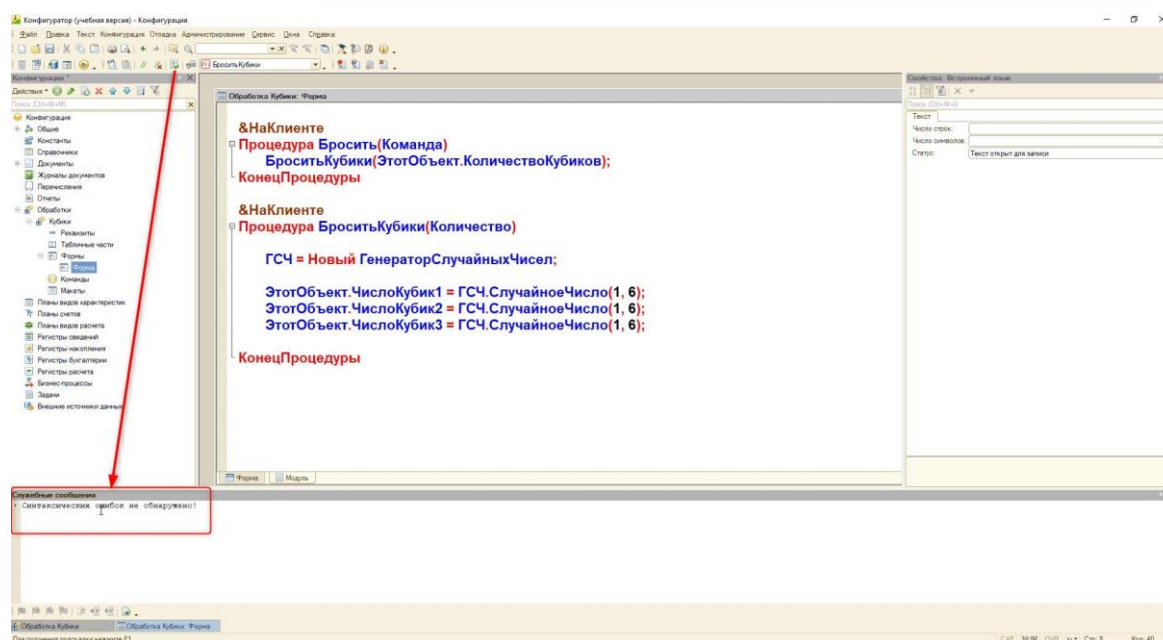


Рис. 1.2.71. Проверка модуля на ошибки

Теперь необходимо в модуле обрабатывать разное количество бросков кубиков, которые выбирает игрок. А также, если он не выбрал количество кубиков, вывести предупреждение. Стоит ещё учитывать, что при броске 1 или 2 кубиков нужно обнулять значения тех, которые не участвуют в броске.

Для обработки разных событий можно использовать условный оператор «Если». Его конструкция выглядит следующим образом:

```
Если <Логическое выражение> Тогда
ИначеЕсли <Логическое выражение> Тогда
...
Иначе
КонецЕсли
```

Исходя из приведенного синтаксиса, можно рассказать об основных особенностях оператора:

1. Логический оператор всегда начинается с ключевой фразы «Если» и заканчивается фразой «КонецЕсли»;
2. Логическое выражение – выражение, результатом которого является значение типа Булево, то есть Истина или Ложь. Если результат выполнения логического выражения — Истина, выполняется блок операторов, идущих после ключевого слова Тогда;
3. В логическом выражении могут быть использованы операции сравнения: «=», «<>» (не равно), «>», «<», «>=» (больше или равно), «<=» (меньше или равно);
4. Также в логическом выражении могут применяться булевы операции: «И», «ИЛИ», «НЕ»;
5. Можно создавать неограниченное количество блоков ИначеЕсли, выражение которых будет проверяться, если выражения предыдущих блоков Если и ИначеЕсли вернули Ложь;
6. Также можно использовать один блок Иначе. Он будет выполняться, если не выполнен ни один из предыдущих блоков Если и ИначеЕсли.

К примеру:

```
Если Число1 = Число2 Тогда
    Сообщить("Числа равны");
Иначе
    Сообщить("Числа не равны");
КонецЕсли;
```

Если применить эту конструкцию, то код процедуры «БроситьКубики» будет выглядеть следующим образом:

```
&НаКлиенте
Процедура БроситьКубики(Количество)
    ГСЧ = Новый ГенераторСлучайныхЧисел;
    //Если количество кубиков 0 - обнуляем значения всех кубиков
    Если Количество = 0 Тогда
        ЭтотОбъект.ЧислоКубик1 = 0;
        ЭтотОбъект.ЧислоКубик2 = 0;
        ЭтотОбъект.ЧислоКубик3 = 0;

        //Выведем предупреждение пользователю:
        //Есть два похожих метода - Предупреждение и ПредупреждениеАсинх
        //Предупреждение - используется в модальном режиме
        //Модальный режим блокирует все окна пользователю, пока не закроется окно с
        предупреждением, такой вид работы окон сейчас не используется

        //ПредупреждениеАсинх - более гибкий и настраиваемый и используется опытными
        специалистами
        //Синтаксис выглядит его следующим образом:
        //ПредупреждениеАсинх(<ТекстПредупреждения>, <Таймаут>, <Заголовок>)
        //Таймаут - Интервал времени в секундах, в течение которого система будет ожидать
        ответа пользователя.

        //По истечении интервала окно предупреждения будет закрыто.
        //Если параметр не указан, то время ожидания не ограничено.
        //В коде значения с типом строки указываются с помощью двойных кавычек - ""
        ПредупреждениеАсинх("Не выбрано количество кубиков!", 5);

        //Бросаем один кубик:
        ИначеЕсли Количество = 1 Тогда
            ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);
            ЭтотОбъект.ЧислоКубик2 = 0;
            ЭтотОбъект.ЧислоКубик3 = 0;

            //Бросаем 2 кубика:
            ИначеЕсли Количество = 2 Тогда
                ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);
                ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);
                ЭтотОбъект.ЧислоКубик3 = 0;
```

//Бросаем 3 кубика:

ИначеЕсли Количество = 3 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик3 = ГСЧ.СлучайноеЧисло(1, 6);

//Обязательно не забываем про завершение оператора "Если":

КонецЕсли;

КонецПроцедуры

У программистов есть одно основное правило – придерживаться чистоты кода. Чистый код – это код, который не доставляет никому проблем: ни разработчику, ни компьютеру, ни пользователям программы. Его легко читать (к примеру, нет слипшихся строк), и в нем может разобраться другой разработчик.

Добиться чистоты кода можно и начинающему разработчику: для это есть штатный механизм платформы – форматирование. Находится оно в пункте главного меню «Текст» – «Блок» – «Форматировать» (рис. 1.2.72). Перед его использованием нужно выделить нужную часть кода.

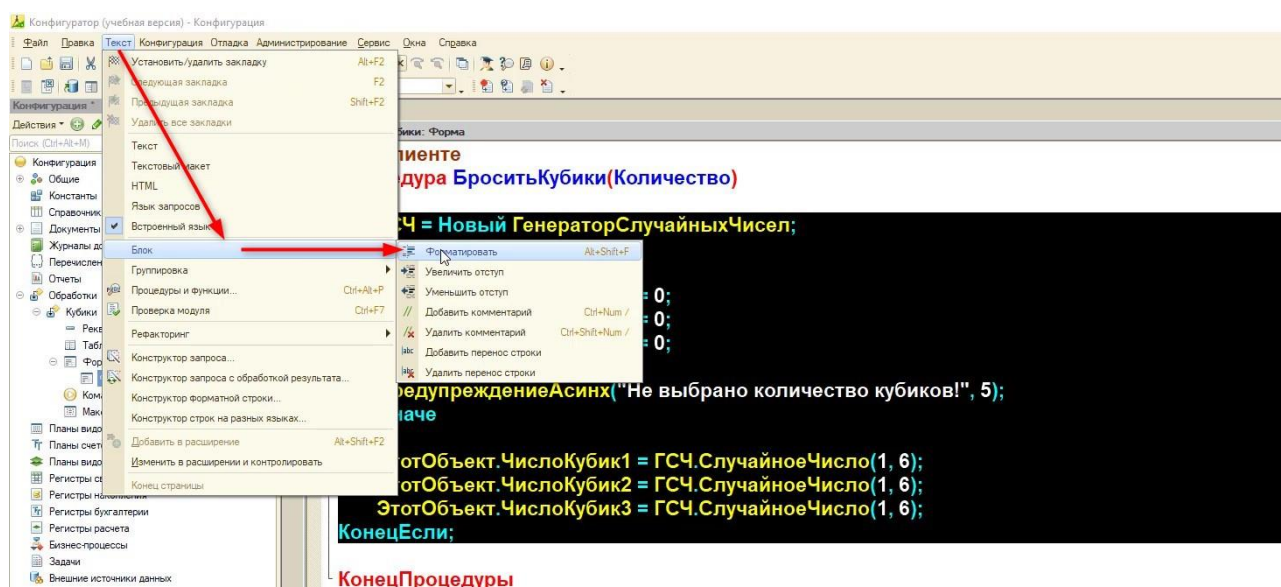


Рис. 1.2.72. Форматирование кода

Теперь пользователь может бросать разное количество кубиков, и код выглядит чистым!

Усложним задачу и добавим подсчёт суммы выброшенных значений кубиков. Для начала нужно добавить новый реквизит «СуммаЗначений» и указать для него тип число с длиной 2 символа и неотрицательное (рис. 1.2.73).

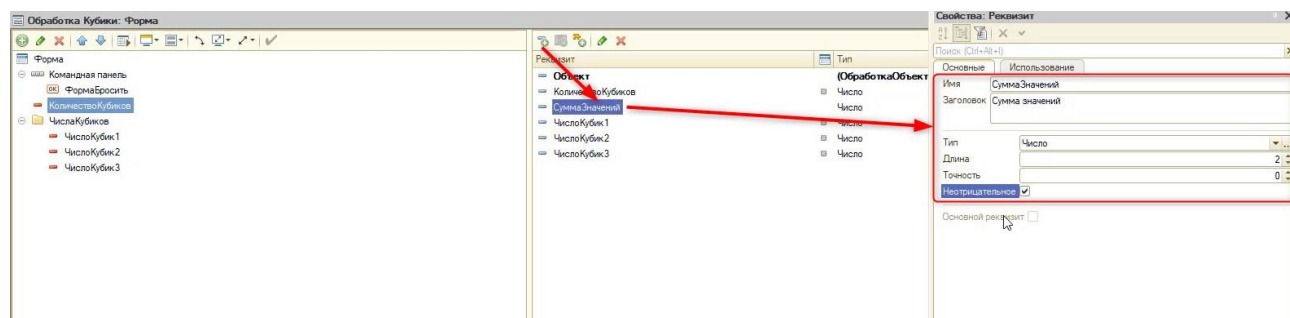


Рис. 1.2.72. Форматирование кода

Перенесём реквизит на форму, как делали ранее. Для удобства с помощью стрелочек «вверх» и «вниз» расположим сумму повыше (рис. 1.2.73).

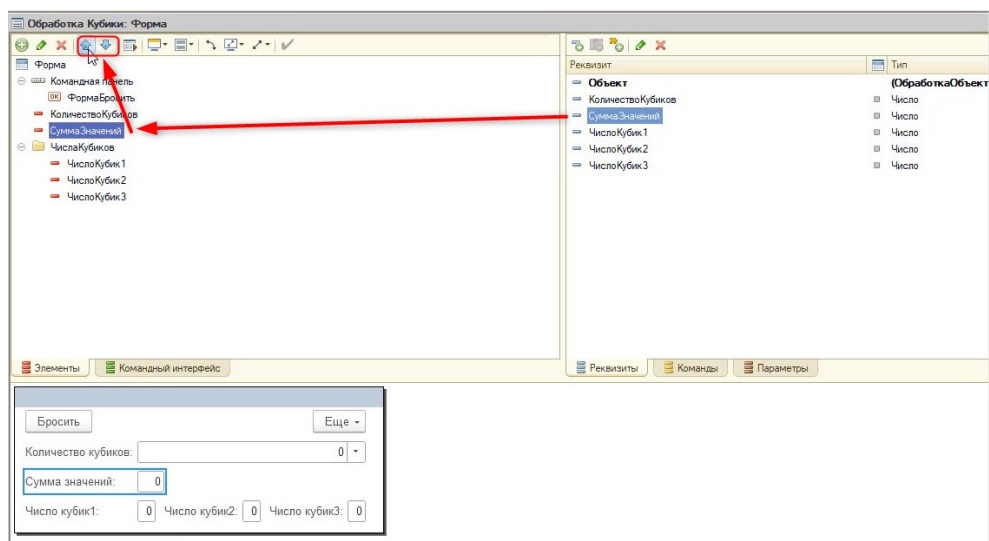


Рис. 1.2.73. Добавление суммы на форму

Игрок не должен влиять на сумму значений броска. Для этого настроим вид «Поле надписи» в свойствах элемента (рис. 1.2.74).

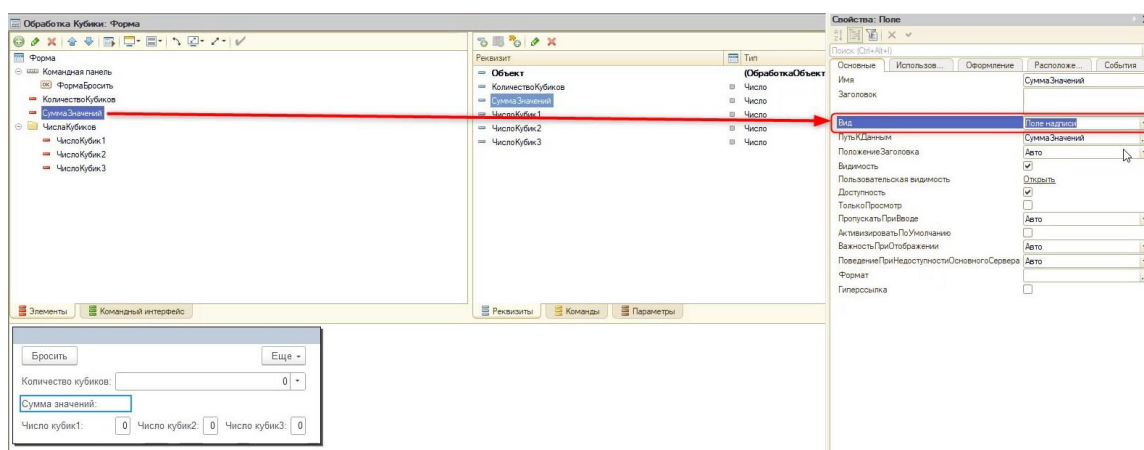


Рис. 1.2.74. Смена вида элемента

Если в данный момент проверить работу игры, подсчёт происходить не будет. Для этого нужно программировать.

Вернёмся в модуль формы и допишем после обработки бросков кубиков расчет суммы:

&НаКлиенте

Процедура БроситьКубики(Количество)

ГСЧ = Новый ГенераторСлучайныхЧисел;

Если Количество = 0 Тогда

ЭтотОбъект.ЧислоКубик1 = 0;

ЭтотОбъект.ЧислоКубик2 = 0;

ЭтотОбъект.ЧислоКубик3 = 0;

ПредупреждениеАсинх("Не выбрано количество кубиков!", 5);

ИначеЕсли Количество = 1 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = 0;


```

ЭтотОбъект.ЧислоКубик3 = 0;
ИначеЕсли Количество = 2 Тогда
    ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);
    ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);
    ЭтотОбъект.ЧислоКубик3 = 0;
ИначеЕсли Количество = 3 Тогда
    ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);
    ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);
    ЭтотОбъект.ЧислоКубик3 = ГСЧ.СлучайноеЧисло(1, 6);
КонецЕсли;
//После броска посчитаем сумму значений:
ЭтотОбъект.СуммаЗначений = ЭтотОбъект.ЧислоКубик1 + ЭтотОбъект.ЧислоКубик2 +
ЭтотОбъект.ЧислоКубик3;
КонецПроцедуры

```

Теперь после каждого броска у пользователя будет происходить подсчёт суммы (рис. 1.2.75).

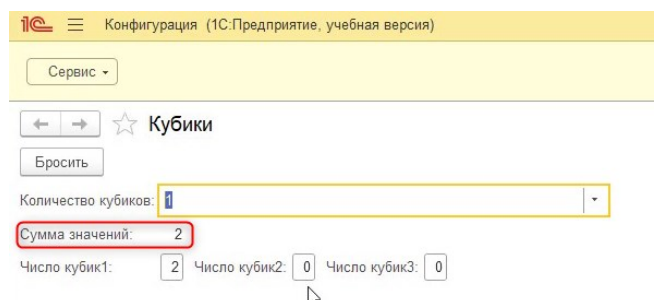


Рис. 1.2.75. Сумма значений в пользовательском режиме

Следующим этапом реализуем вывод картинок для наших кубиков.

Для начала создадим три реквизита, в которых будет находиться изображение – «КартинкаКубика1», «КартинкаКубика2», «КартинкаКубика3». Тип у этих реквизитов должен быть «Картинка».

Типов данных в 1С большое множество, поэтому в 1С есть поиск по названию. Открыть его можно, нажав на кнопку выбора рядом с свойством «Тип». В открывшемся окне через поиск нужно найти необходимый тип данных (рис. 1.2.76).

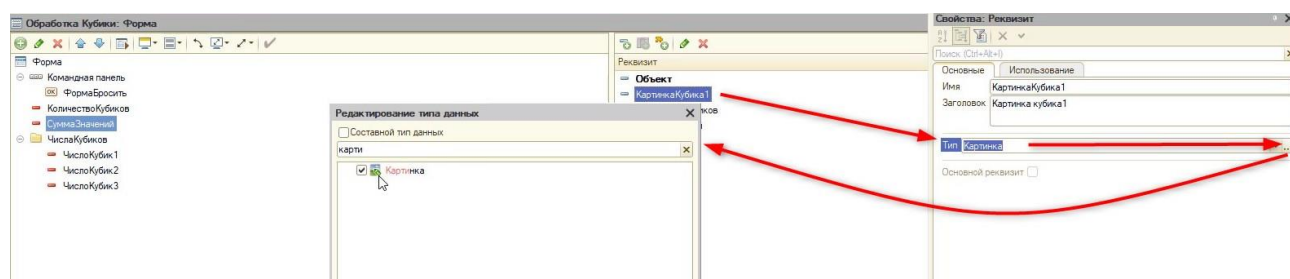


Рис. 1.2.76. Поиск определенного типа данных

Перенесем все три созданных реквизита на форму (рис. 1.2.77).

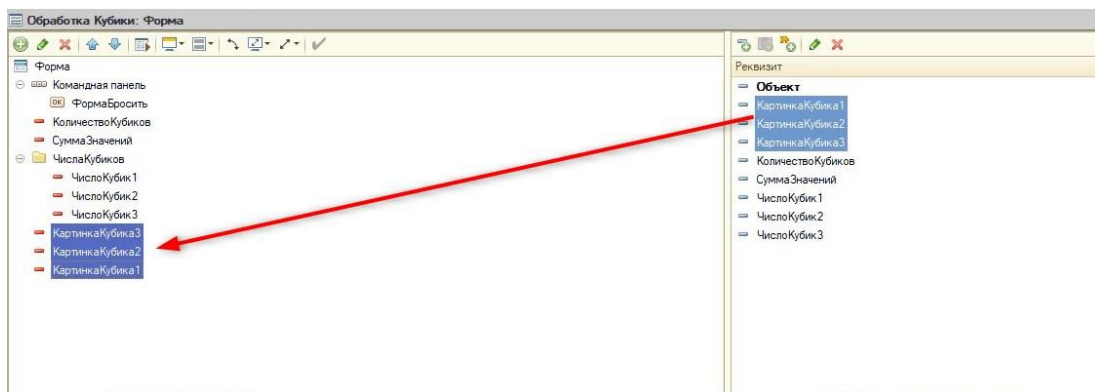


Рис. 1.2.77. Создание элементов для отображения картинок

После переноса картинки расположились друг под другом, но в примере ранее изображения располагались по горизонтали. Поменять это несложно.

Добавим группу «КартинкиКубиков» без отображения и перенесём в неё все картинки. Для того чтобы отображение всегда было горизонтальным, укажем в свойствах группы группировку – «Горизонтальная всегда» (рис. 1.2.78).

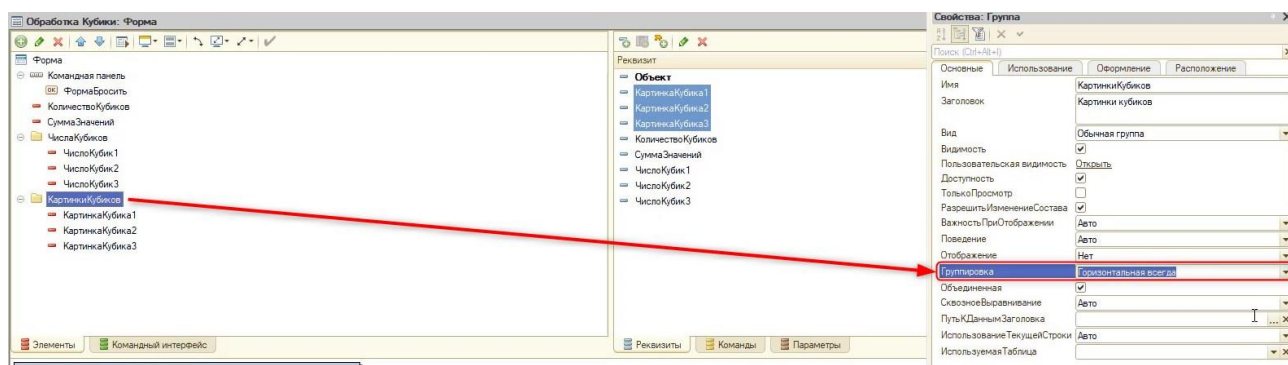


Рис. 1.2.78. Смена группировки

Хранить картинки будем непосредственно на форме. Перед этим создадим группу без отображения с названием «ГраниКубиков». Выключим её видимость (рис. 1.2.79), так как пользователю не нужно видеть сразу все грани кубов – это нужно только нам, разработчикам, чтобы впоследствии эти картинки подставлять в соответствующие элементы.

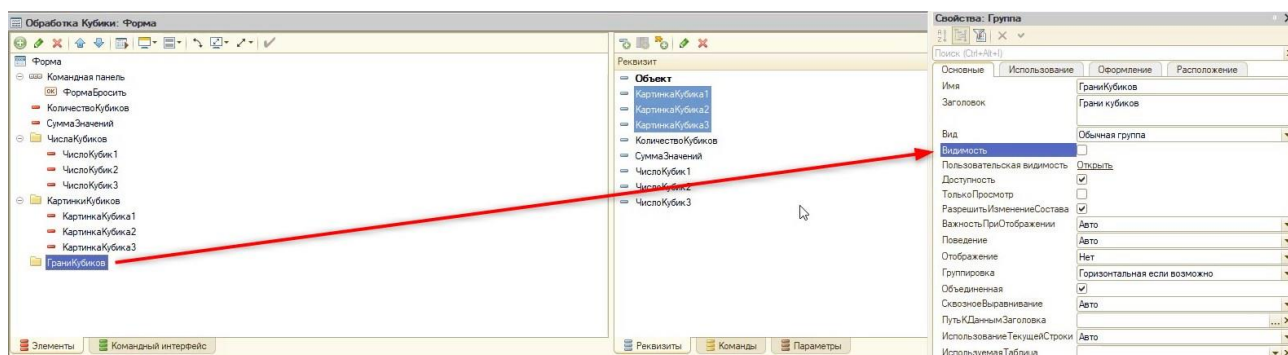


Рис. 1.2.79. Выключение видимости у группы

В созданную группу мы добавим 6 изображений граней. Для добавления картинки необходимо нажать на кнопку «Добавить» в панели элементов конструктора и выбрать «Декорация – Картина» (рис. 1.2.80).

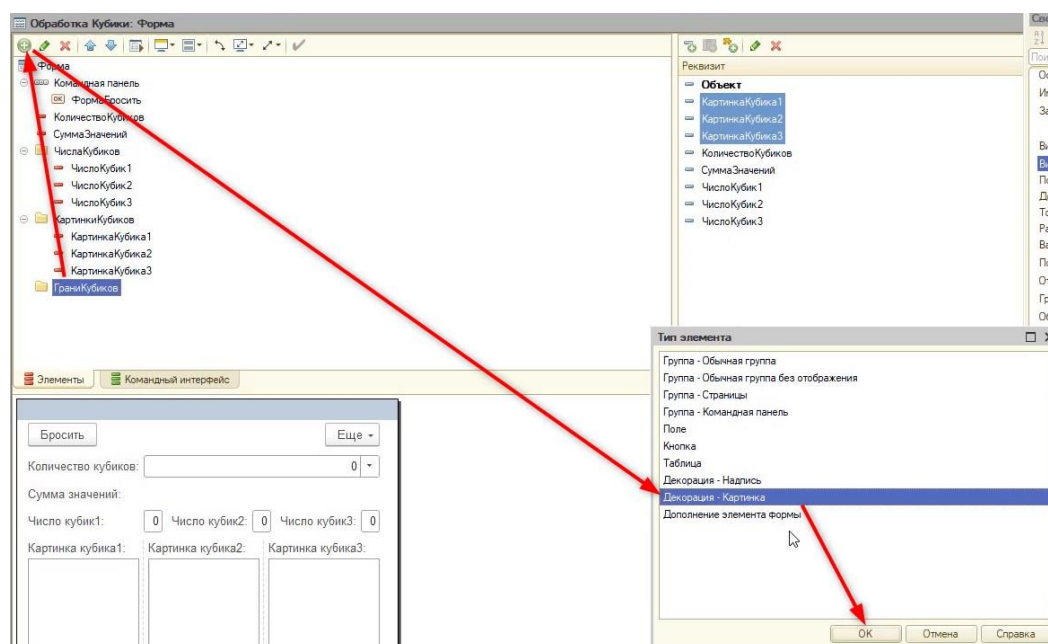


Рис. 1.2.80. Добавление картинки

Новую декорацию назовём «Грани1» и в свойстве «Картинка» нажмём кнопку выбора (рис. 1.2.81).

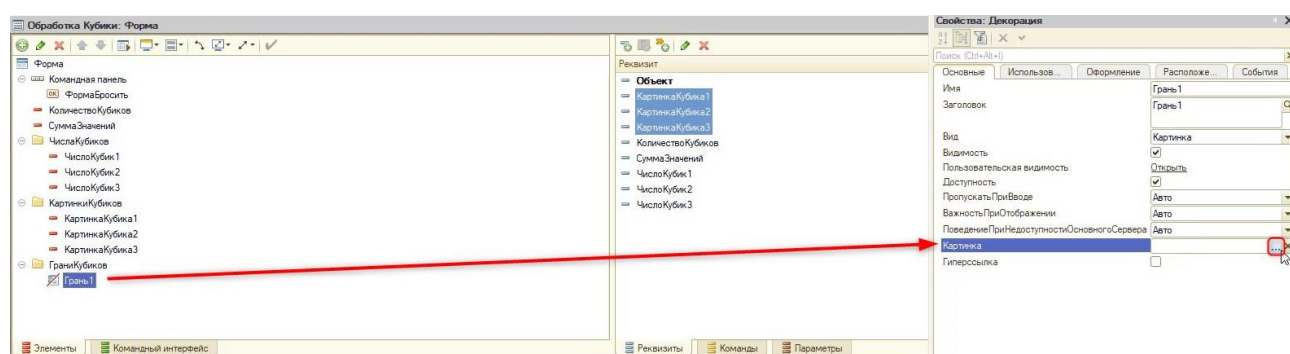


Рис. 1.2.81. Выбор картинки

Предварительно нужно скачать и сохранить все нужные грани на устройство. В открывшемся окне нужно выбрать вкладку «Из файла» и нажать кнопку «Выбрать файл» (рис. 1.2.82). После выбора картинки нужно нажать кнопку «Ок».

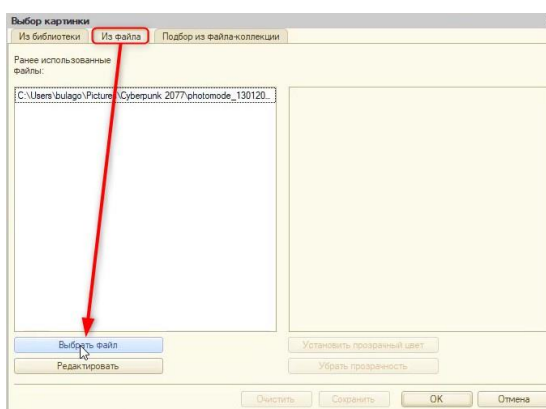


Рис. 1.2.82. Выбор файла картинки

Аналогично поступим с оставшимися 5 картинками граней. У нас получится следующая картина (рис. 1.2.83).

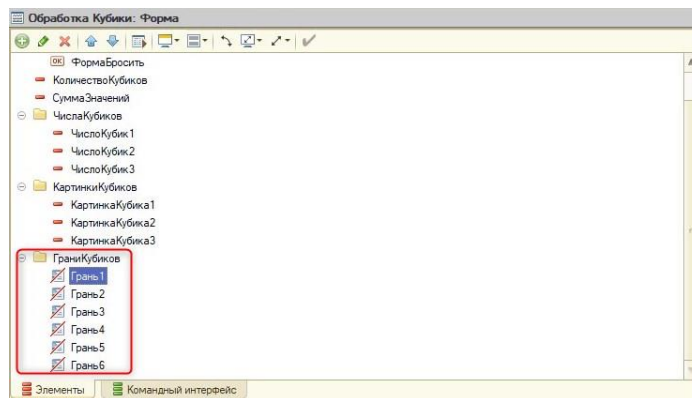


Рис. 1.2.83. Грани кубиков

Создадим алгоритм, который будет подставлять нужные картинки граней при броске. Для этого вернёмся в модуль и доработаем наши условия в процедуре «БроситьКубики»:

&НаКлиенте

Процедура БроситьКубики(Количество)

ГСЧ = Новый ГенераторСлучайныхЧисел;

Если Количество = 0 Тогда

ЭтотОбъект.ЧислоКубик1 = 0;

ЭтотОбъект.ЧислоКубик2 = 0;

ЭтотОбъект.ЧислоКубик3 = 0;

//Так как количество кубиков не задано, скроем все картинки

// с помощью свойства элемента "Видимость":

// Элементы - обращение ко всем элементам на форме

// КартинкаКубика - обращение к конкретному элементу

// Истина и Ложь – это тип булево, он может иметь только два значения и записывается без кавычек.

Элементы.КартинкаКубика1.Видимость = Ложь;

Элементы.КартинкаКубика2.Видимость = Ложь;

Элементы.КартинкаКубика3.Видимость = Ложь;

ПредупреждениеАсинх("Не выбрано количество кубиков!", 5);

ИначеЕсли Количество = 1 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = 0;

ЭтотОбъект.ЧислоКубик3 = 0;

//Скрываем видимость у двух кубиков:

Элементы.КартинкаКубика1.Видимость = Истина;

Элементы.КартинкаКубика2.Видимость = Ложь;

Элементы.КартинкаКубика3.Видимость = Ложь;

ИначеЕсли Количество = 2 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик3 = 0;

//Скрываем видимость у трех кубиков:

Элементы.КартинкаКубика1.Видимость = Истина;

Элементы.КартинкаКубика2.Видимость = Истина;

Элементы.КартинкаКубика3.Видимость = Ложь;

ИначеЕсли Количество = 3 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик3 = ГСЧ.СлучайноеЧисло(1, 6);

//Показываем пользователю все кубики:

Элементы.КартинкаКубика1.Видимость = Истина;

Элементы.КартинкаКубика2.Видимость = Истина;

Элементы.КартинкаКубика3.Видимость = Истина;

КонецЕсли;

ЭтотОбъект.СуммаЗначений = ЭтотОбъект.ЧислоКубик1 + ЭтотОбъект.ЧислоКубик2 + ЭтотОбъект.ЧислоКубик3;

КонецПроцедуры

Перед проверкой кода уберём заранее видимость всех картинок кубиков, чтобы у пользователя не было пустых окон (рис. 1.2.84).

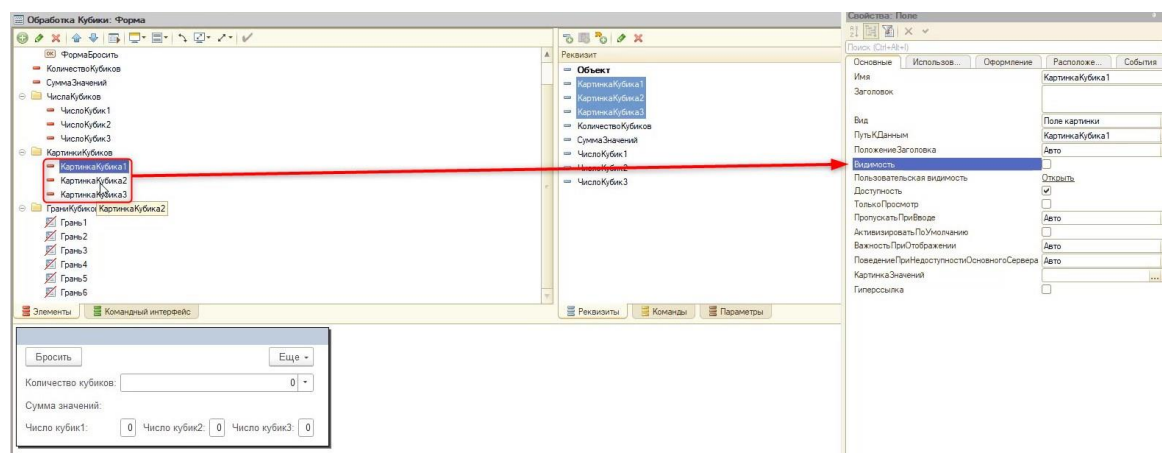


Рис. 1.2.84. Выключение видимости у элементов

В данный момент в пользовательском режиме очень большие окна для будущих картинок граней (рис. 1.2.85).

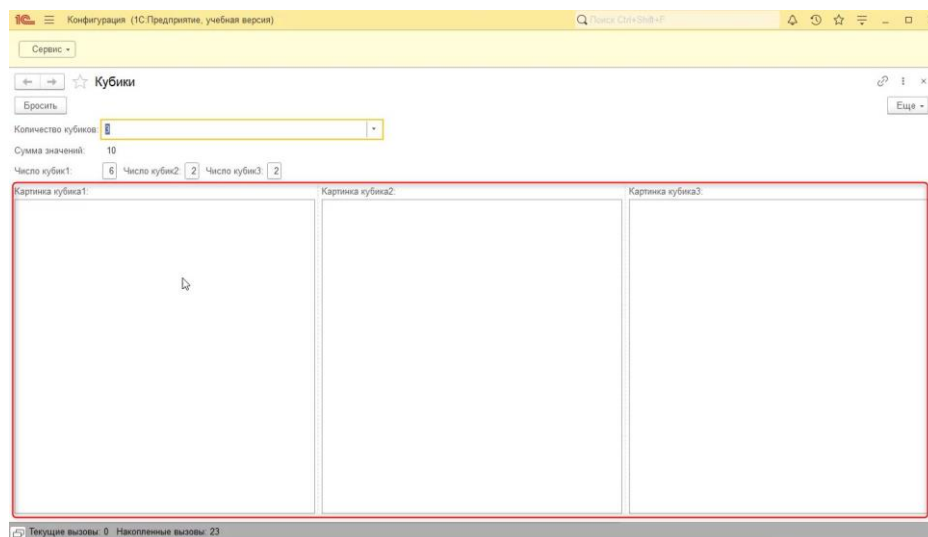


Рис. 1.2.85. Результат в пользовательском режиме

Это можно исправить, выбрав в свойствах картинок «Размер картинки» – «Пропорционально» на закладке «Оформление» (рис. 1.2.86). Для удобства можно выделить сразу все элементы с картинкой с помощью клавиши «Ctrl».

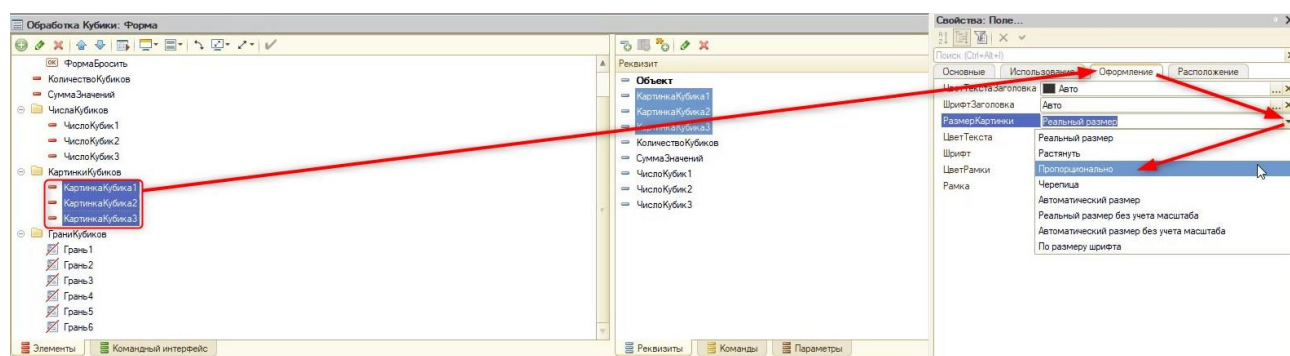


Рис. 1.2.86. Смена размера картинки

Дополнительно нужно выключить на вкладке «Расположение» свойства «РастягиватьПоГоризонтали» и «РастягиватьПоВертикали» (рис. 1.2.87).

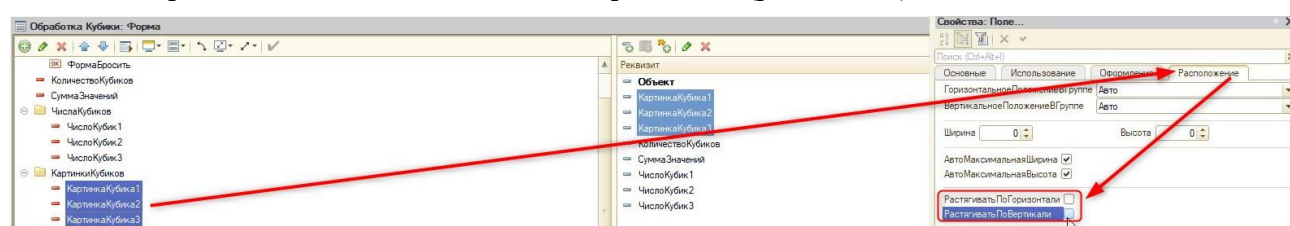


Рис. 1.2.87. Выключение растягивания

Таким образом картинки граней не будут растягиваться на весь экран.

Научим алгоритм подставлять картинки граней, для этого перейдём в модуль формы. Так как картинку подставлять нужно будет не один раз, создадим отдельную функцию «ПолучитьКартинку».

Важно!

Функция, в отличие от процедуры, может иметь возвращаемое значение. Т. е. получать определенный результат и передавать его в процедуру или функцию, где была вызвана.

Функция создается так же, как и процедура. В конце модуля нужно написать «функ» и нажать горячую клавишу «Ctrl + Q» (рис. 1.2.88). Во всплывающем окне нужно выбрать «Функция».

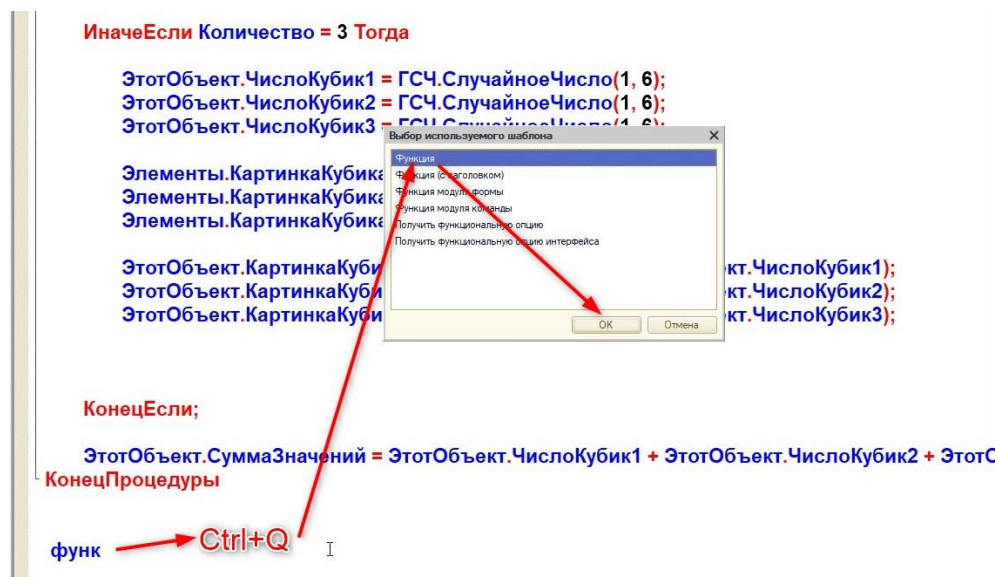


Рис. 1.2.88. Создание функции

Обязательно добавим директиву «НаКлиенте» (рис. 1.2.89).

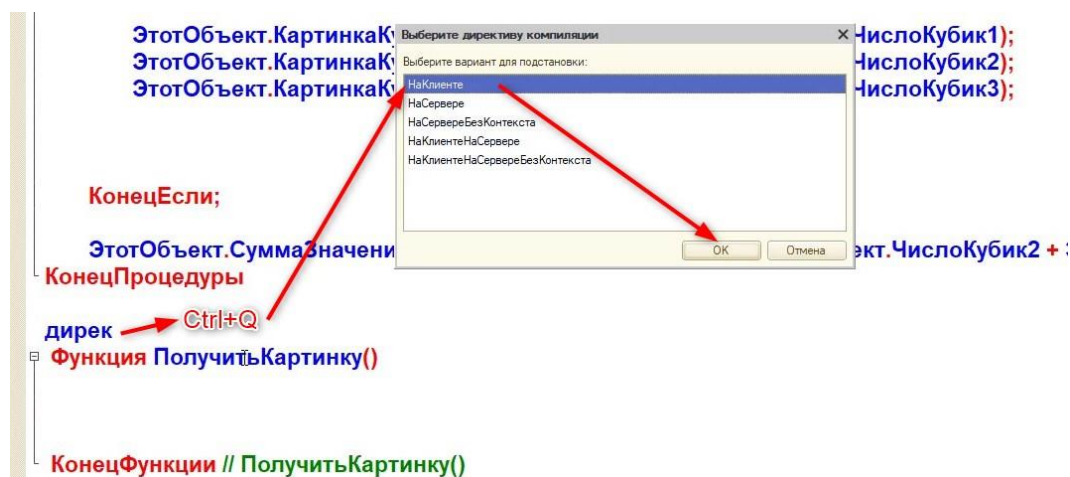


Рис. 1.2.89. Добавление директивы

Подбор картинки происходит по номеру, поэтому в функцию нам нужно передать этот номер, чтобы определить для него соответствующую грань. Для этого добавим параметр «Число» новой функции (рис. 1.2.90).

```
&НаКлиенте
Функция ПолучитьКартинку(Число)
    КонецФункции // ПолучитьКартинку()
```

Рис. 1.2.90. Добавление параметра

Код функции будет выглядеть следующим образом:

```
&НаКлиенте
Функция ПолучитьКартинку(Число)
```



```
//Грань - фиксированное название, которое мы указали для всех декораций
//Число - передаваемое выпавшее число (1-6)
ИмяКартинки = "Грань" + Число;

//Вернём полученную картинку (возврат доступен только функции):
//"" - операторы, которые позволяют анализировать массивы, таблицы, списки
//Кртинка - свойство элемента, где хранится нужная картинка грани
Возврат Элементы[ИмяКартинки].Картинка;
КонецФункции // ПолучитьКартинку()
```

Теперь подкорректируем код процедуры «БроситьКубики»:

&НаКлиенте

Процедура БроситьКубики(Количество)

ГСЧ = Новый ГенераторСлучайныхЧисел;

Если Количество = 0 Тогда

ЭтотОбъект.ЧислоКубик1 = 0;

ЭтотОбъект.ЧислоКубик2 = 0;

ЭтотОбъект.ЧислоКубик3 = 0;

Элементы.КартинкаКубика1.Видимость = Ложь;

Элементы.КартинкаКубика2.Видимость = Ложь;

Элементы.КартинкаКубика3.Видимость = Ложь;

ПредупреждениеАсинх("Не выбрано количество кубиков!", 5);

ИначеЕсли Количество = 1 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = 0;

ЭтотОбъект.ЧислоКубик3 = 0;

Элементы.КартинкаКубика1.Видимость = Истина;

Элементы.КартинкаКубика2.Видимость = Ложь;

Элементы.КартинкаКубика3.Видимость = Ложь;

//Полученную картинку грани из функции присваеваем элементу "КартинкаКубика1" для отображения пользователю:

ЭтотОбъект.КартинкаКубика1 = ПолучитьКартинку(ЭтотОбъект.ЧислоКубик1);

ИначеЕсли Количество = 2 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик3 = 0;

Элементы.КартинкаКубика1.Видимость = Истина;

Элементы.КартинкаКубика2.Видимость = Истина;

Элементы.КартинкаКубика3.Видимость = Ложь;

ЭтотОбъект.КартинкаКубика1 = ПолучитьКартинку(ЭтотОбъект.ЧислоКубик1);

ЭтотОбъект.КартинкаКубика2 = ПолучитьКартинку(ЭтотОбъект.ЧислоКубик2);

ИначеЕсли Количество = 3 Тогда

ЭтотОбъект.ЧислоКубик1 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик2 = ГСЧ.СлучайноеЧисло(1, 6);

ЭтотОбъект.ЧислоКубик3 = ГСЧ.СлучайноеЧисло(1, 6);

Элементы.КартинкаКубика1.Видимость = Истина;

Элементы.КартинкаКубика2.Видимость = Истина;

Элементы.КартинкаКубика3.Видимость = Истина;

ЭтотОбъект.КартинкаКубика1 = ПолучитьКартинку(ЭтотОбъект.ЧислоКубик1);

ЭтотОбъект.КартинкаКубика2 = ПолучитьКартинку(ЭтотОбъект.ЧислоКубик2);

ЭтотОбъект.КартинкаКубика3 = ПолучитьКартинку(ЭтотОбъект.ЧислоКубик3);

КонецЕсли;

ЭтотОбъект.СуммаЗначений = ЭтотОбъект.ЧислоКубик1 + ЭтотОбъект.ЧислоКубик2 + ЭтотОбъект.ЧислоКубик3;

КонецПроцедуры

Отлично! За занятие мы смогли разработать мини-игру! Осталось только отключить пользователю видимость чисел кубиков (рис. 1.2.91) и спрятать заголовки у картинок (рис. 1.2.92).

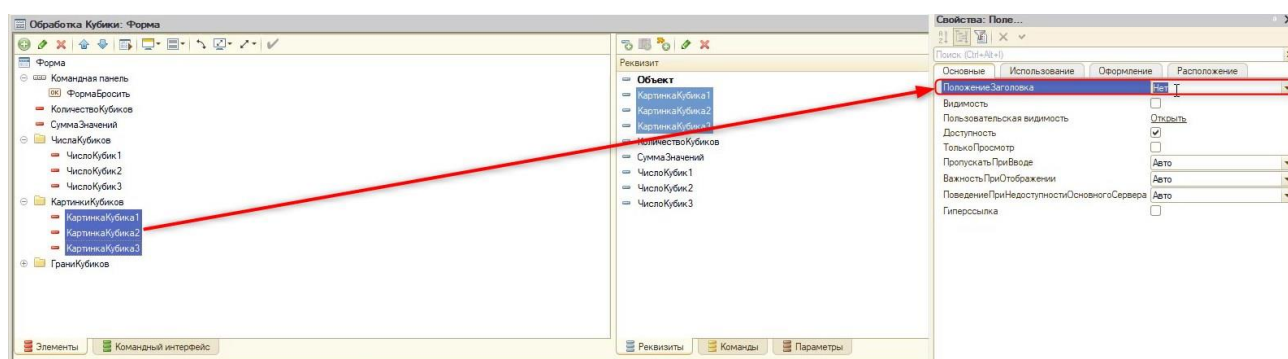


Рис. 1.2.91. Отключение видимости чисел кубиков

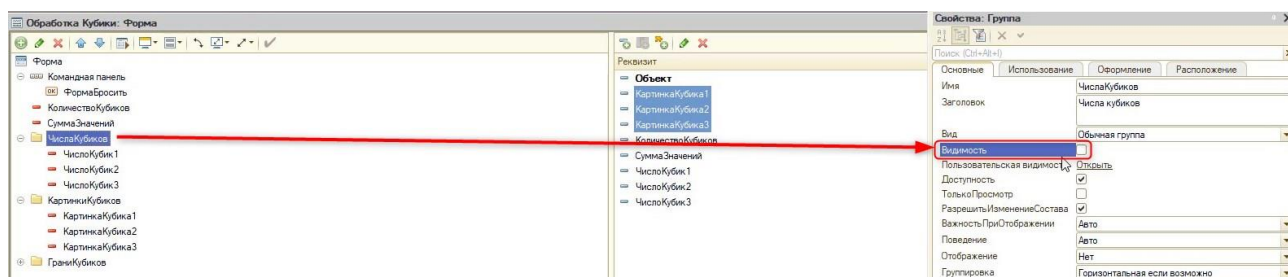


Рис. 1.2.92. Отключение заголовка у декорации

Занятие 3. Игра в три кубика и кнопка «Рестарт»

Резюмируем предыдущее занятие. На данный момент у нас есть информационная база с компьютерной игрой, которая использует механику с бросками кубиков. Теперь предстоит добавить игру в три кубика, а также реализовать кнопку с рестартом игры.

Откроем информационную базу с игрой в режиме разработчика (конфигуратора) и перейдем к форме обработки «Кубики». На вкладке «Форма» в первом занятии определялись элементы, реквизиты и внешний вид формы. А на вкладке «Модуль» был реализован программный код с бросками кубиков.

Откроем модуль формы. В данной обработке планируется сразу несколько разных игр с механикой кубиков, поэтому нужно структурировать код и сделать его более «чистым». Для этого в платформе есть механизм, который разделяет модуль на составляющие – области.

Для того чтобы определить «Область», нужно воспользоваться следующим шаблоном:

```
#Область ИмяОбласти
// Код процедур и функций
#КонецОбласти
```

Задать область можно ещё с помощью текста «#Обл» и сочетания горячих клавиш «Ctrl+Q» (рис. 1.3.1).

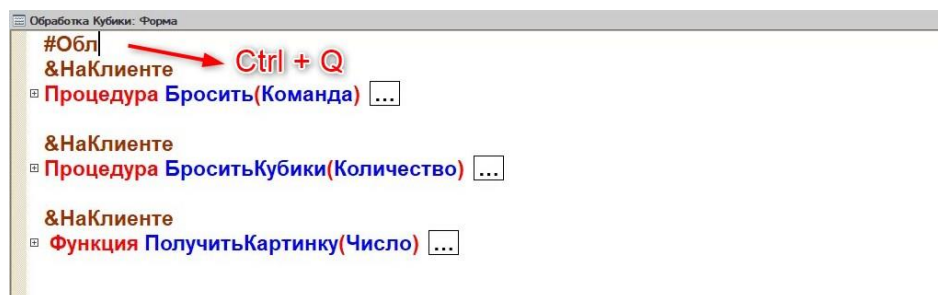


Рис. 1.3.1. Создание области

Создадим новую область и назовём её «Кубики». В неё необходимо перенести сразу три процедуры и функции, созданные ранее. Тогда в модуле формы код будет выглядеть следующим образом (рис. 1.3.2).

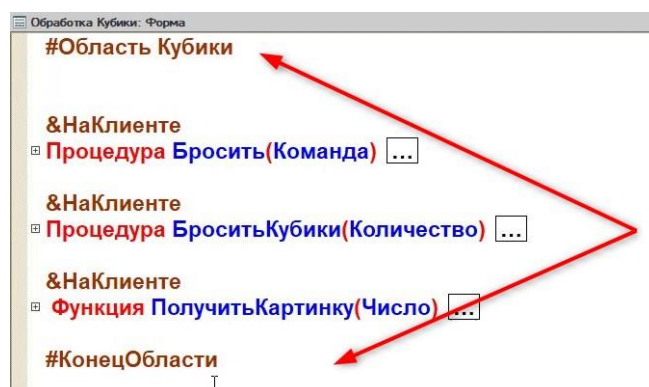


Рис. 1.3.2. Структурирование кода

Если сохранить все изменения и открыть модуль заново, область можно будет свернуть с помощью специальной кнопки «<-» (рис. 1.3.3).

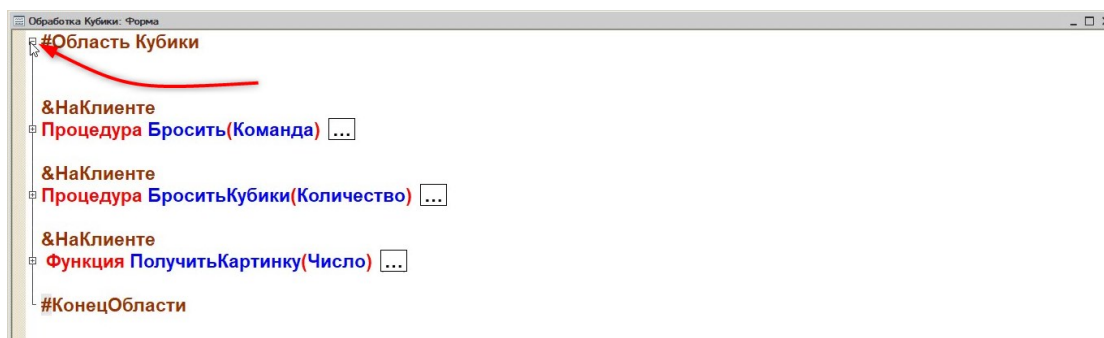


Рис. 1.3.3. Сворачивание области

Так разработчик не будет путаться в строчках программного кода, которые были написаны ранее.

Следующее, что необходимо реализовать – это игра в три кубика. В данной игре игрок загадывает два числа, бросает три кубика и, если сумма значений броска совпадает с одним из загаданных чисел, то пользователь побеждает.

Для этого подготовим новую команду – «Три кубика» и перенесём её в командную панель, чтобы она отобразилась у пользователя (рис. 1.3.4).

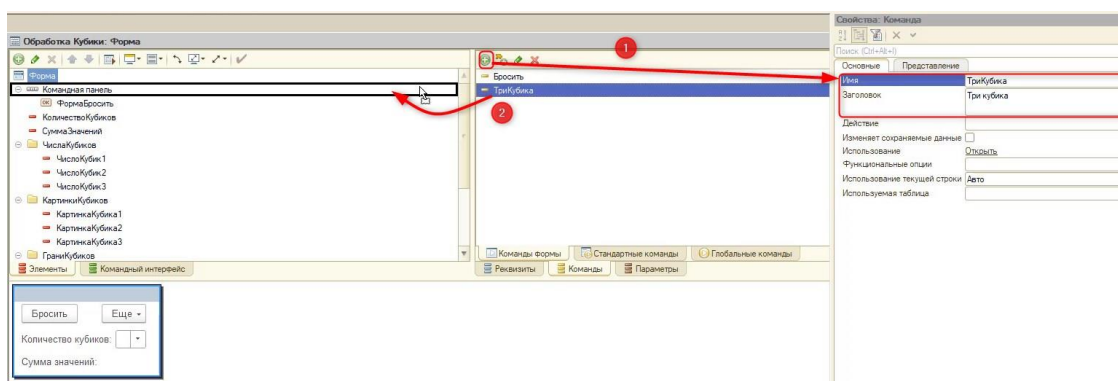


Рис. 1.3.4. Создание новой кнопки

Теперь на форме отобразилась соответствующая кнопка, но при этом при нажатии на нее ничего происходить не будет. Для этого нужно создать действие и написать код. Задать действие можно в свойствах команды (рис. 1.3.5).

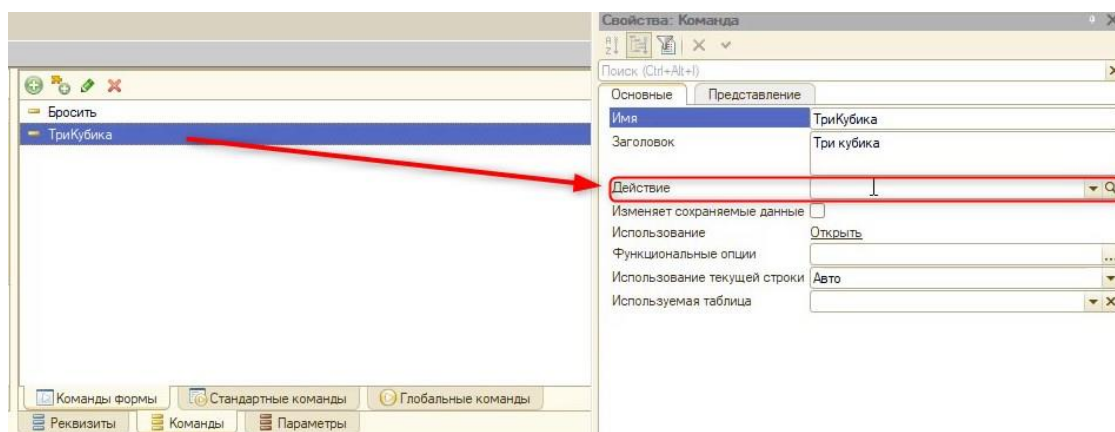


Рис. 1.3.5. Создание действия

Если нажать на кнопку в виде «Лупы», появится окно с выбором создания обработчика команды. Как и ранее оставим активным «Создать на клиенте» и нажмём кнопку «ОК» (рис. 1.3.6).

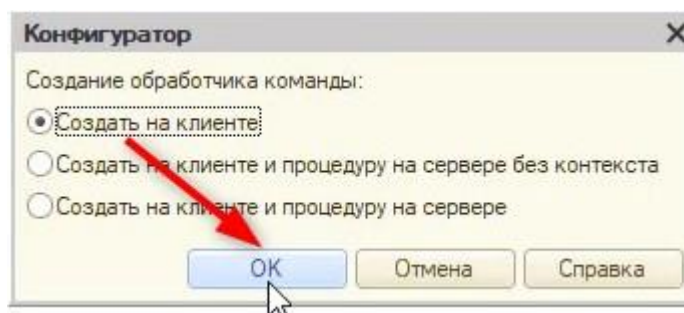


Рис. 1.3.5. Создание действия

Появится обработчик, который нужно вынести в новую область под названием «ТриКубика» (рис. 1.3.6).

```
#Область ТриКубика
&НаКлиенте
Процедура ТриКубика(Команда)
    // Вставить содержимое обработчика.
КонецПроцедуры
#КонецОбласти
```

Рис. 1.3.6. Код области «ТриКубика»

Как было сказано ранее, перед началом игрок загадывает два числа. С точки зрения программирования нужно подготовить две переменные, которые будут их хранить:

ЗагаданноеЧисло1 = 0; ЗагаданноеЧисло2 = 0;
--

Далее перед пользователем должно появиться окно с вводом загаданного числа, и то, что он введёт, должно быть сохранено в новых переменных. Эти переменные в дальнейшем можно будет проанализировать и понять, выиграл игрок или нет.

Для того чтобы пользователь мог интерактивно ввести число, необходимо использовать методы, которые называются «ВвестиЧисло()» или «ВвестиЧислоАсинх()». Ранее мы уже использовали асинхронные методы («Асинх»), продолжим с ними работать, так как они являются более современными и удобными.

Синтаксис асинхронного метода выглядит следующим образом:

ВвестиЧислоАсинх(<Число>, <Подсказка>, <Длина>, <Точность>)

Метод ожидает параметр «Число» – в эту переменную будет помещено введенное число. Поместим выбор пользователя в созданную переменную – «ЗагаданноеЧисло1». Начальное значение переменной будет использовано в качестве начального значения в диалоге (в нашем случае – 0).

После этого нужно указать параметр «Подсказка» – это то, что будет выведено в окне перед игроком. Так как будет происходить ввод числа, напомним игроку следующее: «Введите первое загаданное число».

Третий параметр – «Длина», он означает, какой длины число может ввести пользователь. В нашем случае – это 2 символа, так как сумма с трех кубиков максимально может равняться числу «18».

Последний параметр «Точность» – это количество чисел после запятой. Так как на кубиках только целые числа, он будет равен «0».

Аналогично нужно заполнить метод для ввода второго числа. Тогда код процедуры будет выглядеть следующим образом:

```
#Область ТриКубика
&НаКлиенте
АСИНХ Процедура ТриКубикаАсинх(Команда)
    ЗагаданноеЧисло1 = 0;
    ЗагаданноеЧисло2 = 0;
    ВвестиЧислоАсинх(ЗагаданноеЧисло1, "Введите первое загаданное число", 2, 0);
    ВвестиЧислоАсинх(ЗагаданноеЧисло2, "Введите второе загаданное число", 2, 0);
КонецПроцедуры
#КонецОбласти
```

Откроем пользовательский режим. При нажатии кнопки «Три кубика» откроется сразу два окна ввода (рис. 1.3.8).

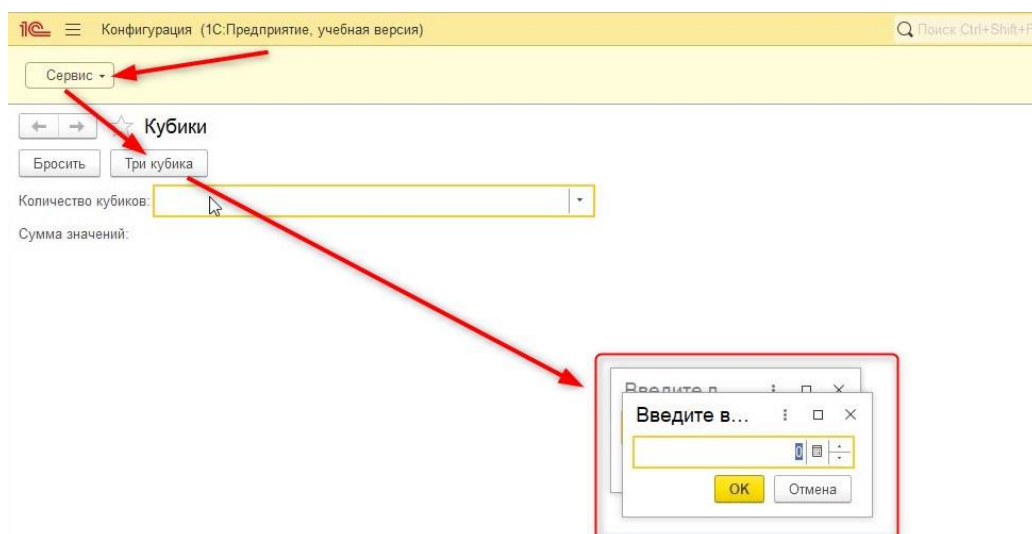


Рис. 1.3.8. Код области «ТриКубика»

Связано это с тем, что у асинхронных методов есть ряд форм работ с ними.

Форма работы с асинхронностью включает несколько составных частей, а именно:

- Тип «Обещание»;
- Конструкции «Асинх» и «Ждать» во встроенном языке.

Рассмотрим последнюю часть – «Асинх» и «Ждать».

«Асинх» представляет собой модификатор, который может применяться к процедуре или функции. Использование данного модификатора делает процедуру или функцию асинхронной:

```
АСИНХ Процедура НазваниеПроцедуры
КонецПроцедуры
//ИЛИ
АСИНХ Функция НазваниеФункции
КонецФункции
```


Также у асинхронных функций/процедур есть оператор «Ждать». Оператор «Ждать» используется для ожидания окончания «Обещания». Обещание – это своеобразный «контейнер» для, возможно, пока неизвестного результата выполнения некоторого действия.

Важно!

Оператор «Ждать» может использоваться только внутри «Асинх» процедур/функций.

Используя приведенную выше теорию, отредактируем код следующим образом:

```
#Область ТриКубика
&НаКлиенте
//Объявление асинхронной процедуры
АСИНХ Процедура ТриКубикаАсинх(Команда)
    ЗагаданноеЧисло1 = 0;
    ЗагаданноеЧисло2 = 0;
    //ВвестиЧислоАсинх(..) - это "Обещание"
    //""ЖДАТЬ" - программа ждёт пока пользователь введёт число
    ЖДАТЬ ВвестиЧислоАсинх(ЗагаданноеЧисло1, "Введите первое загаданное число", 2, 0);
    ЖДАТЬ ВвестиЧислоАсинх(ЗагаданноеЧисло2, "Введите второе загаданное число", 2, 0);
КонецПроцедуры
#КонецОбласти
```

Важно!

При разработке грамотный специалист укажет в конце названия асинхронной процедуры/функции аббревиатуру «Асинх» (рис. 1.3.9).

&НаКлиенте
АСИНХ Процедура ТриКубикаАсинх(Команда)

Рис. 1.3.9. Правило при разработке

Необходимо так же отредактировать код, написанный на предыдущем занятии. Так как в процедуре «БроситьКубики()» был использован асинхронный метод, необходимо саму процедуру сделать асинхронной и добавить перед каждым методом оператор «Ждать» (рис. 1.3.10).

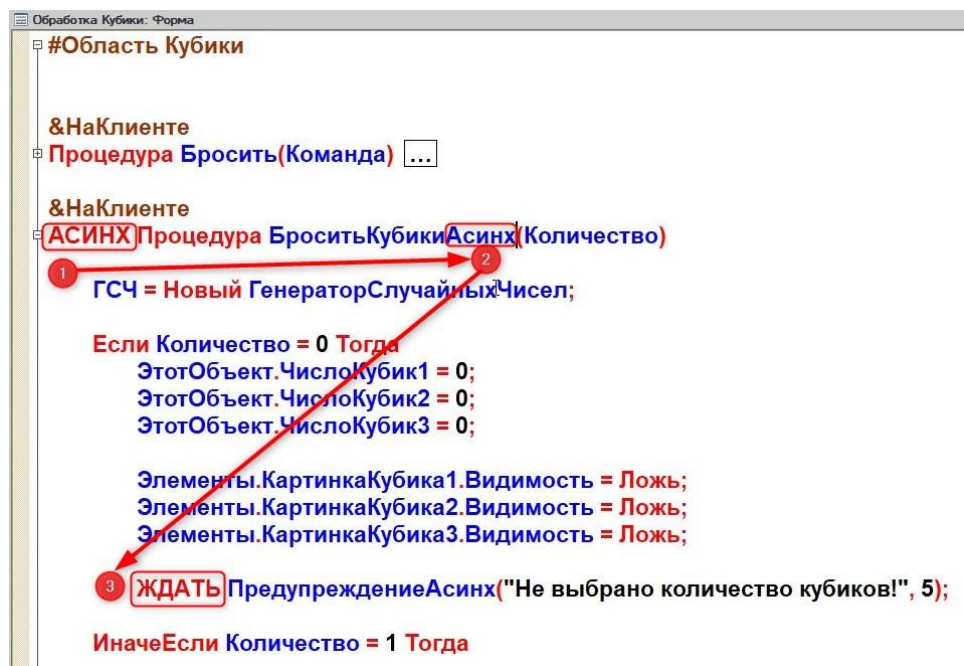


Рис. 1.3.10. Редактирование процедуры прошлого урока

Проверим модуль на наличие ошибок. Для этого нажмем кнопку «Проверка модуля». Код не полностью исправлен, поэтому программа выдаст синтаксическую ошибку (рис. 1.3.11).

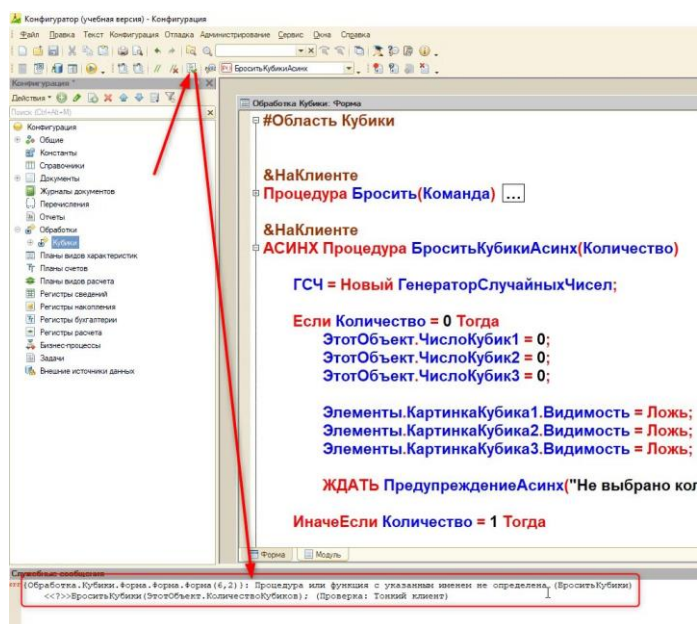


Рис. 1.3.11. Редактирование процедуры прошлого урока

Нажмем по ошибке двойным щелчком мыши, чтобы произошел перенос на строку, где она возникла. Так как имя процедуры было изменено, то процедура, которая вызывалась в команде «Бросить», больше не существует. Изменим «БроситьКубики» на «БроситьКубикиАсинх» в команде «Бросить()» (рис. 1.3.12), и ошибка больше возникать не будет.

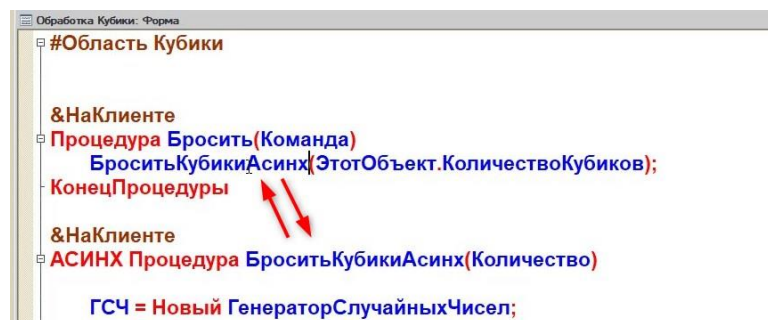


Рис. 1.3.11. Редактирование вызова процедуры

При изменении названия команды, как произошло в случае с «ТриКубикаАсинх()», его привязка к кнопке потеряется. И тогда в пользовательском режиме при нажатии на соответствующую кнопку ничего происходить не будет.

Починить это можно с помощью свойства «Действие» у команды «ТриКубика». В выпадающем списке у свойства нужно выбрать процедуру с измененным названием (рис. 1.3.12).

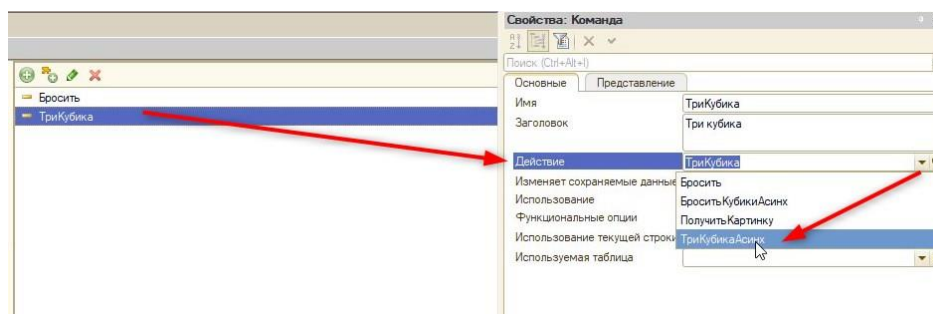


Рис. 1.3.12. Изменения действия у команды

Теперь в пользовательском режиме окна для заполнения суммы будут выводиться друг за другом.

На этом программирование игры не закончилось. Необходимо программно бросить три кубика и после этого проанализировать количество очков, которое на них выпало.

Для этого вернемся в модуль формы обработки и дополним процедуру:

```
#Область ТриКубика
&НаКлиенте
АСИНХ Процедура ТриКубикаАсинх(Команда)
    ЗагаданноеЧисло1 = 0;
    ЗагаданноеЧисло2 = 0;
    ЖДАТЬ ВвестиЧислоАсинх(ЗагаданноеЧисло1, "Введите первое загаданное число", 2, 0);
    ЖДАТЬ ВвестиЧислоАсинх(ЗагаданноеЧисло2, "Введите второе загаданное число", 2, 0);
    //Укажем количество кубиков для броска (три):
    ЭтотОбъект.КоличествоКубиков = 3;
    //Вызовем процедуру БроситьКубикиАсинх и передадим в нее заполненное ранее значение с
    количеством кубиков
    //(Если не передать количество кубиков возникнет ошибка)
    БроситьКубикиАсинх(ЭтотОбъект.КоличествоКубиков);
    КонецПроцедуры
#КонецОбласти
```

Проверим работоспособность кода в пользовательском режиме, перед этим обновив конфигурацию данных. Если код написан корректно, то программа предложит выбрать два значения и сделает бросок с тремя кубиками (рис. 1.3.13).

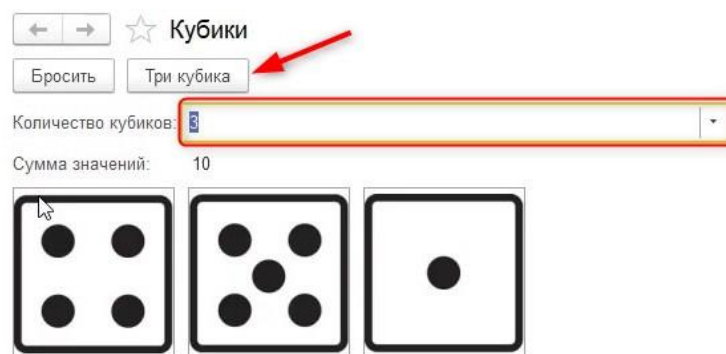


Рис. 1.3.13. Работа программы

Теперь нужно проанализировать сумму значений, которая посчиталась после броска, и сравнить с тем количеством, которое загадывал пользователь. Для этого нам потребуется условие:

```
#Область ТриКубика
```

```
&НаКлиенте
```

```
АСИНХ Процедура ТриКубикаАсинх(Команда)
```

```
    ЗагаданноеЧисло1 = 0;
```

```
    ЗагаданноеЧисло2 = 0;
```

```
    ЖДАТЬ ВвестиЧислоАсинх(ЗагаданноеЧисло1, "Введите первое загаданное число", 2, 0);
```

```
    ЖДАТЬ ВвестиЧислоАсинх(ЗагаданноеЧисло2, "Введите второе загаданное число", 2, 0);
```

```
    ЭтотОбъект.КоличествоКубиков = 3;
```

```
    БроситьКубикиАсинх(ЭтотОбъект.КоличествоКубиков);
```

```
//Если сумма значений, которая выпала при броске, равна хотя бы одному (ИЛИ) из двух
загаданных чисел, тогда:
```

```
    Если ЭтотОбъект.СуммаЗначений = ЗагаданноеЧисло1 ИЛИ ЭтотОбъект.СуммаЗначений =
    ЗагаданноеЧисло2 Тогда
```

```
        //Выведем сообщение пользователю о победе:
```

```
        //ПредупреждениеАсинх(Сообщение, Таймаут, ЗаголовокОкна)
```

```
        //Перед асинхронным методом напишем ключевую конструкцию "ЖДАТЬ"
```

```
        ЖДАТЬ ПредупреждениеАсинх("Вы выиграли!", 3, "Победа!");
```

```
        //Если пользователь не выиграл (Иначе):
```

```
        Иначе
```

```
            //При поражении спросим пользователя хочет ли он сыграть ещё:
```

```
            //Сделаем это с помощью асинхронного метода ВопросАсинх(ТекстВопроса, Кнопки,
            Таймаут, КнопкаПоУмолчанию, Заголовок, КнопкаТаймаута)
```

```
            //Текст вопроса - сам вопрос с типом "Строка"
```

```
            //Кнопки - задает состав и текст кнопок диалога, а также, связанные с кнопками
            значения.
```

```
            //Есть несколько уже заготовленных кнопок диалога в платформе, к примеру Да/Нет
```

```
            //Чтобы выбрать их нужно в параметре "Кнопки" указать РежимДиалогаВопрос.ДаНет
```

```

//Таймаут - интервал времени в секундах, в течение которого система будет ожидать
ответа пользователя.

//При нажатии одной из кнопок (Да, нет) будет происходить возврат выбранного ответа
диалога.

//Для того, чтобы обратиться к такому возвращаемому элементу нужно использовать
конструкцию:

//КодВозвратаДиалога.Да или КодВозвратаДиалога.Нет

//КнопкаТаймаута - определяет кнопку (по типу кнопки или по связанному с ней
значению), на которой отображается количество секунд, оставшихся до истечения таймаута.

    Ответ = ЖДАТЬ ВопросАсинх("Не получилось! Игруем еще?",
РежимДиалогаВопрос.ДаНет, 10, КодВозвратаДиалога.Да, "Выберите
КодВозвратаДиалога.Нет);

//Проанализируем ответ, который выбрал пользователь:

// Если ответ - Да, то заново вызываем процедуру с игрой и начинаем её:

Если Ответ = КодВозвратаДиалога.Да Тогда
    ТриКубикаАсинх(Команда);

КонецЕсли;

КонецЕсли;

КонецПроцедуры

#КонецОбласти

```

Важно!

При программировании на 1С можно использовать пробелы и разрывы строк, их программа не будет анализировать.

Если возникает трудность с пониманием, что делает функция, или какие параметры у нее есть, можно воспользоваться встроенным синтаксис-помощником. Найти его можно на верхней панели (рис. 1.3.14) или на вкладке «Справка» (рис. 1.3.15).

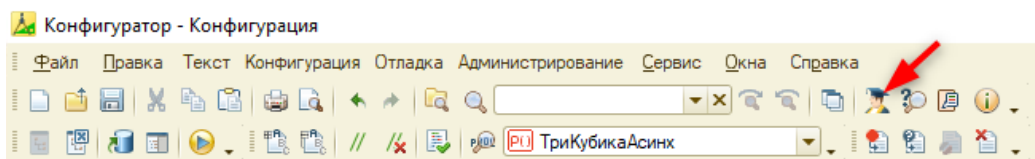


Рис. 1.3.14. Синтаксис-помощник на панели кнопок

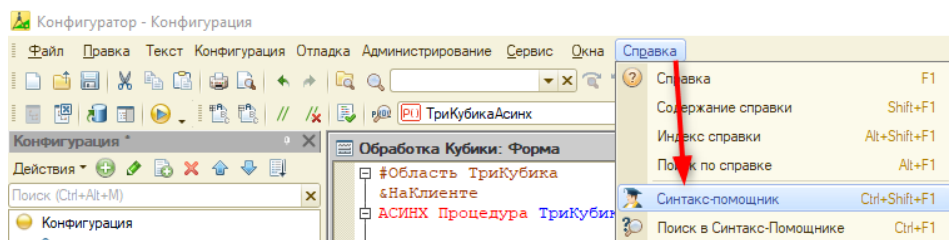


Рис. 1.3.15. Синтаксис-помощник в меню

В нем можно узнать много нужной информации для программиста — от параметров до возвращаемого значения и примера использования (рис. 1.3.16).

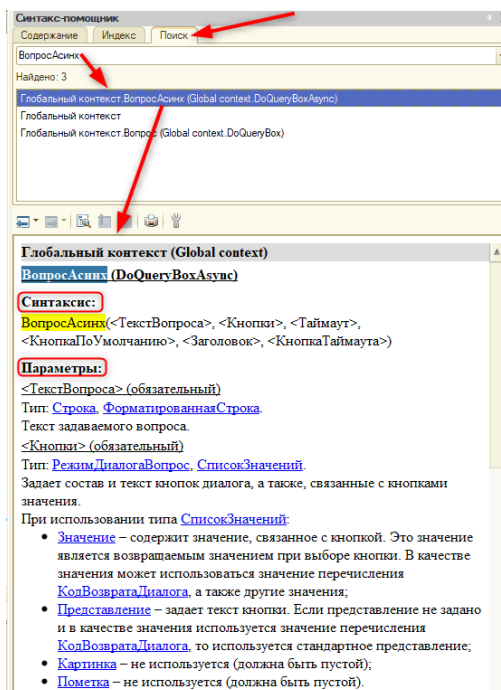


Рис. 1.3.16. Поиск в синтаксис-помощнике

Можно напрямую в коде выделить интересующий метод и посмотреть его описание с помощью правой кнопки мыши и выбора «Поиск в синтаксис-помощнике» (рис. 1.3.17).

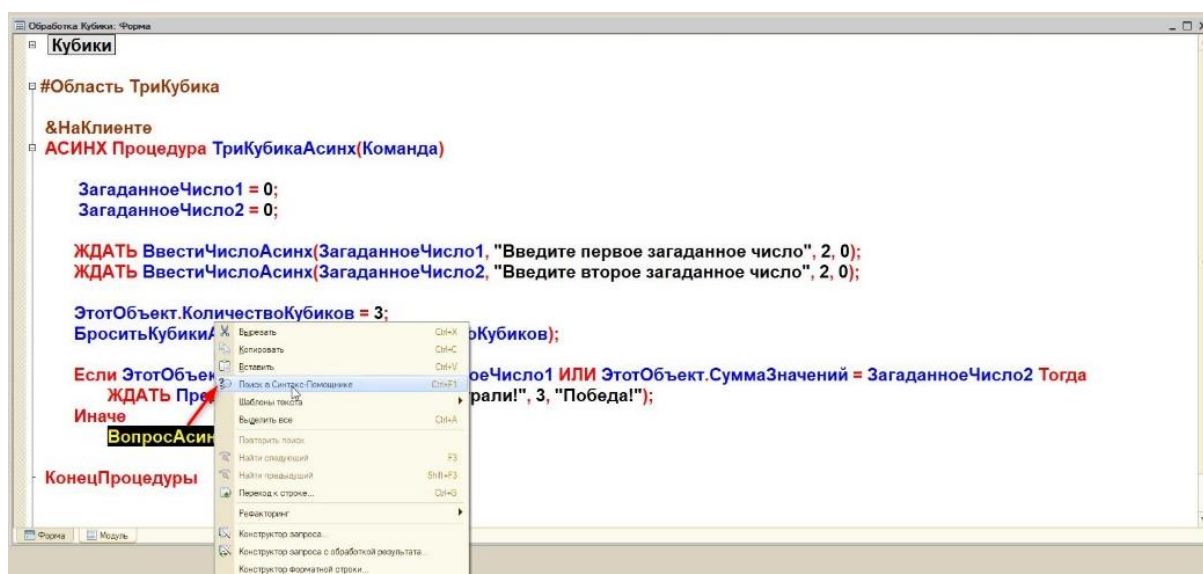


Рис. 1.3.17. Поиск в синтаксис-помощнике метода из кода

Проверим модуль на наличие ошибок и, если ошибок не обнаружено, запустим пользовательский режим.

На этапе тестирования игры можно обнаружить, что даже при выигрыше (когда загаданное число совпало с выпавшим) программа будет обрабатывать условие с проигрышем (рис. 1.3.18).

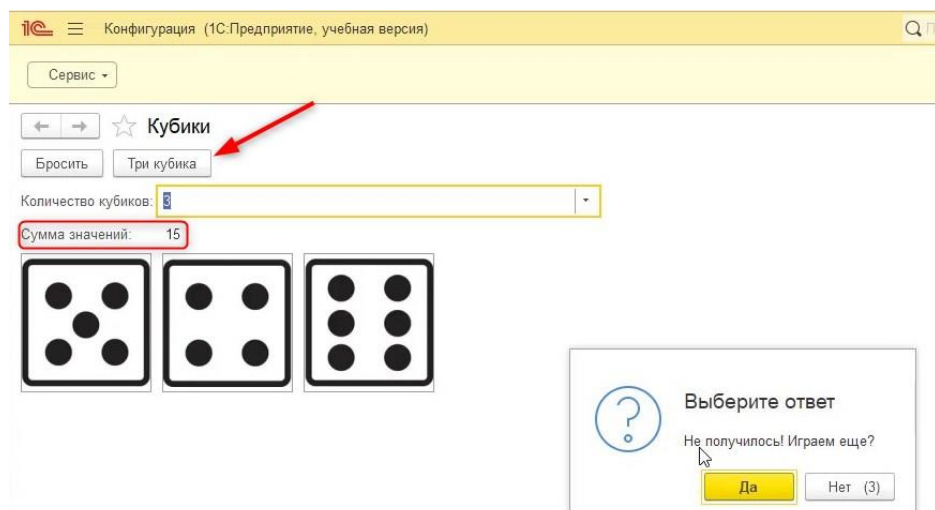


Рис. 1.3.18. Работа программы

Связано это с тем, что система ожидает от пользователя ввода значений через метод «ВвестиЧислоАсинх()», но они у нас никуда не присваиваются. Пользователь их вводит, но загаданные числа не меняются. Поэтому выиграть в данный момент невозможно, так как выбросить сумму равную 0 на кубиках невозможно.

Для того чтобы это исправить, перед ожиданием ввода чисел присвоим результат переменным (рис. 1.3.19).

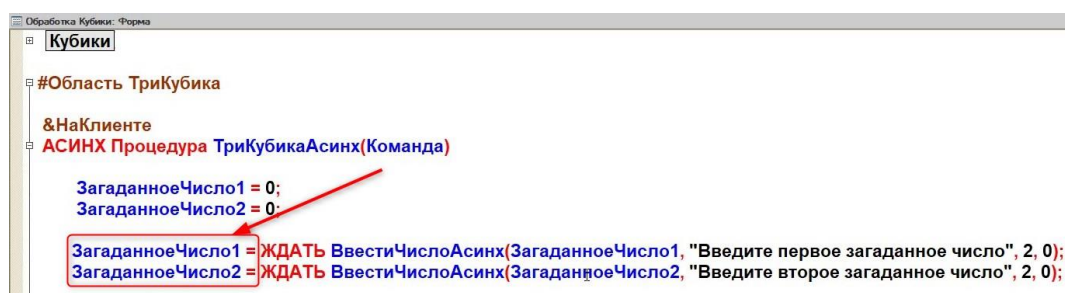


Рис. 1.3.19. Исправление кода

Теперь при проверке работы игры можно убедиться, что срабатывает условие с выигрышем.

Далее реализуем кнопку «Рестарт», которая будет сбрасывать все результаты игры. Для этого создадим команду «Рестарт» и перенесем её в командную панель (рис. 1.3.20).

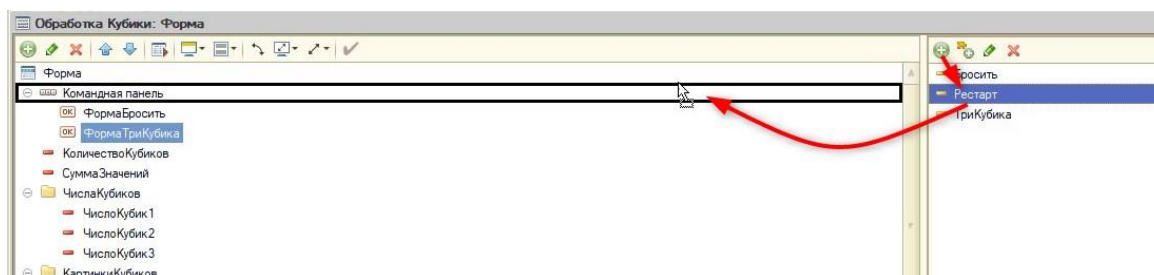


Рис. 1.3.20. Создание новой кнопки

После этого опишем для данной кнопки действие. Сделать это можно не открывая свойства команды – при помощи правой кнопки мыши по соответствующему элементу и выбора «Действие команды» (рис. 1.3.21).

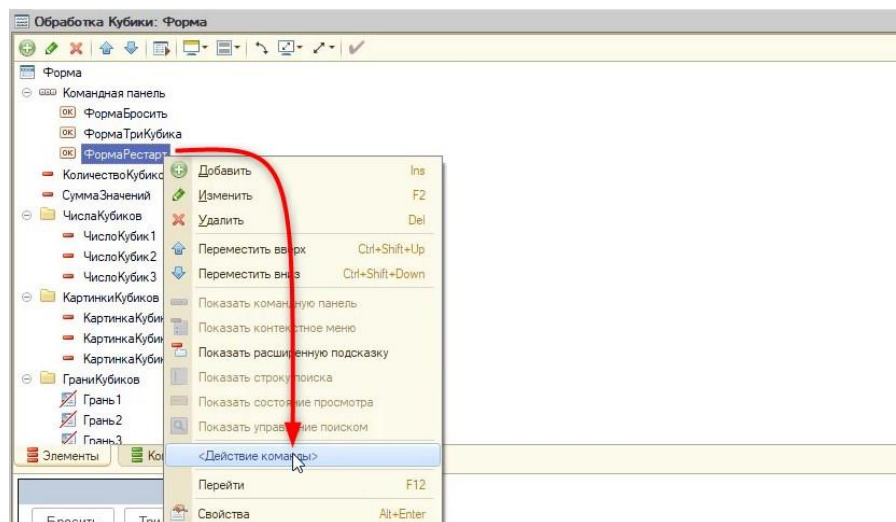


Рис. 1.3.21. Создание действия команды

В открывшемся окне выберем «НаКлиенте».

Для новой кнопки создадим новую область «Рестарт» и переместим в обработчик (рис. 1.3.22).

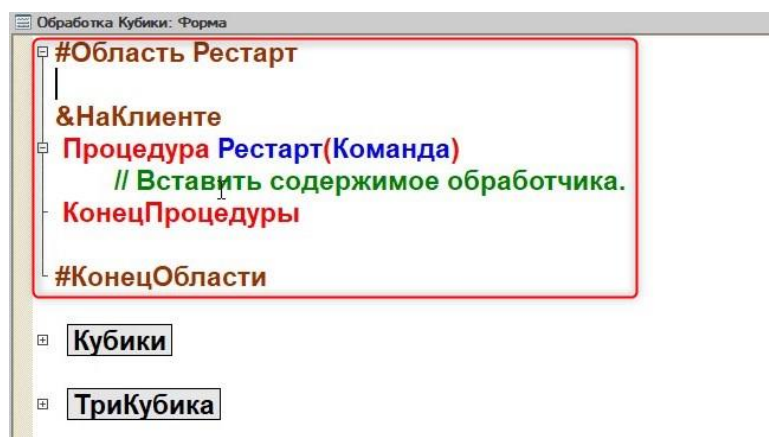


Рис. 1.3.22. Создание новой области

Кнопка «Рестарт» будет убирать изображения граней кубиков (настройка их видимости), обнулять значения бросков, их сумму и количество:

```
#Область Рестарт
&НаКлиенте
Процедура Рестарт(Команда)
    //Спрячем все картинки:
    Элементы.КартинкаКубика1.Видимость = Ложь;
    Элементы.КартинкаКубика2.Видимость = Ложь;
    Элементы.КартинкаКубика3.Видимость = Ложь;

    //Обнулим числовые значения кубиков:
    ЭтотОбъект.ЧислоКубик1 = 0;
    ЭтотОбъект.ЧислоКубик2 = 0;
    ЭтотОбъект.ЧислоКубик3 = 0;

    //Обнулим сумму и количество:
```

```
ЭтотОбъект.СуммаЗначений = 0;  
ЭтотОбъект.КоличествоКубиков = 0;  
КонецПроцедуры  
#КонецОбласти
```

Проверим работу кнопки в пользовательском режиме, попутно обновив конфигурацию. Если код написан верно, то после броска кубиков и нажатия кнопки «Рестарт» все значения и картинки сбросятся (рис. 1.3.23).

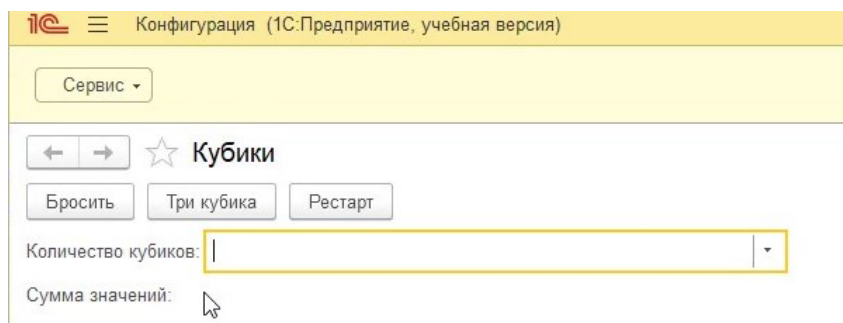


Рис. 1.3.23. Работа кнопки «Рестарт»

Занятие 4 – «Крэпс» и «Два кубика»

1.1.1. Игра «Крэпс»

Для начала реализуем игру «Крэпс» с правилами, представленными на скриншоте (рис. 1.4.1).



Правила игры в Крэпс

- Первый бросок:
 - Если выпадает 7 или 11, тогда победа;
 - Если выпадает 2, 3 или 12, тогда проигрыш;
 - Если выпадает другое значение, тогда оно записывается как Пойнт.
- Последующие броски:
 - Если выпадает 7, тогда проигрыш;
 - Если выпадает Пойнт (значение первого броска), тогда победа.

Рис. 1.4.1. Правила игры в «Крэпс»

Откроем в конфигураторе форму обработки «Кубики» (рис. 1.4.2).

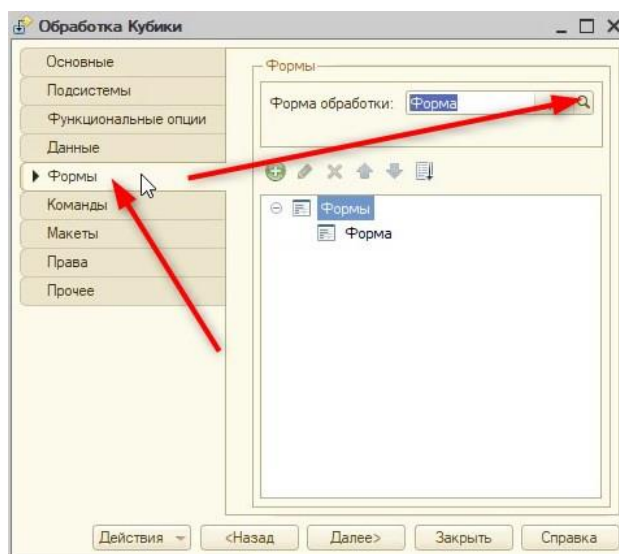


Рис. 1.4.2. Формы обработки

В компьютерных играх есть такое определение, как «point». Пойнт — это первый результат броска игрока. К примеру, если с первого броска пользователь выбросил «10» — это его пойнт.

Для того чтобы пойнт хранился в системе, необходимо создать одноименный реквизит с типом «Число» (рис. 1.4.3), длиной 2 и включенным свойством «Неотрицательное».

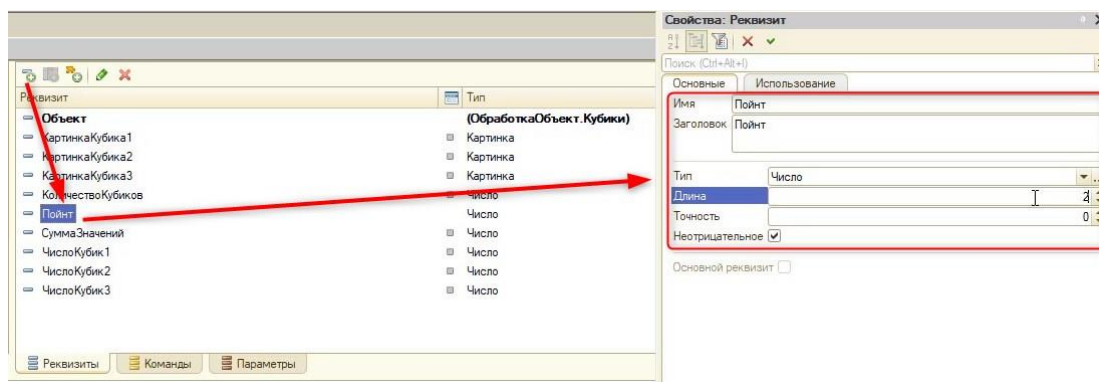


Рис. 1.4.3. Создание реквизита формы «Пойнт»

Данный реквизит будет первым в обработке, который не требуется располагать на форме. Работа с ним будет происходить только программно.

Далее создадим новую команду «Крэпс» и перенесём её на панель команд на форме (рис. 1.4.4).

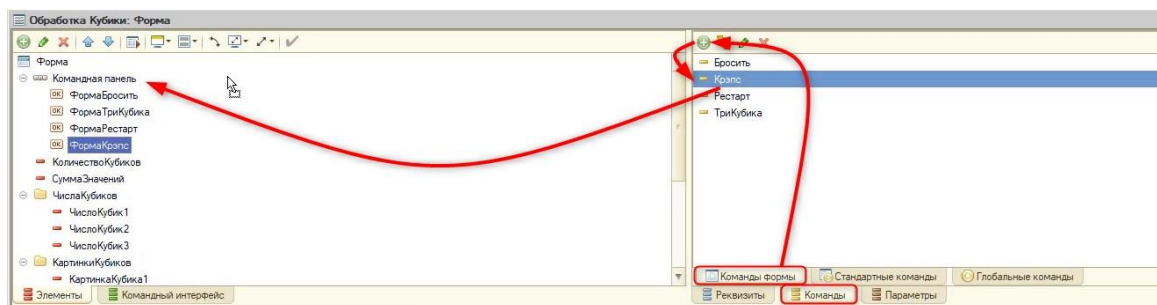


Рис. 1.4.4. Создание команды и кнопки «Крэпс»

Перенесём с помощью стрелок «Вверх» и «Вниз» новую кнопку на форме так, как показано на рис. 1.4.5.

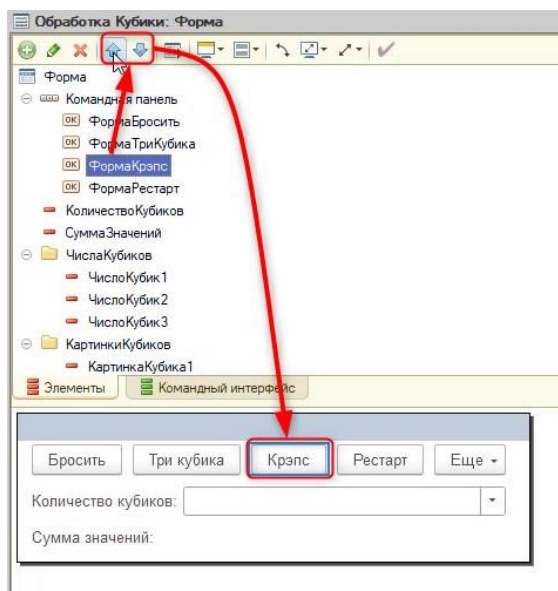


Рис. 1.4.5. Смена расположения кнопки на форме

После этого создадим действие для нашей команды на клиенте (рис. 1.4.6-1.4.7).

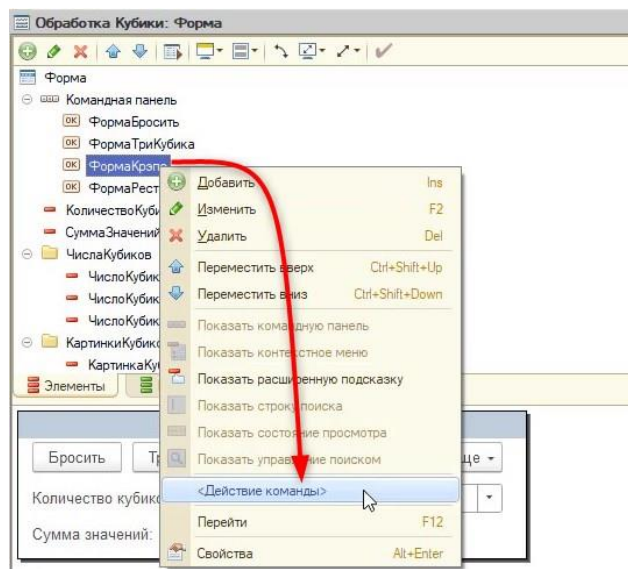


Рис. 1.4.6. Создание действия команды

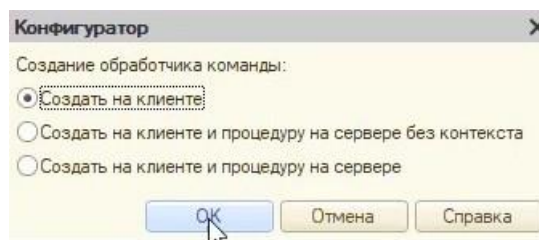


Рис. 1.4.6 Выбор директивы

Так как «Крэпс» — это отдельная игра, она будет описываться в отдельной новой области с одноименным названием (рис. 1.4.7).

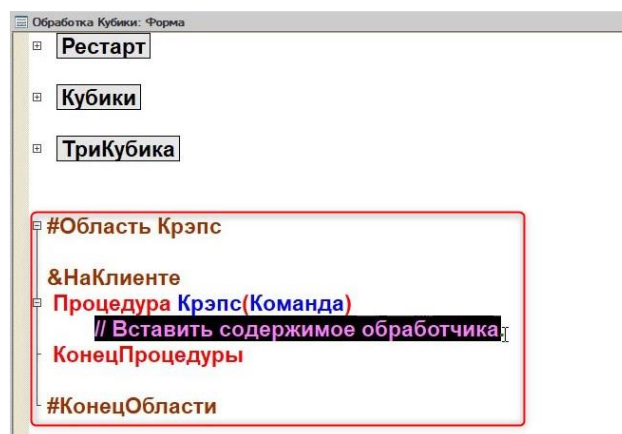


Рис. 1.4.7. Создание новой области

Реализуем код процедуры:

```
#Область Крэпс
```

```
&НаКлиенте
```

```
//Так как в процедуре будут использоваться асинхронные методы, она тоже должна быть асинхронной:
```

```
АСИНХ Процедура КрэпсАсинх(Команда)
```

```
//Зададим количество кубиков, которое нужно бросить.
```

```
ЭтотОбъект.КоличествоКубиков = 2;
```

```
//Вызовем процедуру с броском кубиков и передадим в нее заданное количество кубиков.
```

```
БроситьКубикиАсинх(ЭтотОбъект.КоличествоКубиков);
```



```

//Для начала проанализируем, является ли бросок первым.
//Если бросок первый, то Пойнт будет равняться 0
Если ЭтотОбъект.Пойнт = 0 Тогда
    //Проверка на моментальную победу:
    //Если сумма броска 7 или 11 - победа
    Если ЭтотОбъект.СуммаЗначений = 7 ИЛИ ЭтотОбъект.СуммаЗначений = 11
Тогда
    //Выведем сообщение о победе.
    ЖДАТЬ ПредупреждениеАсинх("Вы выиграли!", 3, "Победа!");
    //Проверка на моментальное поражение:
    //Если сумма равна 2 или 3 или 12 - поражение
    ИначеЕсли ЭтотОбъект.СуммаЗначений = 2 ИЛИ ЭтотОбъект.СуммаЗначений
= 3 ИЛИ ЭтотОбъект.СуммаЗначений = 12 Тогда
        //Выведем сообщение о поражении.
        ЖДАТЬ ПредупреждениеАсинх("Вы проиграли!", 3, "Поражение!");
    Иначе
        //Если не выполнены условия, записываем поинт:
        ЭтотОбъект.Пойнт = ЭтотОбъект.СуммаЗначений;
    КонецЕсли;
//Ход игры, если бросок является не первым, т.е. поинт больше 0.
Иначе
    //Если игрок выкидывает сумму кубиков 7, то он проигрывает:
    Если ЭтотОбъект.СуммаЗначений = 7 Тогда
        //Выведем сообщение о поражении.
        ЖДАТЬ ПредупреждениеАсинх("Вы проиграли!", 3, "Поражение!");
        //Так как игра закончилась, обнуляем значение поинта:
        ЭтотОбъект.Пойнт = 0;
    //Если игрок выкинул сумму 3, то он выигрывает:
    ИначеЕсли ЭтотОбъект.СуммаЗначений = ЭтотОбъект.Пойнт Тогда
        //Выведем сообщение о победе
        ЖДАТЬ ПредупреждениеАсинх("Вы выиграли!", 3, "Победа!");
        //Так как игра закончилась, обнуляем значение поинта:
        ЭтотОбъект.Пойнт = 0;
    КонецЕсли;
КонецЕсли;
КонецПроцедуры
#КонецОбласти

```

Так как в процессе написания кода мы поменяли название процедуры на «КрэпсАсинх» (согласно правилам написания кода), нужно заново привязать процедуру к команде (рис. 1.4.8-1.4.9).

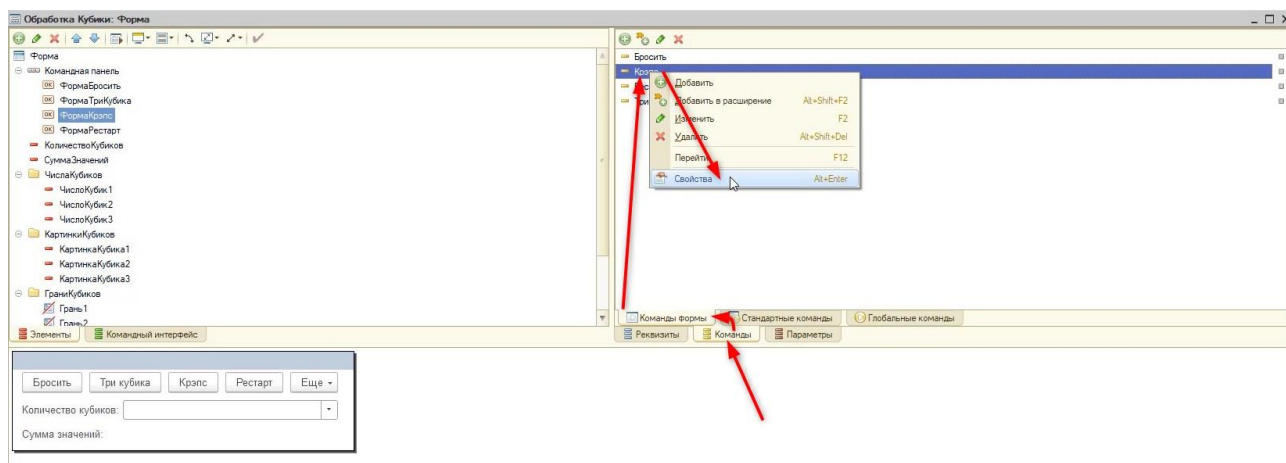


Рис. 1.4.8. Откроем свойства команды «Крэпс»

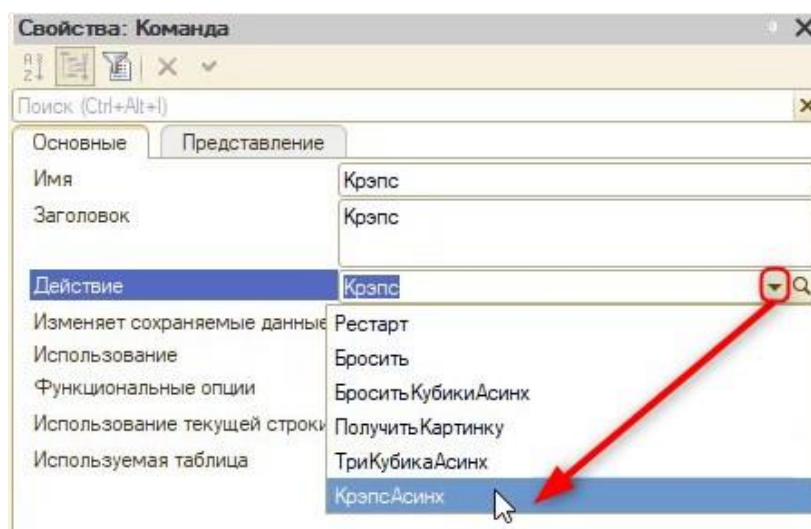


Рис. 1.4.9. Изменим действие на «КрэпсАсинх»

Готово! Теперь в пользовательском режиме можно протестировать игру. Если всё сделано верно — игра будет работать корректно.

На данный момент кнопка «Рестарт» не сбрасывает значение поинта, значит, не обнуляет игру «Крэпс». Исправим это и допишем код в процедуре «Рестарт» (рис. 1.4.10).

```
#Область Рестарт
&НаКлиенте
Процедура Рестарт(Команда)
    Элементы.КартинкаКубика1.Видимость = Ложь;
    Элементы.КартинкаКубика2.Видимость = Ложь;
    Элементы.КартинкаКубика3.Видимость = Ложь;

    ЭтотОбъект.ЧислоКубик1 = 0;
    ЭтотОбъект.ЧислоКубик2 = 0;
    ЭтотОбъект.ЧислоКубик3 = 0;

    ЭтотОбъект.СуммаЗначений = 0;
    ЭтотОбъект.КоличествоКубиков = 0;
    ЭтотОбъект.Пойнт = 0;

КонецПроцедуры
#КонецОбласти
```

Рис. 1.4.10. Добавление обнуления поинта

Теперь пользователь может играть в «Крэпс»!

1.1.2. «Два кубика»

Игра «Два кубика» имеет следующую механику (рис. 1.4.11):

- 1) Сумма выброшенного кубика — это победные очки.
- 2) Сумма второго выброшенного кубика — это проигрышные очки.
- 3) Счёт игрока считается с помощью разности значений первого и второго кубиков.

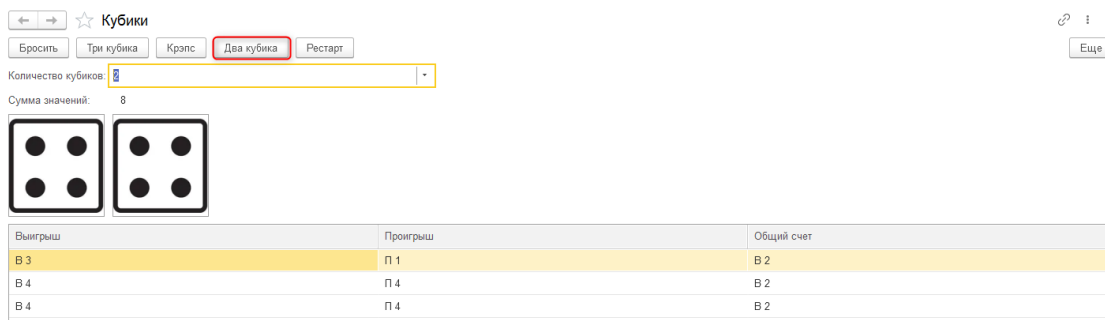


Рис. 1.4.11. Игра в два кубика

Для начала необходимо добавить два реквизита: «ТекущийСчёт» и «ПредыдущийСчёт». Текущий счёт будет анализировать, идем мы в плюс или минус, а предыдущий счёт будет запоминать значение предыдущего этапа. Оба новых реквизита будут иметь тип «Число» с длиной 10 и возможностью использования отрицательных чисел (рис. 1.4.12-1.4.13).

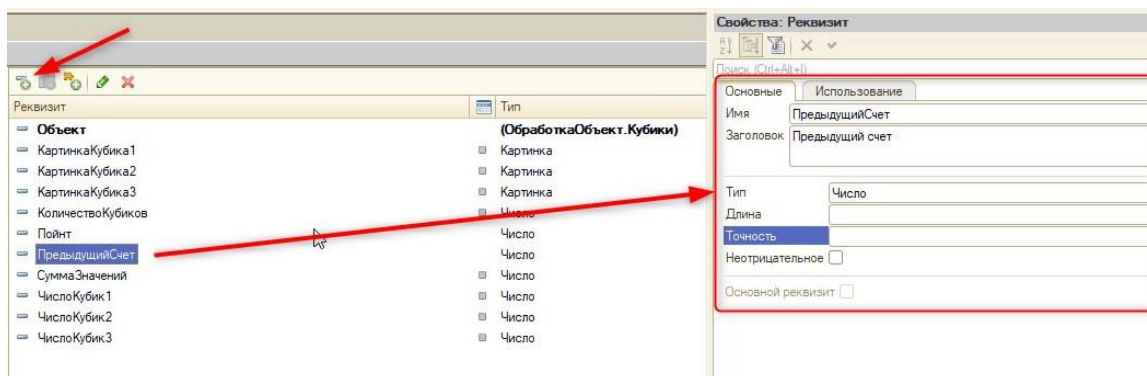


Рис. 1.4.12. Создание нового реквизита формы

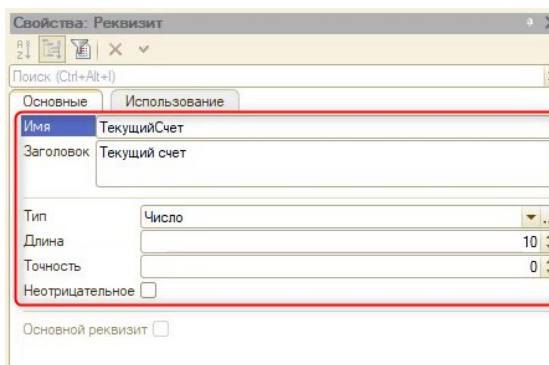


Рис. 1.4.13. Создание второго реквизита формы

Счёт игрока должен выводиться в таблицу. Для хранения такой информации потребуется реквизит «ТаблицаСчета» с типом «Таблица значений».

Важно!

Таблица значений — специальный объект в программировании 1С, который позволяет хранить значения в виде таблицы со строками и колонками.

Создадим реквизит «ТаблицаСчета» с типом «Таблица значений» (рис. 1.4.14).

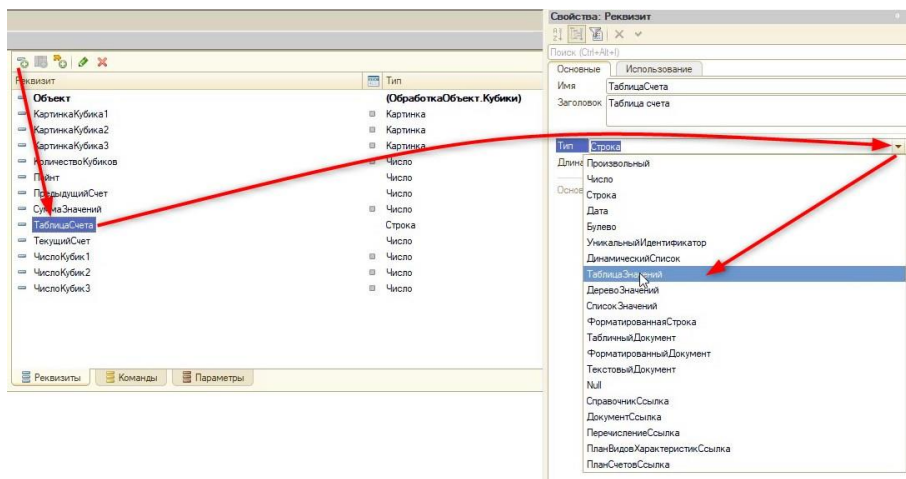


Рис. 1.4.14. Создание таблицы значений

Далее для таблицы необходимо описать колонки. Для этого нужно нажать по самой таблице правой кнопкой мыши, выбрав «Добавить колонку реквизита» (рис. 1.4.15).

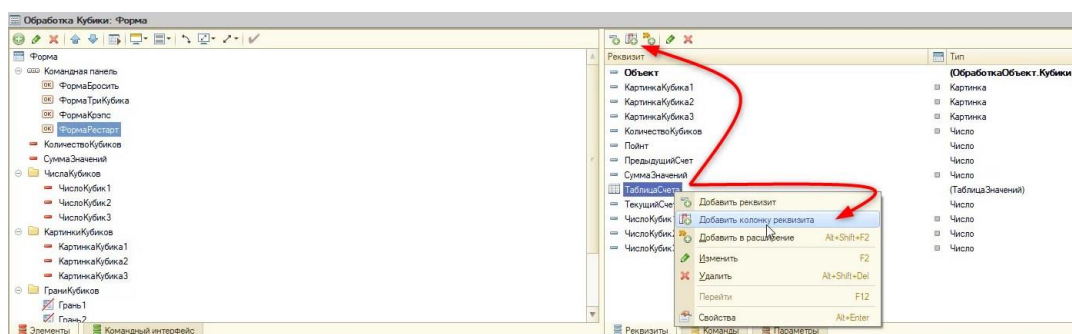


Рис. 1.4.15. Создание колонки реквизита

Создадим три колонки. Первая и вторая колонки будут называться «Выигрыш» и «Проигрыш», тип данных — «Строка». Кроме очков будут выводиться буквы: «В» (Выигрыш) или «П» (Проигрыш). Третья колонка, «ОбщийСчёт», будет также иметь тип «Строка» (рис. 1.4.16).

Реквизит	Использовать всегда	Тип
Объект		(ОбработкаОбъект.Кубики)
КартинкаКубика1		Картинка
КартинкаКубика2		Картинка
КартинкаКубика3		Картинка
КоличествоКубиков		Число
Пойнт		Число
ПредыдущийСчет		Число
СуммаЗначений		Число
ТаблицаСчета		(ТаблицаЗначений)
Выигрыш	☒	Строка
Проигрыш	☒	Строка
ОбщийСчет	☒	Строка
ТекущийСчет		Число
ЧислоКубик1		Число
ЧислоКубик2		Число
ЧислоКубик3		Число

Рис. 1.4.15. Колонки таблицы счета

Таблицу счёта, удерживая правой кнопкой мыши, перенесём на форму (рис. 1.4.16).

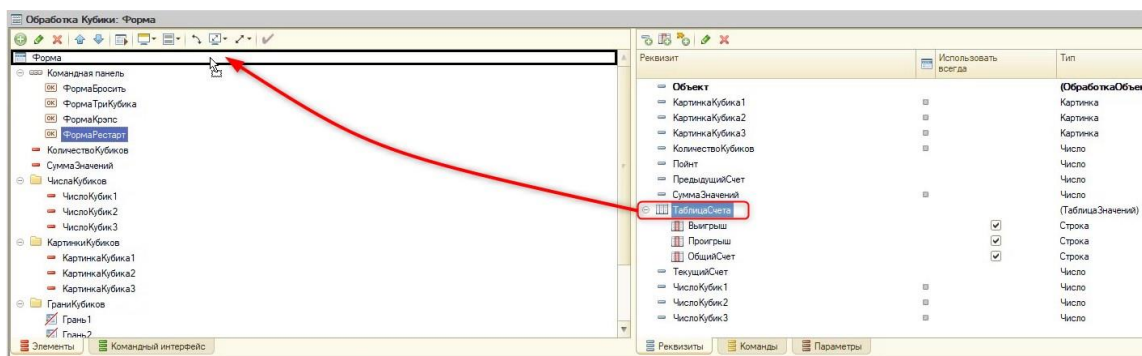


Рис. 1.4.16. Добавление таблицы на форму

После этого в окне предпросмотра отобразится таблица (рис. 1.4.17).

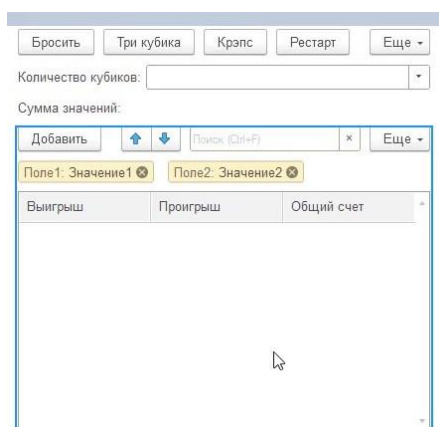


Рис. 1.4.17. Таблица на форме

Так как таблица будет заполняться только программно, нужно исключить вмешательство игрока. Для этого скроем командную панель таблицы в её свойствах (рис. 1.4.18-1.4.19).

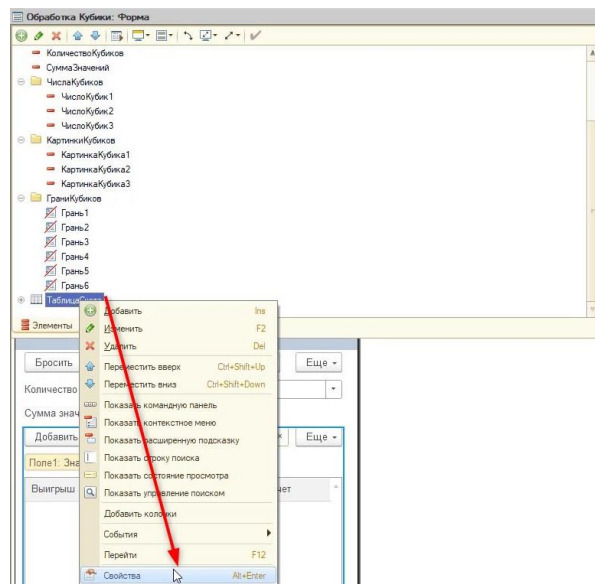


Рис. 1.4.18. Открытие свойств таблицы

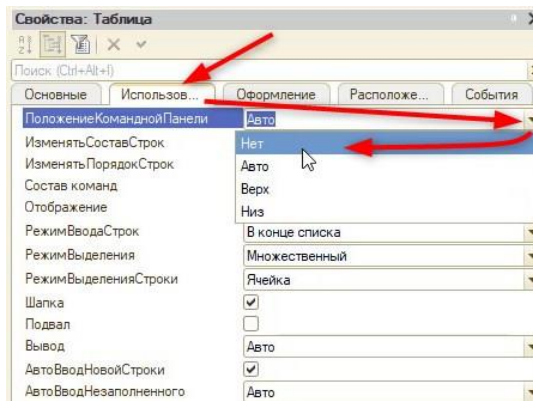


Рис. 1.4.19. Скрытие командной панели

Чтобы игрок не мог менять значения таблицы, на вкладке «Основные» нужно включить свойство «ТолькоПросмотр» (рис. 1.4.20).

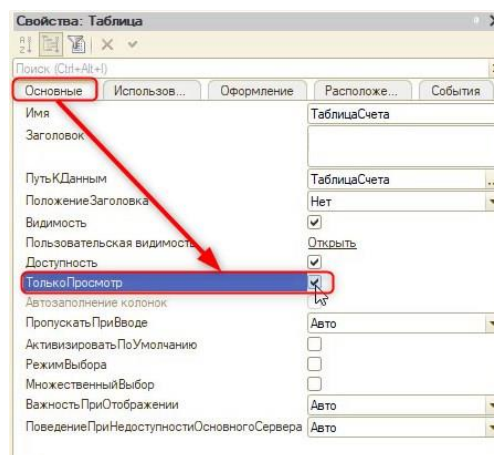


Рис. 1.4.20. Свойство «Только просмотр»

В рамках одной обработки присутствует сразу несколько игр. Чтобы таблица не мешала игроку в процессе других игр, скроем её видимость на вкладке свойств «Основные» (рис. 1.4.21).

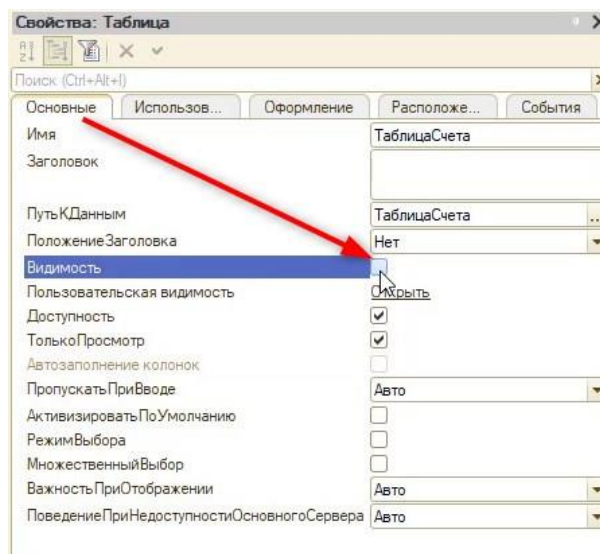


Рис. 1.4.21. Свойство «Только просмотр»

Теперь всё готово, и можно приступить к описанию программной логики. Создадим команду «ДваКубика» и расположим кнопку на форме, как делали ранее (рис. 1.4.22).

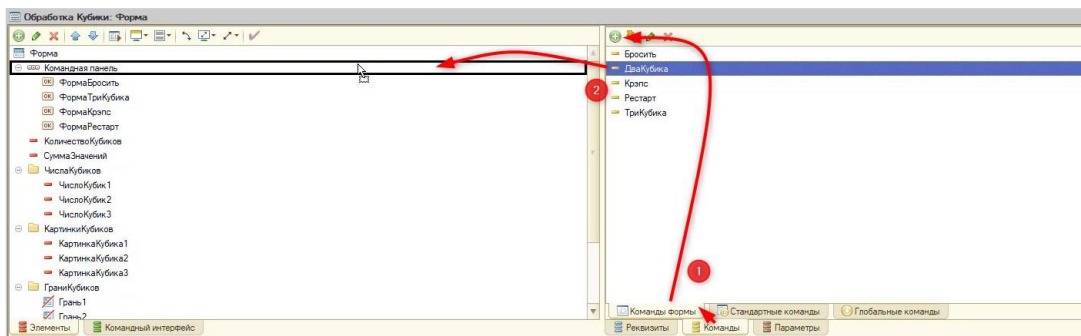


Рис. 1.4.21. Свойство «Только просмотр»

После этого опишем действия для новой команды на клиенте (рис. 1.4.22).

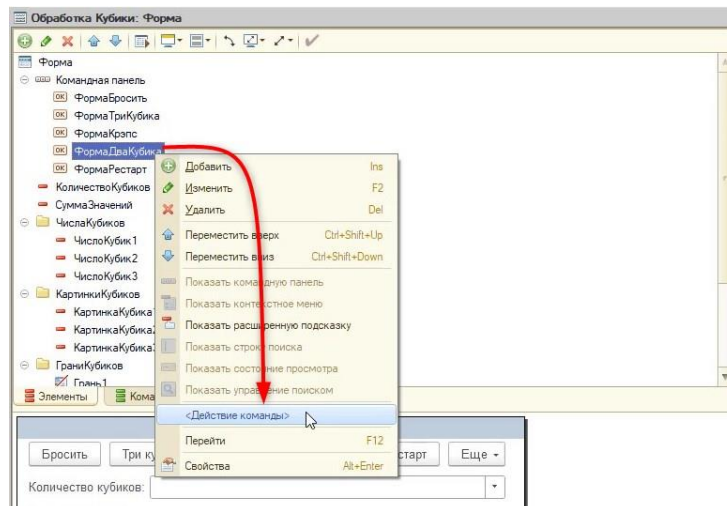


Рис. 1.4.22. Создание действия для команды

Создадим новую область «ДваКубика» и поместим туда новую процедуру:

#Область ДваКубика

&НаКлиенте

Процедура ДваКубика(Команда)

//Зададим количество кубиков, которое нужно бросить.

ЭтотОбъект.КоличествоКубиков = 2;

//Вызовем процедуру с броском кубиков и передадим в нее заданное количество кубиков.

БроситьКубикиАсинх(ЭтотОбъект.КоличествоКубиков);

//Включим видимость табличной части для пользователя:

Элементы.ТаблицаСчета.Видимость = Истина;

//Программно добавляем строку в таблицу:

НоваяСтрока = ЭтотОбъект.ТаблицаСчета.Добавить();

//Обратимся к колонкам таблицы и заполним их:

НоваяСтрока.Выигрыш = "В " + ЭтотОбъект.ЧислоКубик1;

НоваяСтрока.Проигрыш = "П " + ЭтотОбъект.ЧислоКубик2;

//Посчитаем текущий счет с помощью разности значений выигрышных и проигрышных очков:

```
//Значения первого кубика - выигрышные очки
//Значения второго кубика - проигрышные
ЭтотОбъект.ТекущийСчет = ЭтотОбъект.ЧислоКубик1 - ЭтотОбъект.ЧислоКубик2;

//Посчитаем суммарный счет с помощью сложения текущего и предыдущего счета:
СуммарныйСчет = ЭтотОбъект.ТекущийСчет + ЭтотОбъект.ПредыдущийСчет;
//Далее необходимо заполнить последнюю колонку
//Заполнять нужно, аналогично, с буквами "П" или "В"
```

//Так как заранее мы не знаем, выигрывает или проигрывает игрок, нужно проверить положительное или отрицательное число

```
//Если число больше нуля (положительное):
```

```
Если СуммарныйСчет > 0 Тогда
```

```
    //Добавляем к счету букву "В" (Выигрыш)
```

```
    НоваяСтрока.ОбщийСчет = "В " + СуммарныйСчет;
```

```
//Если число меньше нуля (отрицательное):
```

```
ИначеЕсли ЭтотОбъект.ТекущийСчет < 0 Тогда
```

```
    //Добавляем к счету букву "П" (Проигрыш)
```

```
    //И умножаем суммарный счет на «-1», чтобы пользователю не отображался
```

минус

```
    НоваяСтрока.ОбщийСчет = "П " + (-1 * СуммарныйСчет);
```

```
//Если число равно 0
```

```
Иначе
```

```
    //Добавим к счету букву "Н" (Ничья)
```

```
    НоваяСтрока.ОбщийСчет = "Н " + СуммарныйСчет;
```

```
КонецЕсли;
```

```
//Обновим счет:
```

```
ЭтотОбъект.ПредыдущийСчет = СуммарныйСчет;
```

```
КонецПроцедуры
```

```
#КонецОбласти
```

После этого обновим код кнопки «Рестарт»:

```
#Область Рестарт
```

```
&НаКлиенте
```

```
Процедура Рестарт(Команда)
```

```
    Элементы.КартинкаКубика1.Видимость = Ложь;
```

```
    Элементы.КартинкаКубика2.Видимость = Ложь;
```

```
    Элементы.КартинкаКубика3.Видимость = Ложь;
```

```
    ЭтотОбъект.ЧислоКубик1 = 0;
```

```
    ЭтотОбъект.ЧислоКубик2 = 0;
```

```
    ЭтотОбъект.ЧислоКубик3 = 0;
```

```
    ЭтотОбъект.СуммаЗначений = 0;
```

```
    ЭтотОбъект.КоличествоКубиков = 0;
```

```
    ЭтотОбъект.Пойнт = 0;
```

```
//Очистим табличную часть:  
ЭтотОбъект.ТаблицаСчета.Очистить();  
//Скроем видимость таблицы:  
Элементы.ТаблицаСчета.Видимость = Ложь;  
//Обнулим значения Предыдущего и Текущего счета:  
ЭтотОбъект.ПредыдущийСчет = 0;  
ЭтотОбъект.ТекущийСчет = 0;
```

КонецПроцедуры

#КонецОбласти

Готово! За одно занятие мы создали две игры с разными механиками. Далее мы приступим к созданию карточной игры.

Занятие 5 – Карточная игра. Хранение карт

Продолжим разработку компьютерных игр. Следующая на очереди игра — аналог карточной игры «УНО». Цель игры — разыграть все имеющиеся карты на руке. Розыгрыш происходит по принципу совпадения цвета или числа на карте.

Игра будет реализована с одним игроком и ботом.

Для начала потребуется реализовать функционал, который позволит сохранять карту с её числовым номером, цветом и изображением (рис. 1.5.1).

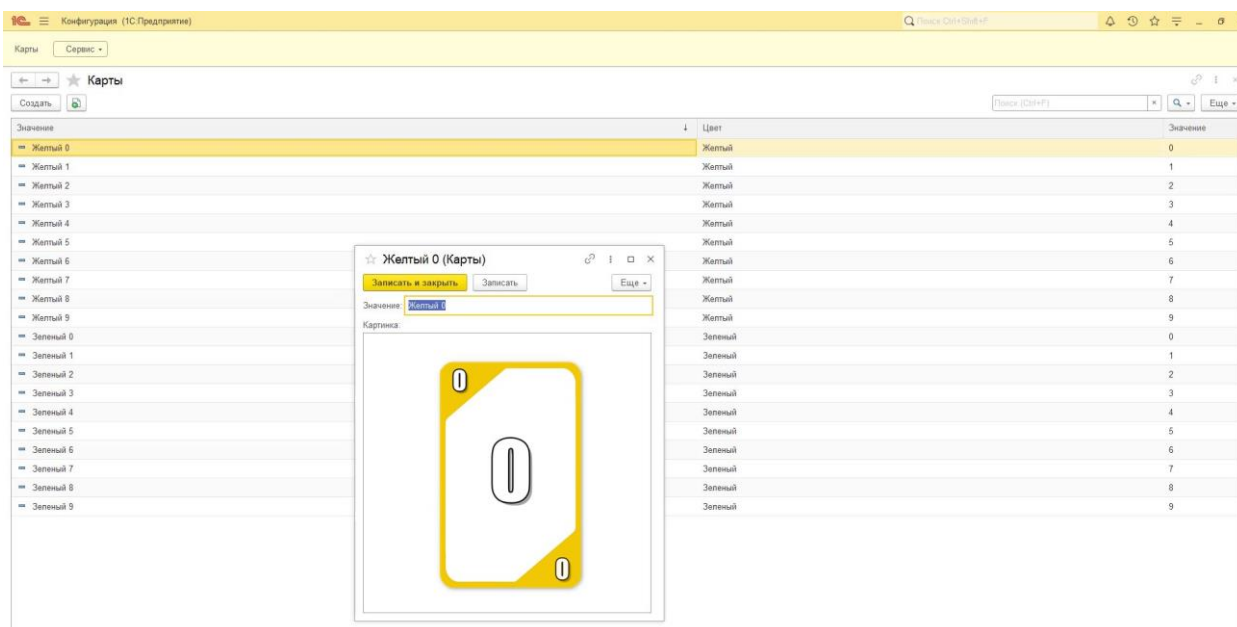


Рис. 1.5.1. Хранение карт

Перед этим откроем информационную базу с кубиками в режиме конфигуратора для продолжения разработки.

Открыв конфигуратор, добавим в систему справочник. **Справочники в 1С** хранят в информационной базе данные, имеющие одинаковую структуру и списочный характер. К примеру, список учителей, перечень товаров или, как в нашем случае, список карт.

Для того чтобы добавить справочник, нужно найти этот объект в дереве конфигурации, нажать правой кнопкой мыши по нему и выбрать «Добавить» (рис. 1.5.2).

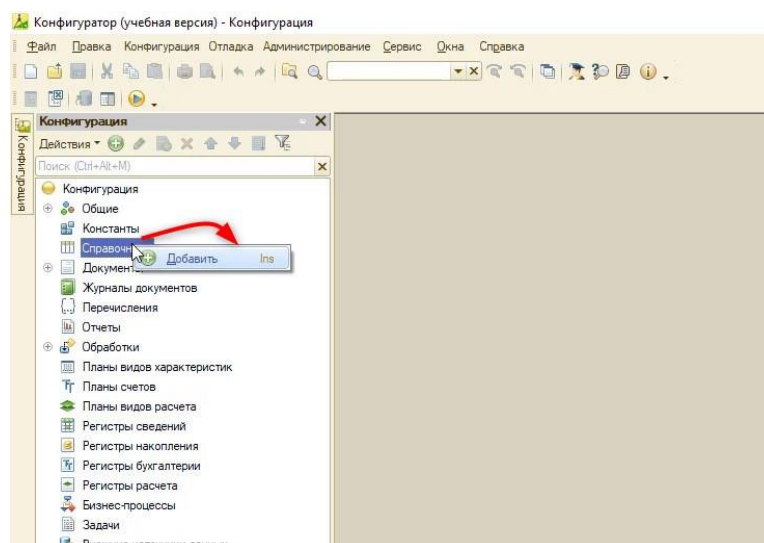


Рис. 1.5.2. Добавление справочника

Новый справочник назовём «СписокКарт» (рис. 1.5.2).

Справочник СписокКарт

Основные

Подсистемы

Функциональные опции

Иерархия

Владельцы

Данные

Нумерация

Формы

Поле ввода

Команды

Макеты

Ввод на основании

Права

Обмен данными

Прочее

Имя: СписокКарт

Синоним: Список карт

Комментарий:

Представление объекта:

Расширенное представление объекта:

Представление списка:

Расширенное представление списка:

Пояснение:

Действия <Назад Далее> Закреть Справка

Рис. 1.5.3. Наименование нового справочника

Запустим режим пользователя, сохранив все изменения.

В пользовательском режиме появляется новая команда для открытия справочника (рис. 1.5.4).

1С Конфигурация (1С:Предприятие, учебная версия)

Список карт Сервис

← → ☆ Список карт

Создать

Наименование

Рис. 1.5.4. Открытие справочника в пользовательском режиме

Для заполнения справочника нужно нажать кнопку «Создать» (рис. 1.5.5).

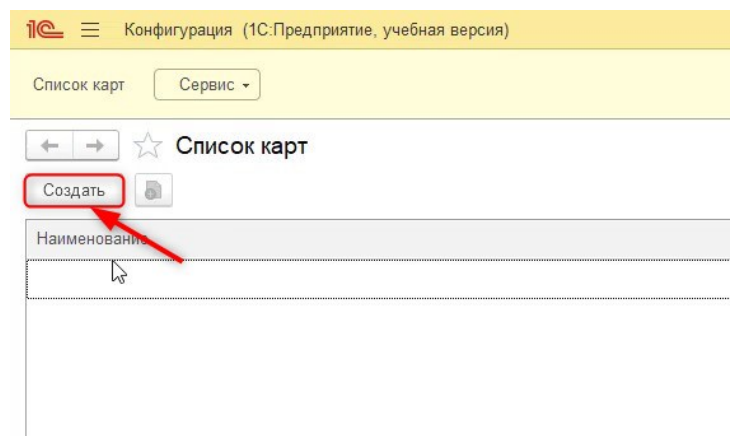


Рис. 1.5.5. Создание новой записи в справочнике

Изначально у справочника уже есть два поля – это «Код» и «Наименование» (рис. 1.5.6). Такие поля называют «Стандартные реквизиты». Они у каждого прикладного объекта (справочники, документы) разные.

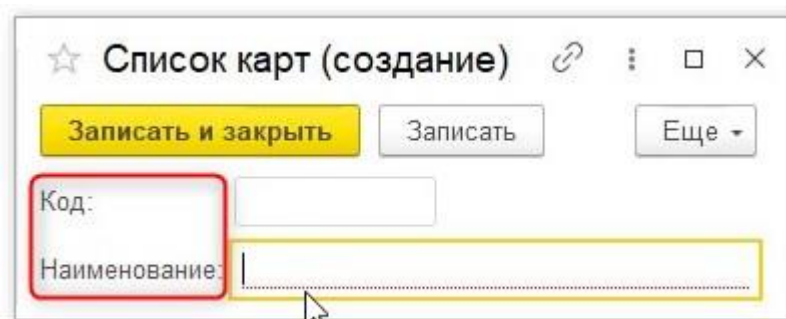


Рис. 1.5.6. Стандартные реквизиты справочника

Попробуем заполнить справочник картами «Синий 0», «Красный 0» (рис. 1.5.7).

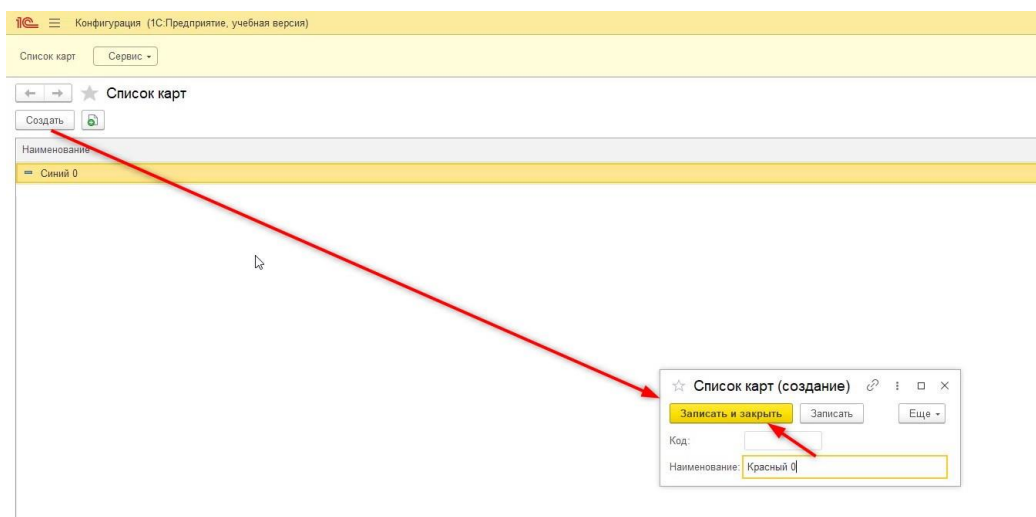


Рис. 1.5.7. Заполнение справочника

Помимо поля «Наименование», которое заполнялось вручную, есть поле «Код». Это поле формируется автоматически (стандартно от 0000000001 и далее). Интересно данное поле тем, что по нему можно производить контроль уникальности. Если задать два одинаковых кода в конкретном справочнике, то система выдаст ошибку, предварительно предупредив, что код заполняется автоматически (рис. 1.5.8).

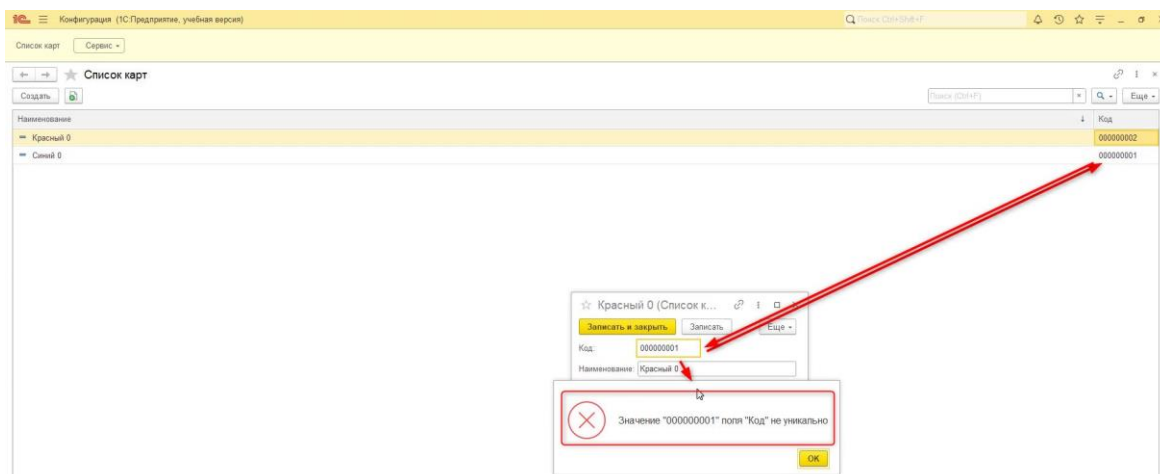


Рис. 1.5.8. Контроль уникальности

Такой механизм будет удобно использовать для контроля уникальности названий карт. Вместо числового типа в коде можно использовать строковый, а наименование — убрать.

Первоначально уберём стандартное поле «Наименование». Для этого перейдём в режим разработчика на вкладку «Данные» справочника с картами и укажем длину наименования — 0 (рис. 1.5.9).

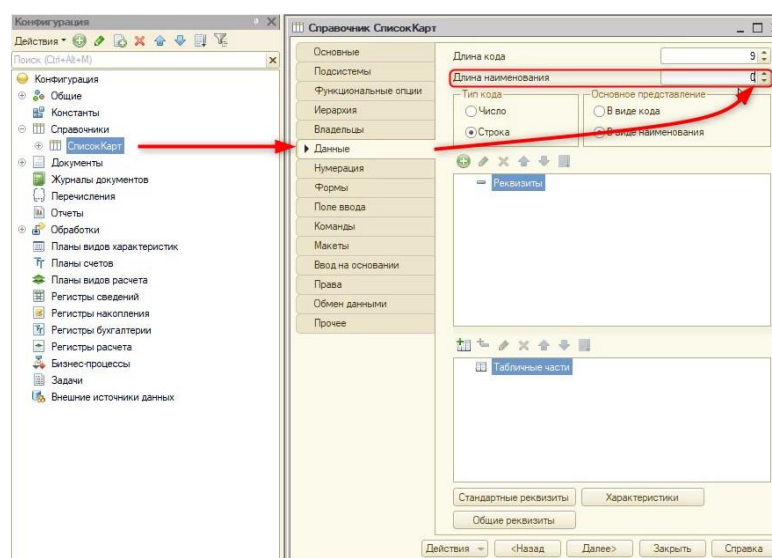


Рис. 1.5.9. Длина стандартного реквизита «Наименование»

Так как после этого останется только реквизит «Код», увеличим его длину до 50 символов (рис. 1.5.10).

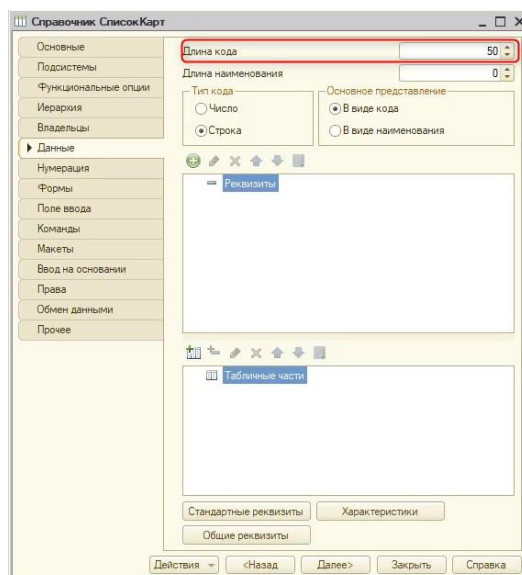


Рис. 1.5.10. Длина кода

Если на данном этапе попробовать сохранить конфигурацию баз данных, то программа выдает ошибку (см. рис. 1.5.11).

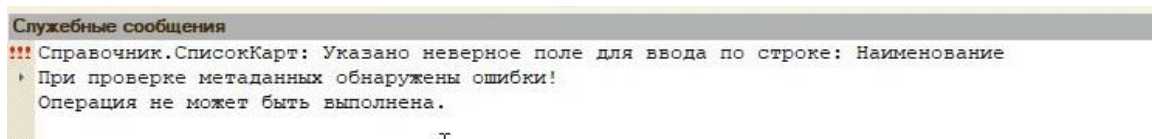


Рис. 1.5.11. Окно служебных сообщений

Связано это с тем, что элементы справочника могут участвовать в других прикладных объектах (справочник, документы и т. д.) системы. Поиск таких элементов может происходить по их названию, а оно на данный момент отключено. Соответственно, нужно отключить и поиск по наименованию. Сделать это можно на вкладке окна редактирования справочника «Поле ввода», в разделе «Ввод по строке» нужно исключить поле «Наименование» из списка (рис. 1.5.12).

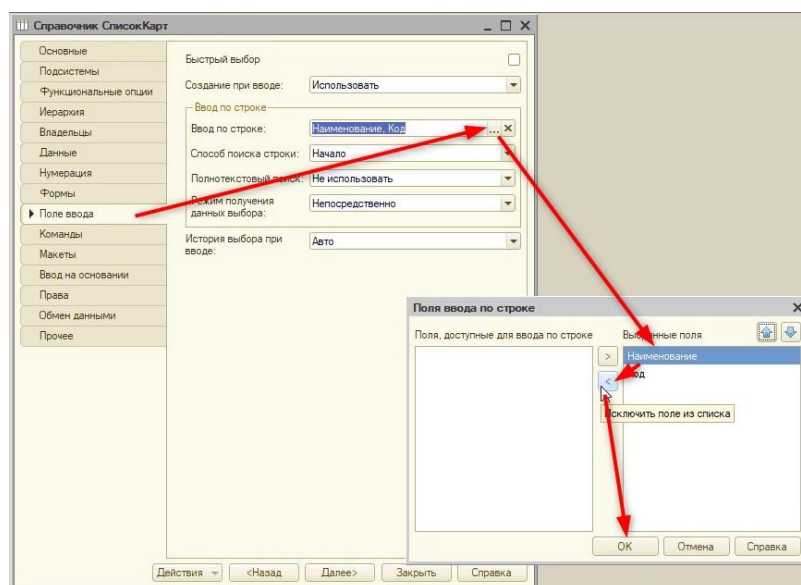


Рис. 1.5.12. Исправление ошибки

После этого можно будет сохранить конфигурацию базы данных и запустить пользовательский режим. Перед этим отключим оповещение, которое возникало ранее при редактировании поля «Код», чтобы справочник с картами заполнялся удобнее и быстрее (рис. 1.5.13).

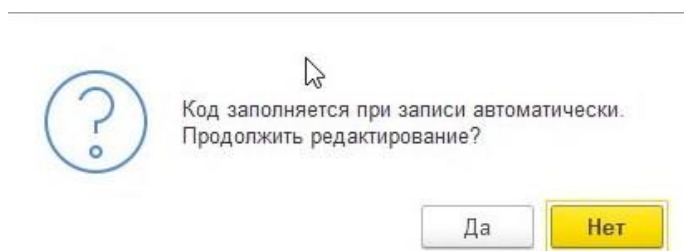


Рис. 1.5.13. Предупреждение автонумерации

Отключается автонумерация на вкладке редактирования справочника «Нумерация» (см. рис. 1.5.14).

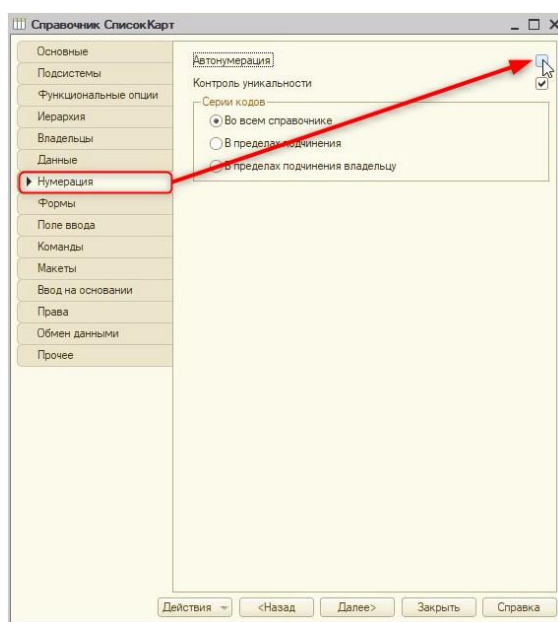


Рис. 1.5.14. Выключение автонумерации

Так как игроку может быть не понятно, что должно находиться в поле под названием «Код», изменим его наименование на «Значение». За этот заголовок отвечает синоним стандартного реквизита «Код». **Синоним** — это имя, которое будет отображаться в пользовательском режиме и будет видно пользователю.

Для того чтобы изменить синоним, нужно найти окно со стандартными реквизитами. Оно открывается по кнопке «Стандартные реквизиты» на вкладке «Данные» (рис. 1.5.15).

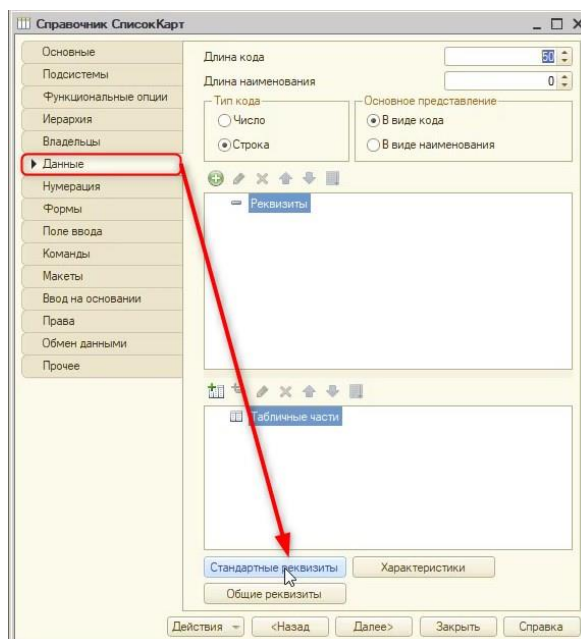


Рис. 1.5.15. Вкладка «Данные»

Далее осталось только указать синоним «Значение» у реквизита «Код» (рис. 1.5.16) и запустить пользовательский режим, предварительно всё сохранив.

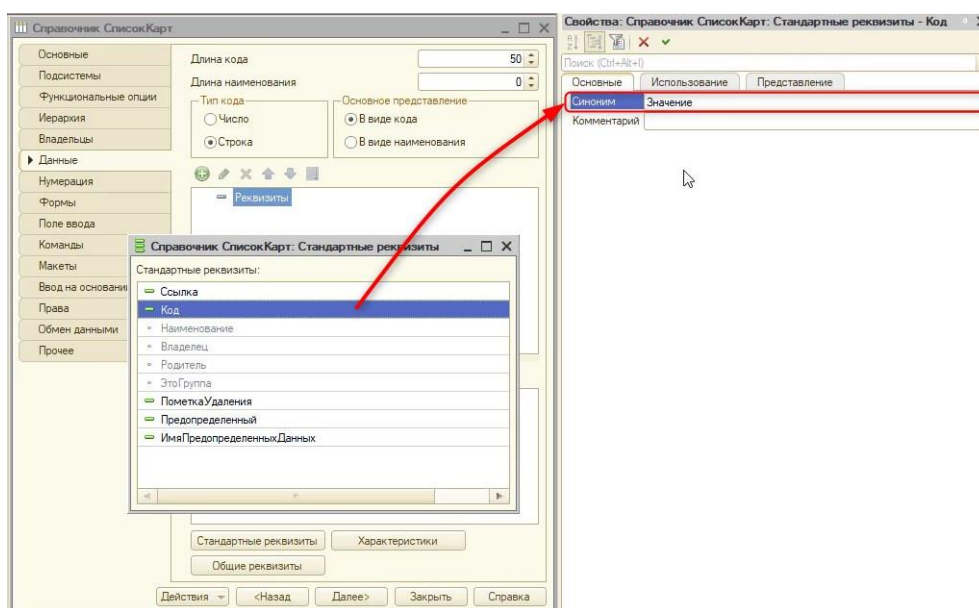


Рис. 1.5.16. Синоним у стандартного реквизита

Поменяем значения созданных ранее карт (рис. 1.5.17).

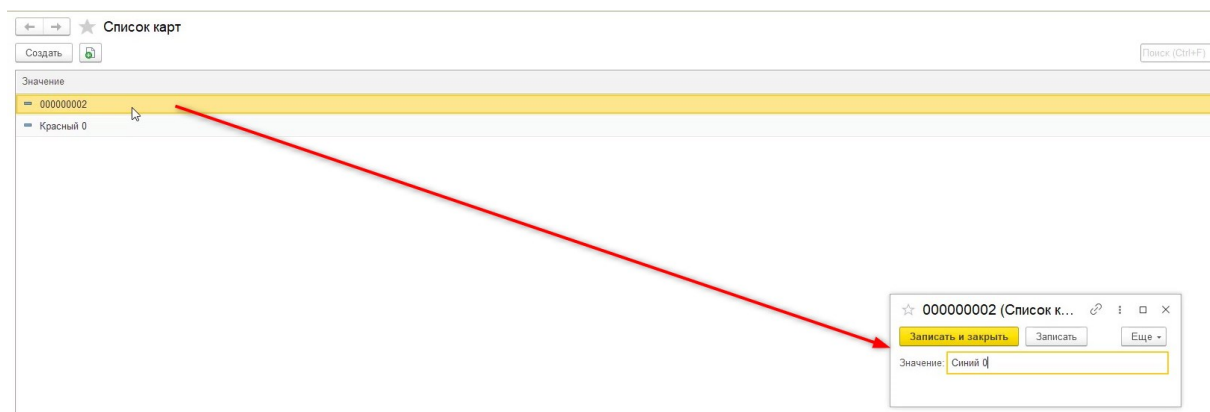


Рис. 1.5.17. Редактирование значений карт

Однако писать вручную названия карт не совсем удобно. Лучше, если бы появился анализ названия картинок, которые прикрепляет пользователь (рис. 1.5.18), и значение заполнялось автоматически.

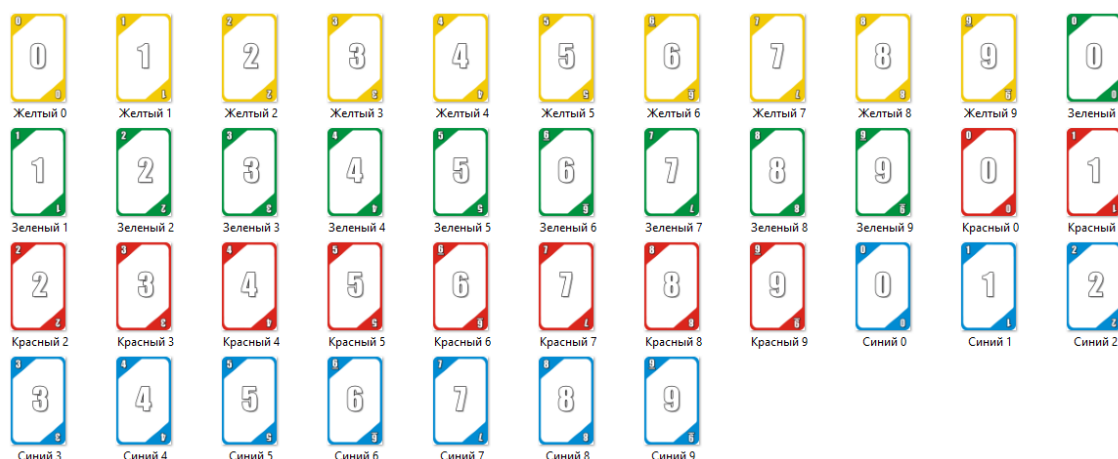


Рис. 1.5.18. Изображения карт

Чтобы программа анализировала картинки и подставляла их в справочник, потребуется сделать следующее:

- 1) Создать в справочнике новые поля, которые будут отдельно хранить цвет, числовое значение (в дальнейшем потребуется для игры) и картинку.
- 2) Реализовать загрузку карт.

Вернёмся в режим конфигулятора и на вкладке «Данные» добавим следующие реквизиты:

- 1) «Цвет», с типом «Строка» и длиной в 30 символов (рис. 1.5.19).

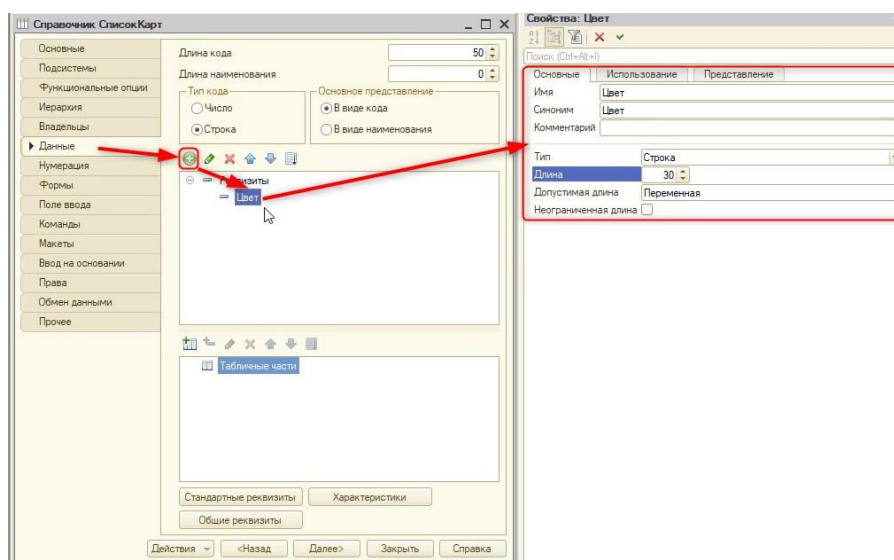


Рис. 1.5.19. Создание реквизита «Цвет»

- 2) «Значение», с типом «Строка» и длиной в 30 символов (рис. 1.5.20).

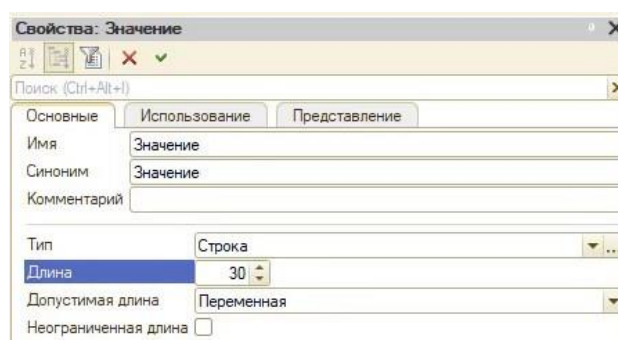


Рис. 1.5.20. Свойства реквизита «Значение»

- 3) «Картинка». Для того чтобы прикреплять в системе «1С:Предприятие» картинки, нужно использовать не примитивный тип данных — «Хранилище значения» (рис. 1.5.21).

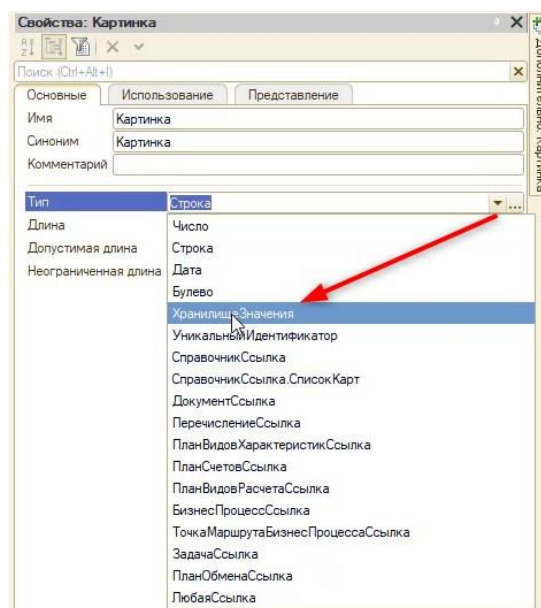


Рис. 1.5.21. Свойства реквизита «Картинка»

Примитивные типы данных — это типы Строка, Число, Дата, Булево и другие. Они не являются чем-то особенным для «1С:Предприятия 8». Как правило, такие типы данных существуют и в других программных системах.

Тип «Хранилище значений» позволяет сохранять прямо в базе различные данные о картинке, Word-документе и т. д. Для того чтобы отображать эту картинку, понадобится создать форму и новый реквизит на ней, так как само по себе «Хранилище значений» не может отображать изображение. Хранилище значения — это просто набор данных, без отображения.

Создаются формы на одноимённой вкладке в окне редактирования справочника. Отображать и загружать картинку программа должна будет на форме элемента (окно редактирования одной карты), поэтому при создании выберем её (рис. 1.5.22).

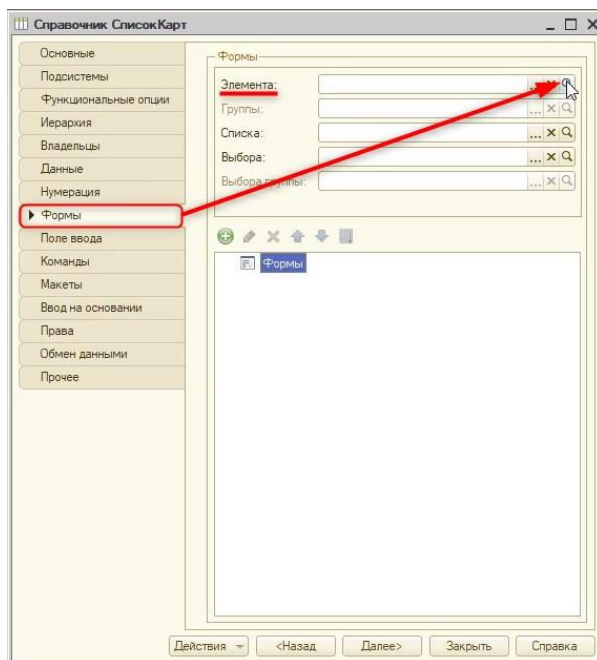


Рис. 1.5.22. Создание формы элемента справочника

В открывшемся окне ничего не меняем и нажимаем кнопку «Готово» (рис. 1.5.23).

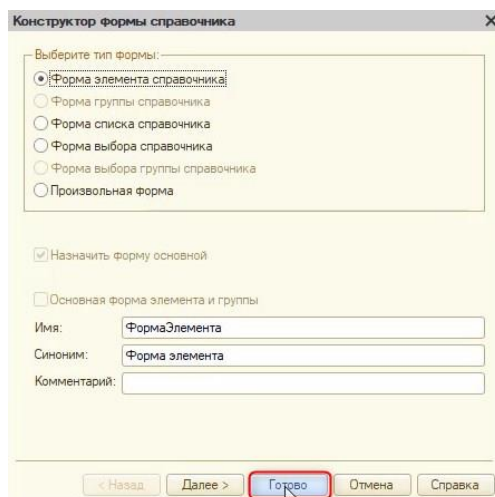


Рис. 1.5.23. Конструктор формы справочника

В появившемся окне редактирования формы первоначально уберём отображение элементов «Цвет» и «Значение», поскольку они нужны только для программного анализа. Для этого выберем элементы и нажмём на иконку «Крестик» (рис. 1.5.24).

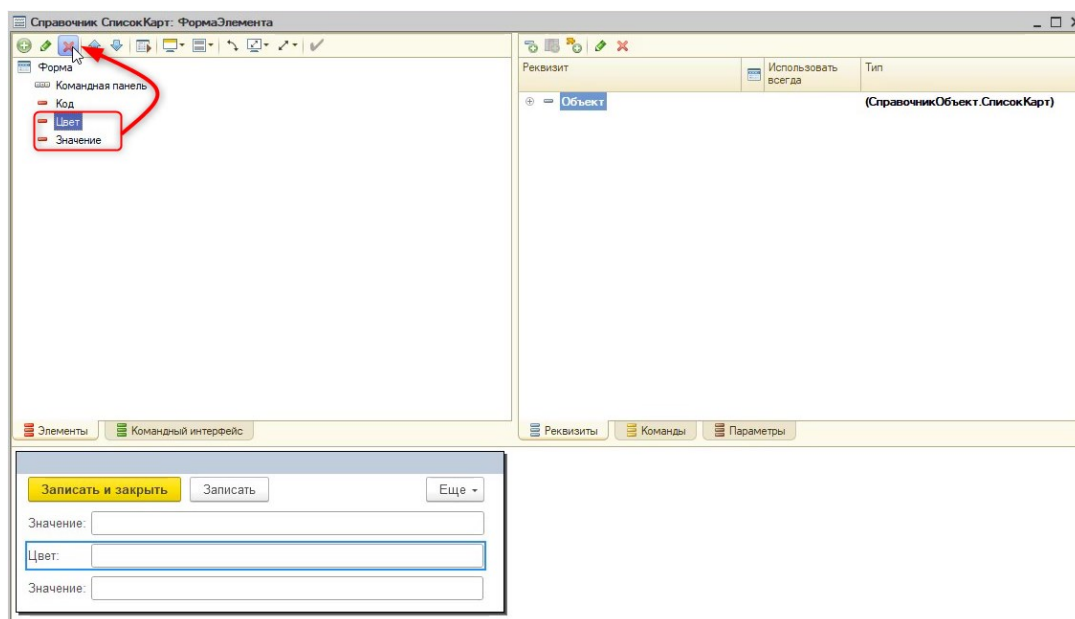


Рис. 1.5.24. Удаление элементов

Для того чтобы показать картинку пользователю, создадим реквизит на форме. Реквизит на форме не хранит значение, но будет показывать картинку, которая хранится в реквизите «Картинка». Новый реквизит назовём «КартинкаДляОтображения», назначим тип «Строка» и перенесём его на форму, удерживая левую кнопку мыши (рис. 1.5.25).

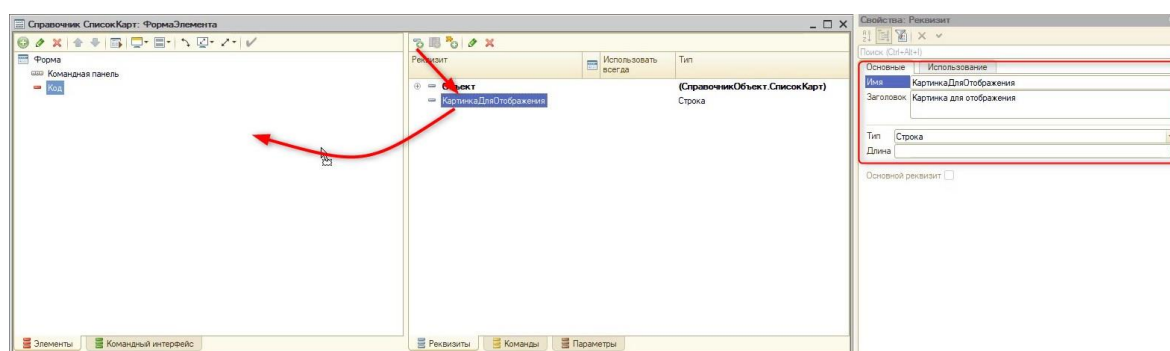


Рис. 1.5.25. Создание и перенос реквизита формы

Тип «Строка» задаётся новому реквизиту, потому что «ХранилищеЗначения» хранит в себе адрес (некий набор символов), и этот адрес можно трактовать как строку. Соответственно, необходимо проанализировать этот адрес и по нему получить картинку.

На данный момент элемент «Картинка для отображения» на форме выглядит как поле для текста. Для того чтобы можно было в него подставлять картинку, откроем свойства этого элемента и выберем вид «Поле картинки» (рис. 1.5.26).

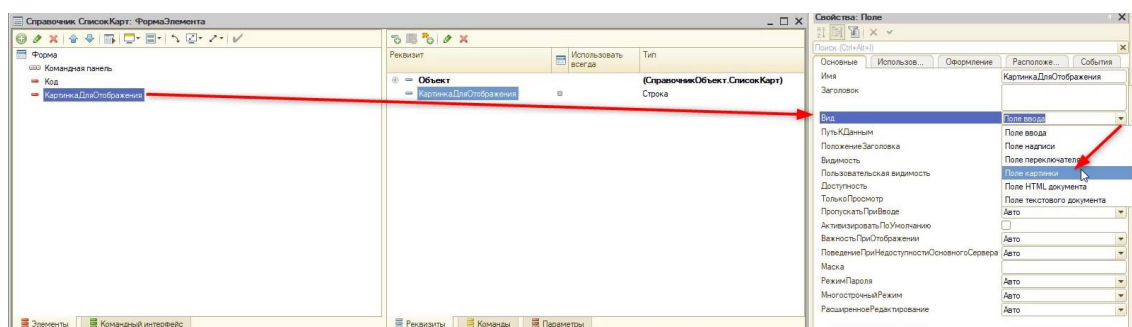


Рис. 1.5.26. Смена вида поля

Можно убрать заголовок, поскольку пользователю и так будет понятно, что это картинка. У элемента обратимся к свойству «Положение заголовка» и выберем «Нет» (рис. 1.5.27).

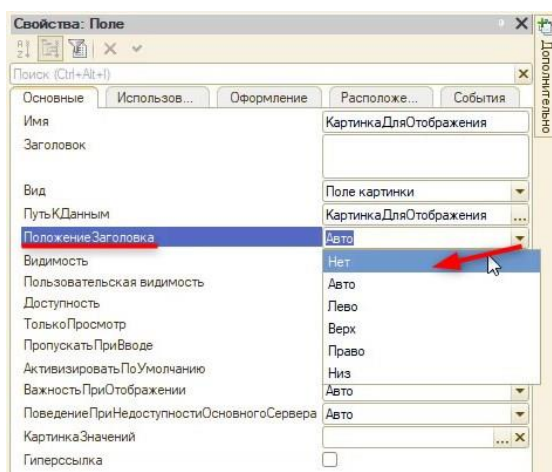


Рис. 1.5.27. Отключение заголовка

Далее, для того чтобы на поле невыбранной картинки можно было нажать (загрузить картинку), включим свойство «Гиперссылка» (рис. 1.5.28).

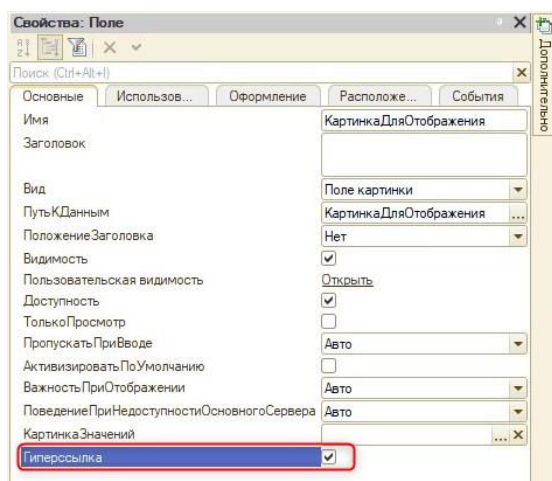


Рис. 1.5.28. Включение гиперссылки

Оставим подсказку пользователю («Нажмите для выбора картинки») с помощью свойства «ТекстНевыбраннойКартинки» (рис. 1.5.29).

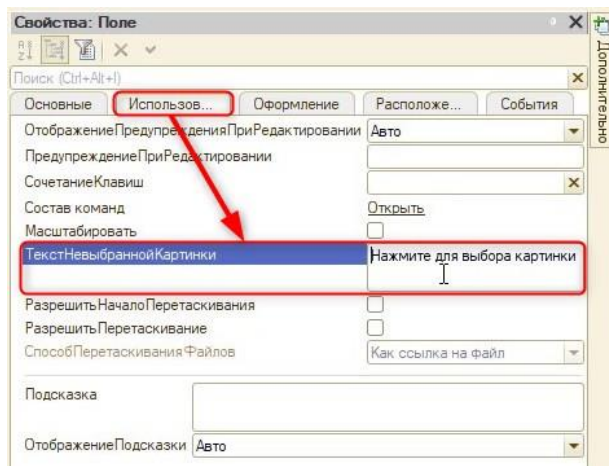


Рис. 1.5.29. Сообщение пользователю

Если сейчас проверить результат в режиме пользователя, то получится следующее (см. рис. 1.5.30).

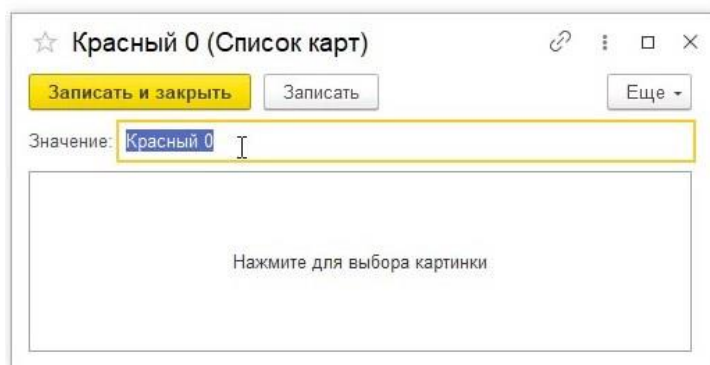


Рис. 1.5.30. Результат в режиме пользователя

При нажатии на поле с картинкой возникнет сообщение с предупреждением (см. рис. 1.5.31), в дальнейшем его понадобится отключить.

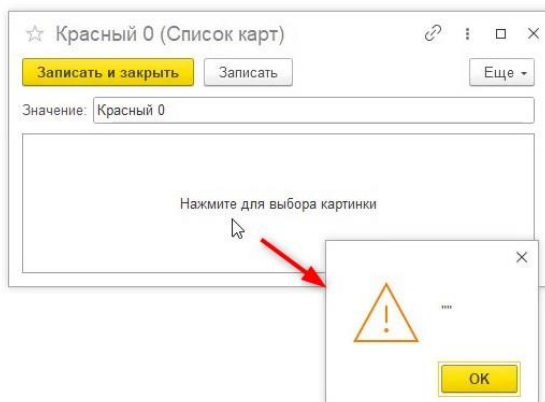


Рис. 1.5.31. Предупреждение при нажатии

Следующее, что необходимо будет сделать — это описать событие для нажатия по невыбранной картинке. Для того чтобы это сделать, откроем свойства элемента «КартинкаДляОтображения», перейдём на вкладку «События» и выберем событие «Нажатие». Чтобы описать это событие, необходимо нажать на кнопку в виде лупы (рис. 1.5.32).

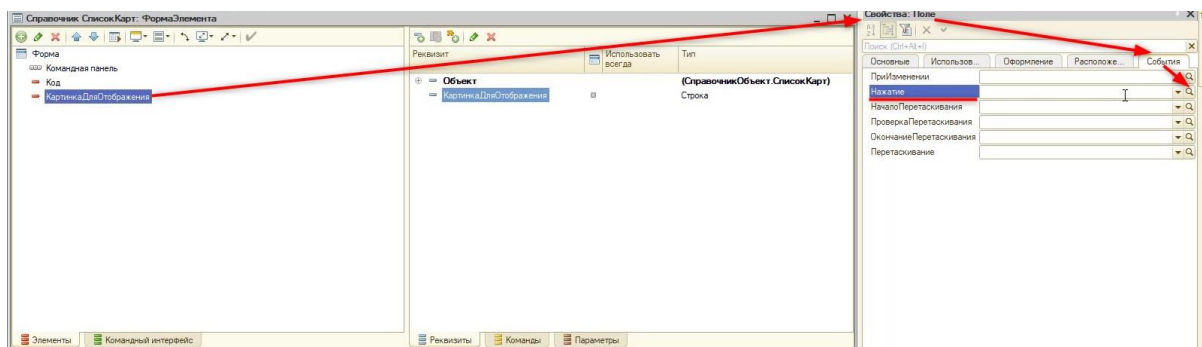


Рис. 1.5.32. События элемента

Обработчик события будет происходить на клиенте, поэтому в новом окне оставляем «Создать на клиенте». После этого программа откроет модуль формы, и мы приступим к написанию кода:

&НаКлиенте

Процедура КартинкаДляОтображенияНажатие(Элемент, СтандартнаяОбработка)

//Первоначально уберем окно с предупреждением, которое возникало при нажатии на картинку:

СтандартнаяОбработка = Ложь;

//Необходимо начать процесс помещения файла из папки в 1С

//Для этого есть метод НачатьПомещениеФайла

//Первым параметром этого метода является <ОписаниеОповещенияОЗавершении>

//Он содержит описание процедуры, которая будет вызвана после завершения помещения файла и имеет тип ОписаниеОповещения

//Предварительно создадим переменную для этого параметра:

// ОписаниеОповещения(ИмяПроцедуры, Модуль,...)

//ИмяПроцедуры - процедура, которая проанализирует картинку. Назовём её "ОбработатьВыборФайла"

//Модуль - где расположена эта процедура. Так как она будет в этом же модуле укажем "ЭтотОбъект"

Оповещение = Новый ОписаниеОповещения("ОбработатьВыборФайла", ЭтотОбъект);

//НачатьПомещениеФайла(<ОписаниеОповещенияОЗавершении>, <Адрес>, <ПомещаемыйФайл>,

// <Интерактивно>, <УникальныйИдентификаторФормы>, <ОписаниеОповещенияПередНачаломПомещенияФайла>)

//Если параметр не задействуется, то он пропускается и ставится пробел и запятая.

//Для данного алгоритма нужны только три параметра:

// <ОписаниеОповещенияОЗавершении>

// <Интерактивно> - Откроет нам проводник для выбора картинки (Истина)

// <УникальныйИдентификаторФормы> - ссылка на эту форму

НачатьПомещениеФайла(Оповещение, , , Истина, УникальныйИдентификатор);

КонецПроцедуры

//Далее опишем процедуру, указанную Выше:

//Эта процедура будет анализировать, выбрал пользователь картинку или нет

&НаКлиенте

//Процедура должна называться идентично, как было указано выше в двойных кавычках.

//Для такой процедуры нужны параметры: Результат, Адрес, ПомещаемыйФайл, ДополнительныеПараметры

//(Описаны в справке метода НачатьПомещениеФайла)

Процедура ОбработатьВыборФайла(Результат, Адрес, ПомещаемыйФайл, ДополнительныеПараметры) Экспорт

//Проведем анализ выбрал пользователь картинку или нет

//Если картинка не выбрана то параметр Результат будет равен значению "Ложь"

Если Результат = Ложь Тогда

//Пользователь не выбрал картинку, прерываем выполнение процедуры:

Возврат;

КонецЕсли;

КонецПроцедуры

Проверим, что получилось в режиме пользователя.

При нажатии на окно с картинкой программа выдаст ошибку (рис. 1.5.33). Когда возникает такая ошибка, она адресована разработчику решения, и в пользовательском режиме не показывается её подробное описание.



Рис. 1.5.33. Ошибка в пользовательском режиме

Для разработчиков есть специальный режим «Отладка». В дальнейшем рекомендуется при разработке игр, в случае возникновения ошибки, запускать именно этот режим. Открыть его можно через пункт главного меню «Отладка» в конфигураторе (рис. 1.5.34).

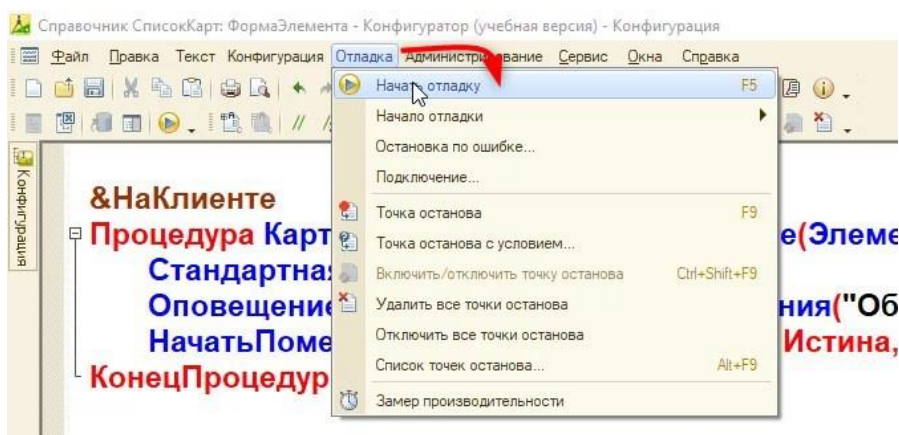


Рис. 1.5.34. Открытие режима отладки

Теперь, если повторить аналогичные действия, ошибка будет выглядеть иначе (рис. 1.5.35)

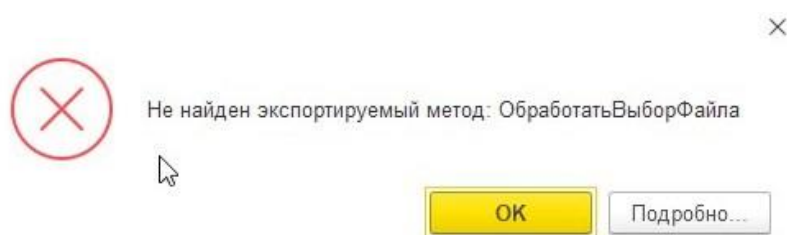


Рис. 1.5.35. Ошибка в режиме отладки

Нажмём кнопку «Подробнее...» в окне с ошибкой и далее на кнопку «Конфигуратор» (рис. 1.5.36), чтобы перейти на строку в конфигураторе, где случилась ошибка.

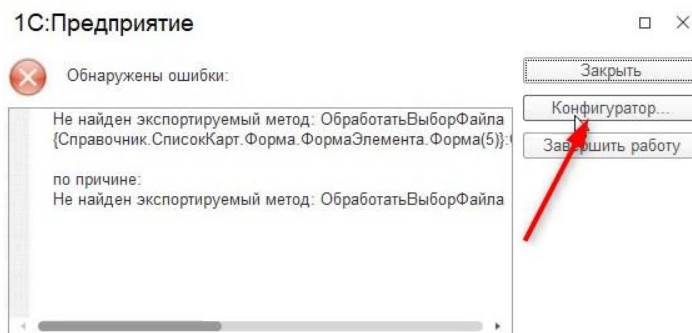


Рис. 1.5.36. Открытие ошибки в конфигураторе

С помощью режима отладки можно намного быстрее найти местоположение ошибки (рис. 1.5.37).

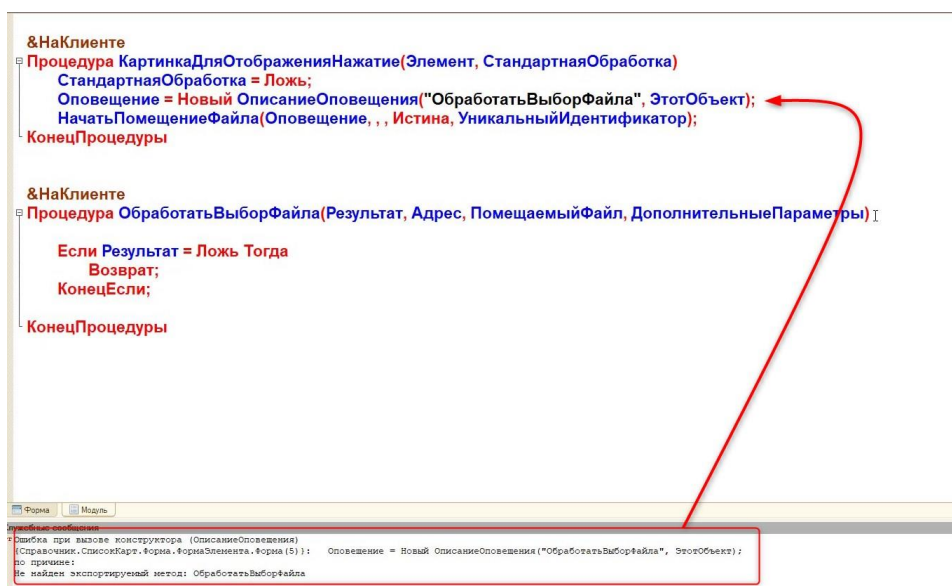


Рис. 1.5.37. Открытие ошибки в конфигураторе

Ошибка возникает из-за того, что программа не может найти процедуру «ОбработатьВыборФайла». В таком случае её нужно сделать экспортной. Для этого нужно после указания параметров в шапке процедуры указать «Экспорт» (рис. 1.5.38).

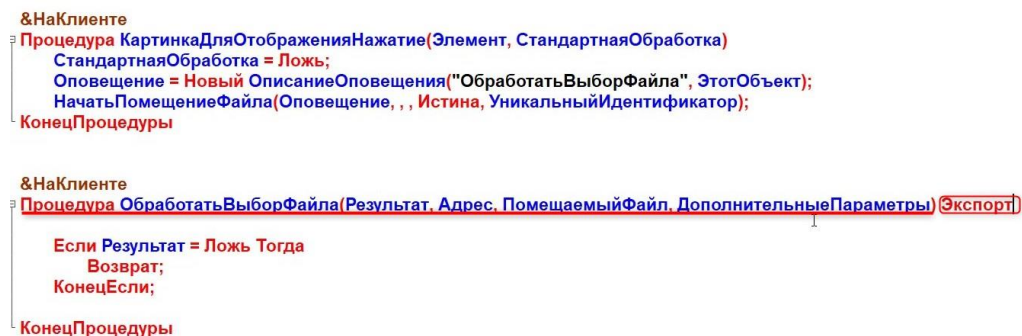


Рис. 1.5.38. Добавление модификатора «Экспорт»

Обновим конфигурацию баз данных и запустим пользовательский режим. Теперь при нажатии на окно с картинкой откроется проводник, через который можно выбрать изображение карты (рис. 1.5.39).

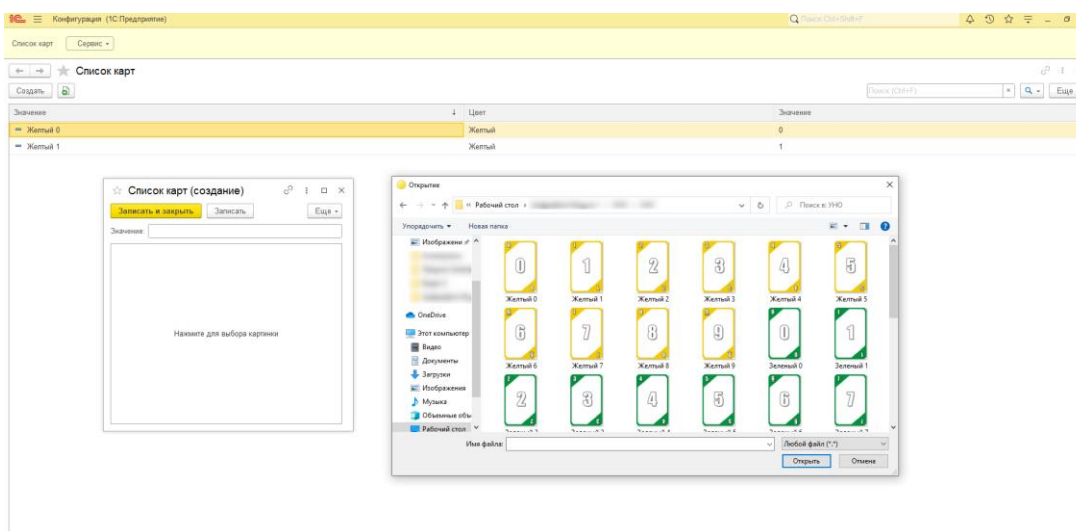


Рис. 1.5.39. Загрузка картинки

Проверив доступный функционал, вернёмся в режим конфигулятора.

Для обработки результата нужно выяснить, в каком из четырех параметров хранится адрес картинки. Сделать это можно с помощью отладки и точки останова. **Точка останова** — это точка, останавливающая выполнение кода программы и производящая вызов отладки в том месте, в котором она стоит.

Поставим точку останова в конце процедуры, для этого нажмём в начале строки два раза левой кнопкой мыши (рис. 1.5.40). Если всё сделано правильно, то на этом месте появится красная точка.

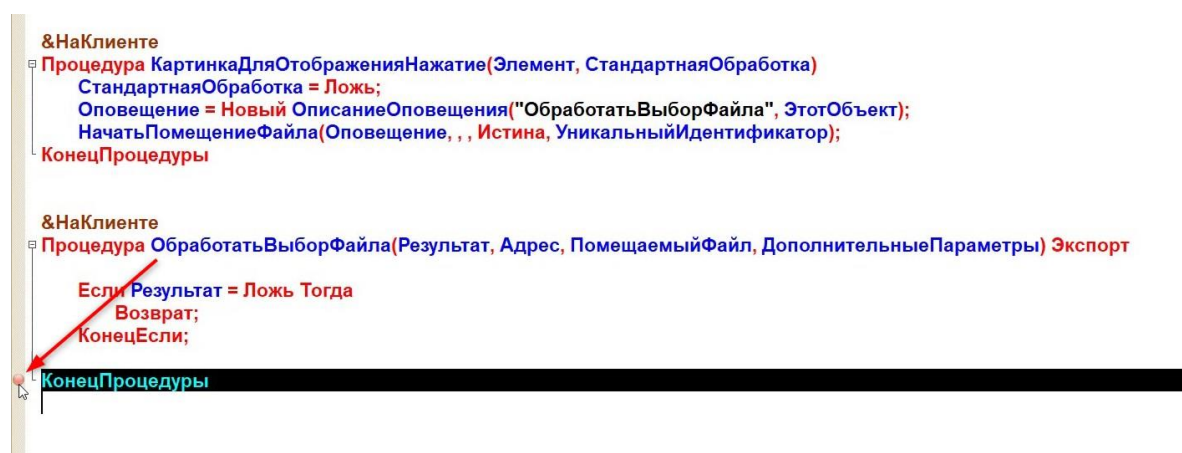


Рис. 1.5.40. Точка останова

Точки останова будут работать только в режиме «Отладка». Поэтому запустим режим отладки через горячую клавишу «F5» или пункт главного меню «Отладка» — «Начать отладку» и выберем файл картинки для загрузки в справочнике с картами.

Если точка останова поставлена верно, после выбора файла картинки программа перенесёт нас обратно в модуль формы в конфигураторе, и иконка точки останова изменится (рис. 1.5.41).

КонецЕсли;

//Обработаем ситуацию, когда пользователь выбрал картинку

//Запишем полученный адрес картинки:

ЭтотОбъект.КартинкаДляОтображения = Адрес;

//Так как мы изменяем нашу форму, это нужно зафиксировать

//указав параметр Модифицированность в положение Истина

Модифицированность = Истина;

//Обработаем параметр "Помещаемый файл" и обрежем все так, чтобы осталось

//Только "Цвет Номинал", к примеру "Красный 0"

//Найдём в строке последний знак "/" После которого и идёт название файла с помощью

метода

//СтрНайти(<Строка>, <СтрокаПоиска>, <НаправлениеПоиска>):

СимволНачалаНазвания = СтрНайти(ПомещаемыйФайл, "\",
НаправлениеПоиска.СКонца);

//СимволНачалаНазвания получил порядковый номер последнего слэша в строке

//Вырежем правую часть строки так, чтобы в него не попал символ "/". Добавим к СимволНачалаНазвания ещё один символ

//Вырезать будем с помощью метода Сред(<Строка>, <НачальныйНомер>)

ИмяКартинки = Сред(ПомещаемыйФайл, СимволНачалаНазвания + 1);

//На данный момент ИмяКартинки выглядит как Красный 0.png

//Найдём аналогично порядковый номер "."

СимволТочки = СтрНайти(ИмяКартинки, ".");

//И вырежим левую часть с расширением

ИмяКартинкиБезРасширения = Лев(ИмяКартинки, СимволТочки - 1);

//Присвоим то, что получилось реквизиту "Код"

Объект.Код= ИмяКартинкиБезРасширения;

КонецПроцедуры

Проверим работу механизма в пользовательском режиме. Выберем картинку игровой карты.

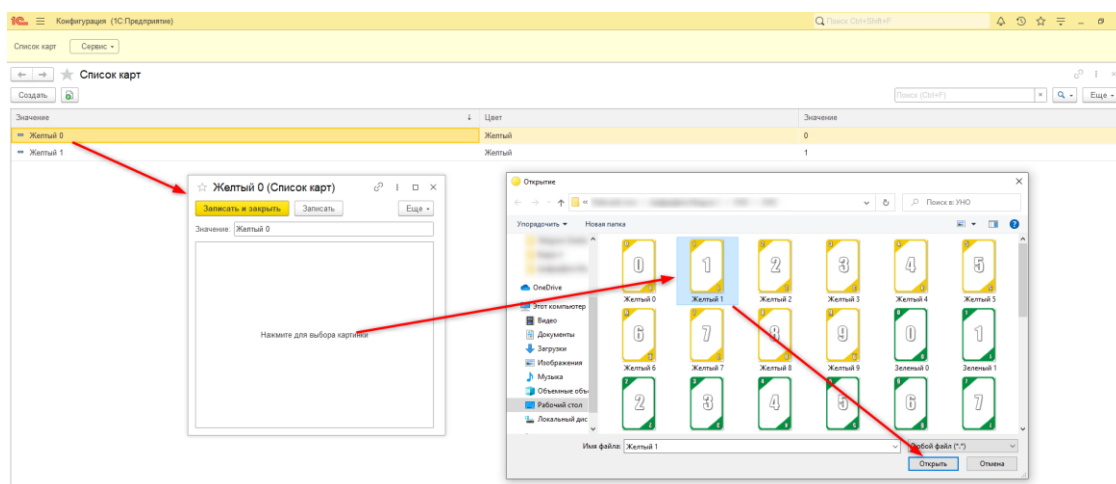


Рис. 1.5.44. Загрузка картинки

При загрузке картинка вставится немного некорректно (рис. 1.5.45), также не запишутся реквизиты «Цвет» и «Значение», которые нужны нам для дальнейшего анализа.



Рис. 1.5.45. Загрузка картинки

Кроме того, если закрыть окно с картой и снова открыть, то картинка пропадёт. Ещё нужно добавить в код запись картинки.

Вернёмся обратно в конфигуратор и сначала настроим отображение картинки на форме. Для этого перейдём к конструктору формы (рис. 1.5.46).

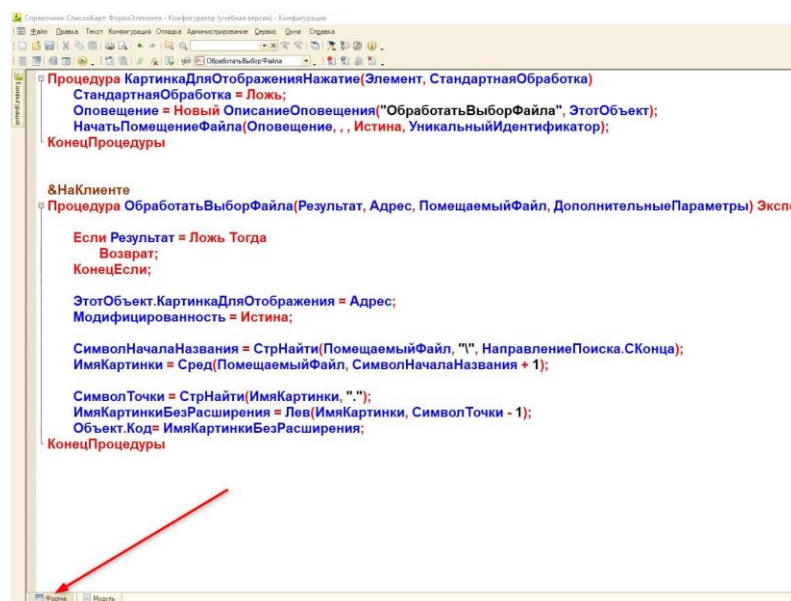


Рис. 1.5.46. Переключение из модуля на форму

И откроем свойства элемента «КартинкаДляОтображения». В свойствах на вкладке «Оформление» нужно указать размер картинки — «Пропорционально» (рис. 1.5.47).

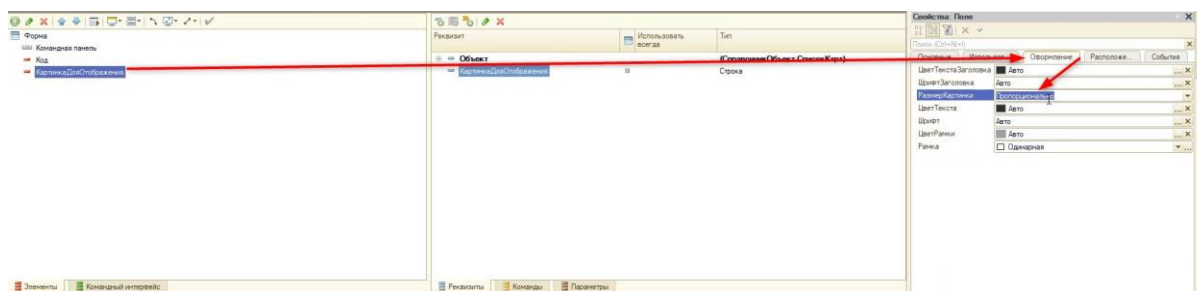


Рис. 1.5.47. Включение пропорционального отображения

Далее нужно определить ещё несколько событий, которые связаны с открытием формы и событием перед записью. В данный момент изображение хранится только в оперативной памяти (т. е. при закрытии формы все данные о ней стираются) и не записывается в реквизит с типом «ХранилищеЗначения». Для того чтобы запись происходила, обратимся к свойствам формы на вкладке «События» и создадим новую процедуру «ПередЗаписьюНаСервере» (рис. 1.5.48).

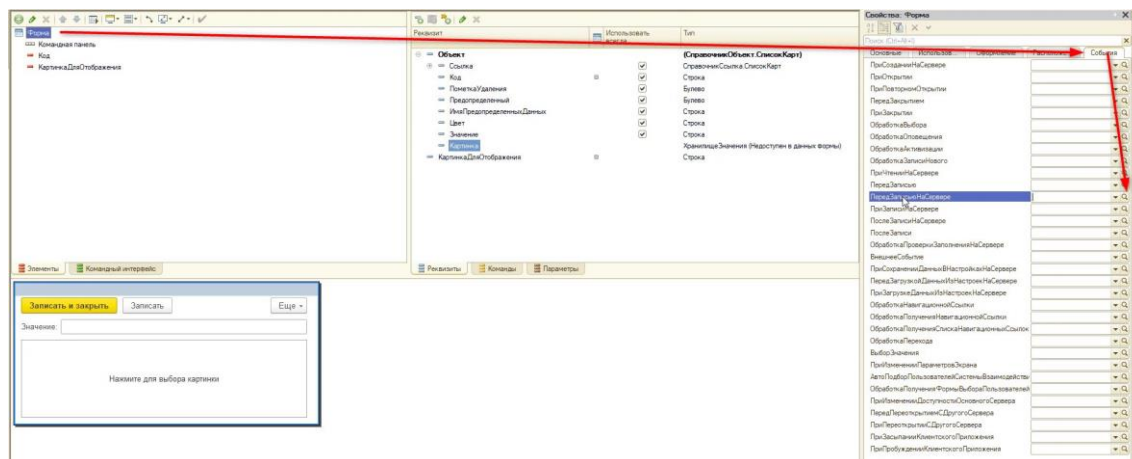


Рис. 1.5.48. События формы

Перед записью на сервере необходимо проверить, что реквизит «КартинкаДляОтображения», в которую подставляется путь, — это адрес временного хранилища. Далее, если условие соблюдается, запишем её в реквизит «Картинка»:

```
&НаСервере
Процедура ПередЗаписьюНаСервере(Отказ, ТекущийОбъект, ПараметрыЗаписи)
Если ЭтоАдресВременногоХранилища(ЭтотОбъект.КартинкаДляОтображения) Тогда
    ТекущийОбъект.Картинка=Новый ХранилищеЗначения
    (ПолучитьИзВременногоХранилища(ЭтотОбъект.КартинкаДляОтображения));
КонецЕсли;
КонецПроцедуры
```

Недостаточно записать значение: нужно при открытии на форме его проверить. Для того чтобы производить такую проверку, потребуется создать обработчик события «ПриСозданииНаСервере» (рис. 1.5.49).

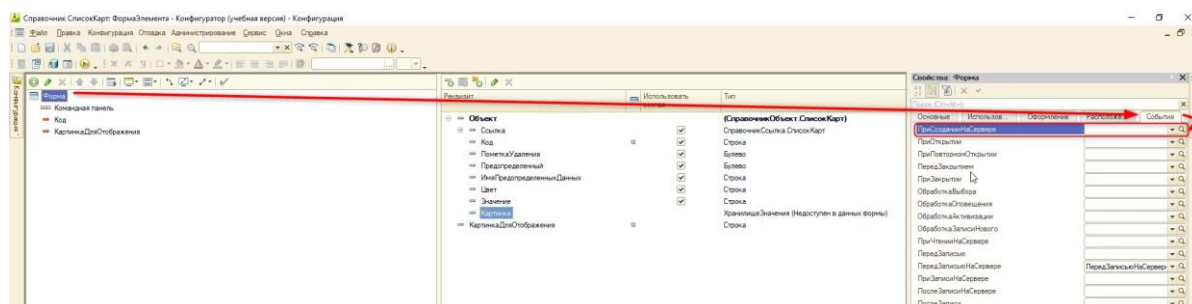


Рис. 1.5.49. События формы

Выбор события «ПриСозданииНаСервере» обусловлен тем, что при открытии карточки создаётся форма. Обрабатываем наш реквизит «КартинкаДляОтображения» в новом обработчике:

```
&НаСервере
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)
```

ЭтотОбъект.КартинкаДляОтображения = ПолучитьНавигационнуюСсылку(Объект.Ссылка, "Картинка"); КонецПроцедуры

В результате картинка будет и прикрепляться, и записываться.

Последнее, что необходимо сделать — это заполнить реквизиты «Цвет» и «Значение». Для этого допишем программный код в процедуре «ОбработатьВыборФайла»:

&НаКлиенте Процедура ОбработатьВыборФайла(Результат, Адрес, ПомещаемыйФайл, ДополнительныеПараметры) Экспорт Если Результат = Ложь Тогда Возврат; КонецЕсли; ЭтотОбъект.КартинкаДляОтображения = Адрес; Модифицированность = Истина; СимволНачалаНазвания = СтрНайти(ПомещаемыйФайл, "\", НаправлениеПоиска.СКонца); ИмяКартинки = Сред(ПомещаемыйФайл, СимволНачалаНазвания + 1); СимволТочки = СтрНайти(ИмяКартинки, "."); ИмяКартинкиБезРасширения = Лев(ИмяКартинки, СимволТочки - 1); Объект.Код= ИмяКартинкиБезРасширения; <i>//Разделим цвет и номинал на два элемента с помощью метода СтрРазделить(<Строка>, <Разделитель>, <ВключатьПустые>)</i> <i>//Разделителем будет в данном случае пробел - " "</i> МассивДанных = СтрРазделить(ИмяКартинкиБезРасширения, " ", Ложь); <i>//Отдельно запишем цвет и значение</i> <i> //(Идексация массива всегда начинается с 0)</i> <i> //Поэтому Цвет будет под индексом 0</i> <i> //А Значение под индексом 1</i> Объект.Цвет = МассивДанных.Получить(0); Объект.Значение = МассивДанных.Получить(1); КонецПроцедуры

Теперь в пользовательском режиме при загрузке картинки будут записываться реквизиты «Цвет» и «Значение». А также будет происходить запись картинки в реквизит. Значит, функционал справочника готов, и можно добавить все нужные карты.

Занятие 6 – Карточная игра. Перед началом игры

После того, как мы создали механизм по добавлению карт в программу, необходимо создать форму для игрового поля. Как и в случае с кубиками, будет использоваться обработка.

Для начала создадим новую обработку (рис. 1.6.1) и назовём её «Карты» (рис. 1.6.2).

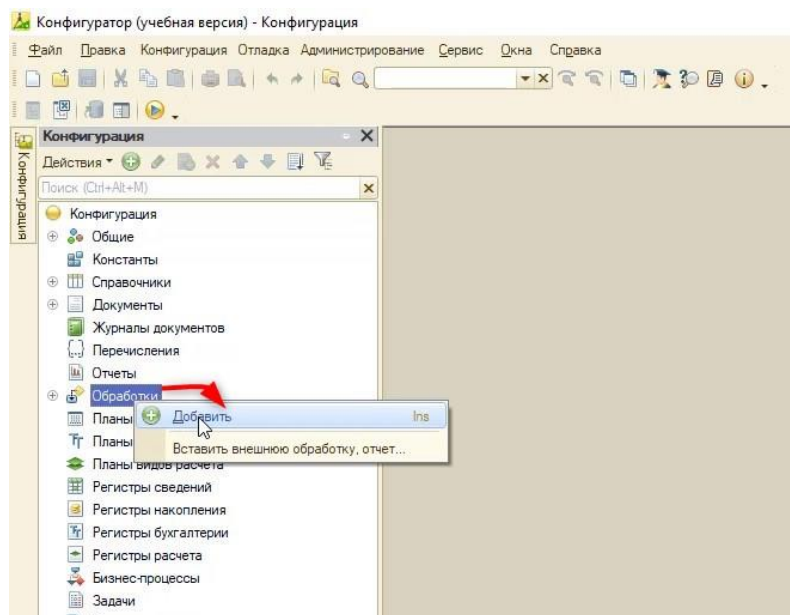


Рис. 1.6.1. Создание обработки

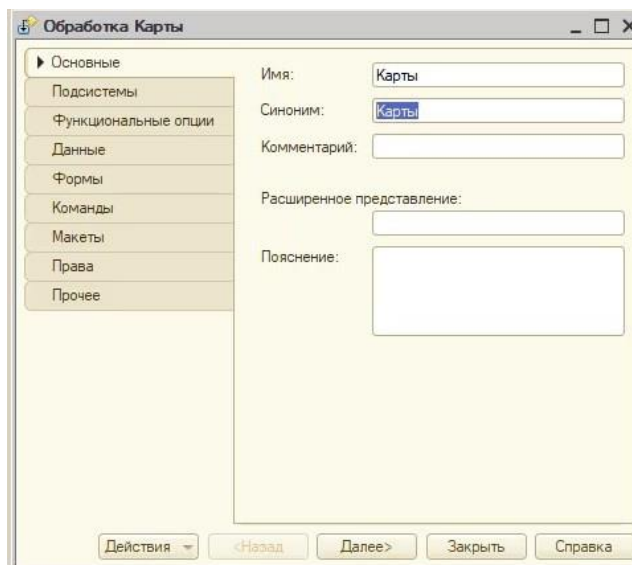


Рис. 1.6.2. Обработка «Карты»

Для того чтобы обработка отображалась в интерфейсе у игрока, нужно создать форму. На вкладке редактирования «Формы» добавим форму обработки (рис. 1.6.3).

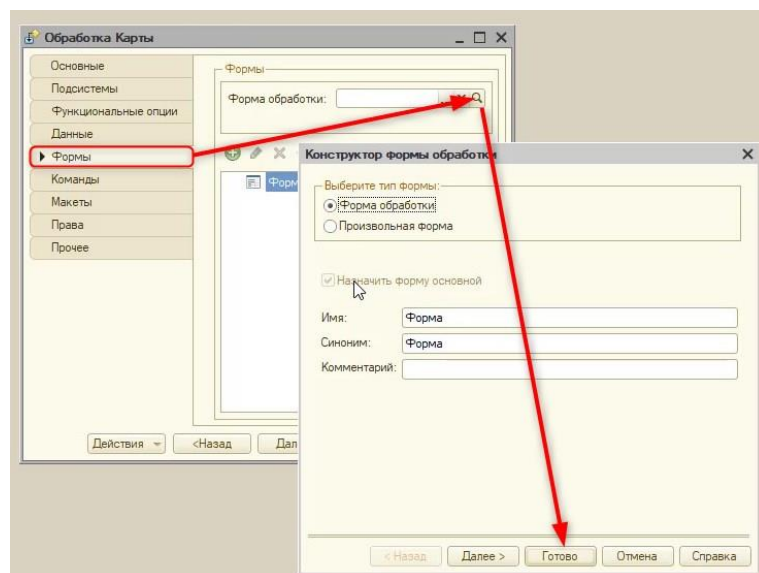


Рис. 1.6.3. Создание формы обработки

Появляется окно редактирования формы, в котором нам потребуется сделать следующее:

- 1) Создать механизм хранения колоды.
- 2) Перемешать случайным образом колоду перед игрой.

Добавим новый реквизит формы «Колода» и укажем тип данных «ТаблицаЗначений» (рис. 1.6.4).

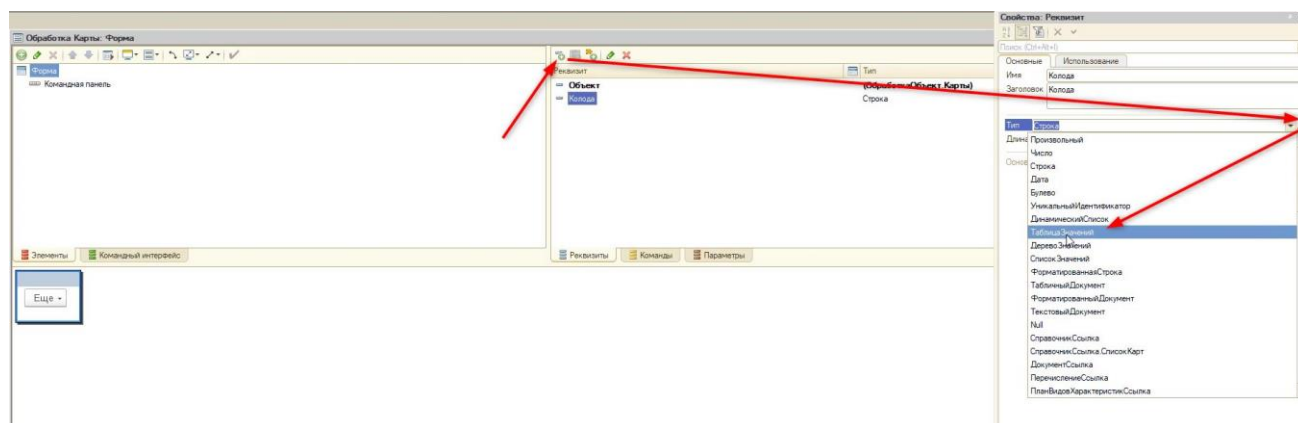


Рис. 1.6.4. Создание формы обработки

Выбор типа данных «ТаблицаЗначений» обусловлен тем, что необходимо хранить номинал (от 0 до 9) и цвет карты по отдельности для дальнейшего анализа.

Затем с помощью правой кнопки мыши (рис. 1.6.5) добавим две колонки реквизита «Цвет» и «Значение».

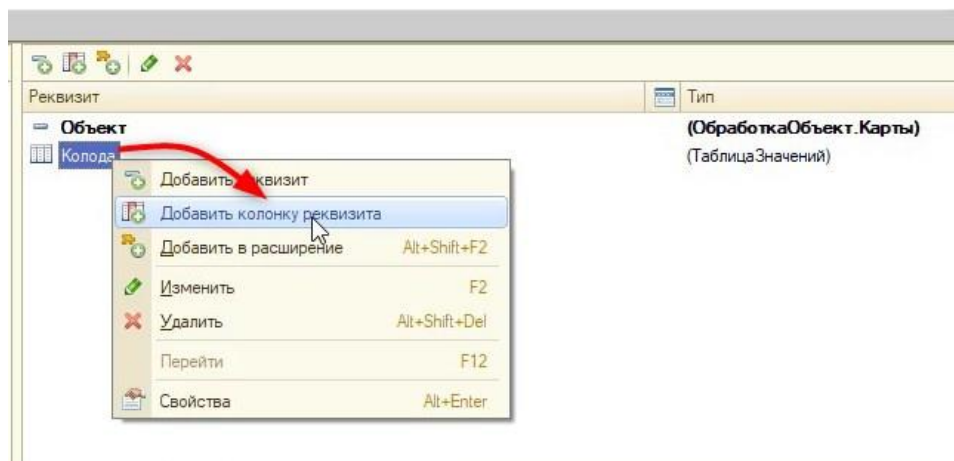


Рис. 1.6.5. Добавление колонки реквизита

Первая и вторая колонка будут с типом строка (рис. 1.6.6-1.6.7).

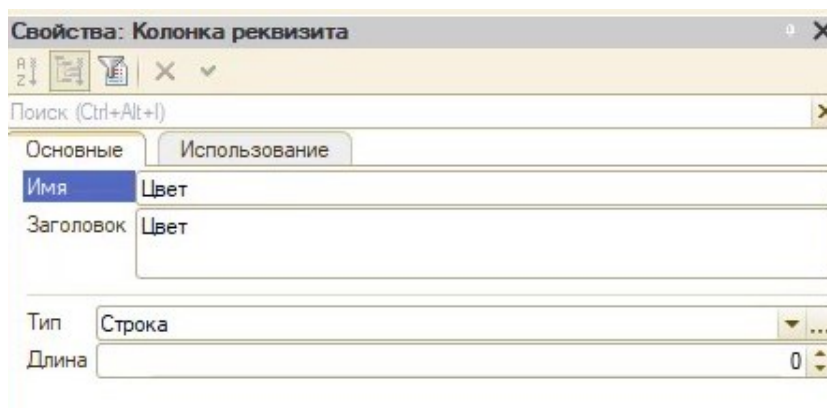


Рис. 1.6.6. Колонка «Цвет»

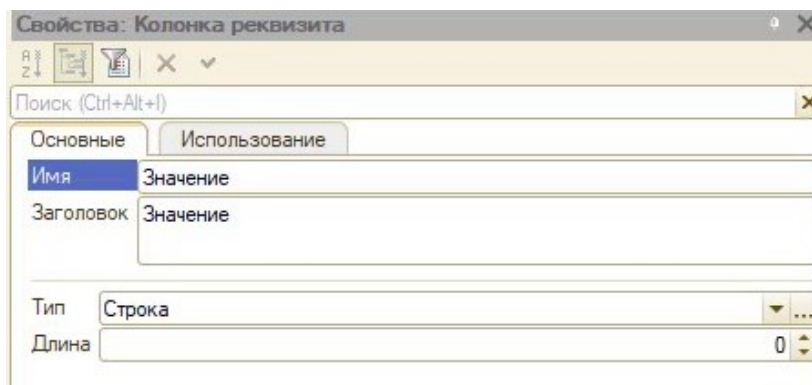


Рис. 1.6.7. Колонка «Значение»

Для удобства свернём табличную часть в области реквизитов (рис. 1.6.8).

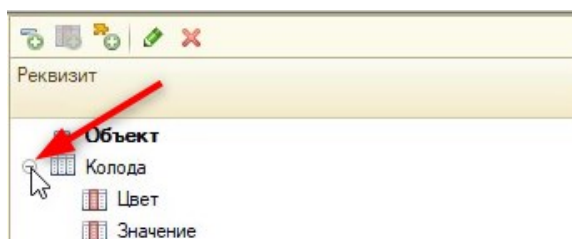


Рис. 1.6.8. Сворачивание табличной части

После этого создадим реквизит «ТекущаяКарта» для хранения текущей карты на поле игрока (рис. 1.6.9).

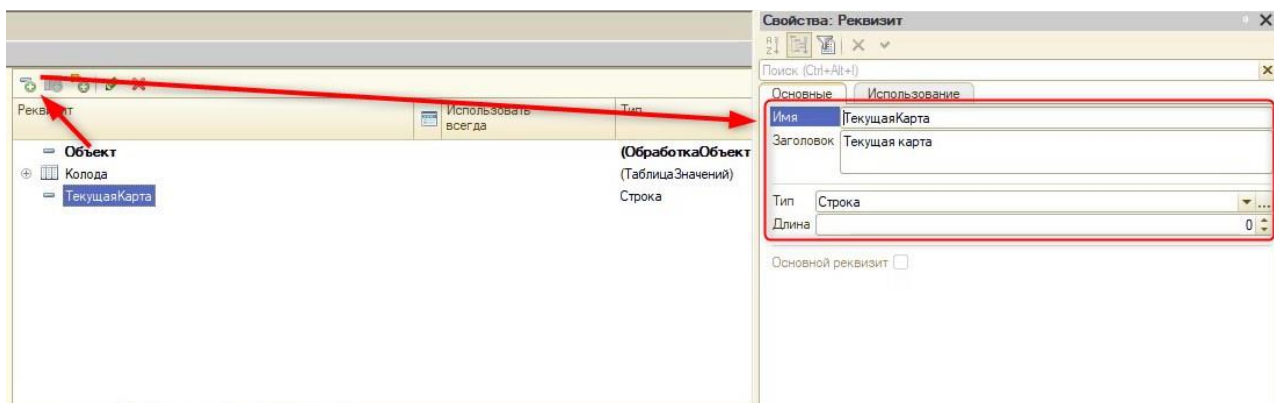


Рис. 1.6.9. Создание нового реквизита

На форме должно быть три раздела:

- 1) Рука (карты) игрока.
- 2) Рука (карты) бота.
- 3) Игровое поле с текущей картой и предыдущей сыгранной.

Для того чтобы отделить всё по разделам, в форме через кнопку «Добавить» создадим несколько групп без отображения (рис. 1.6.10).

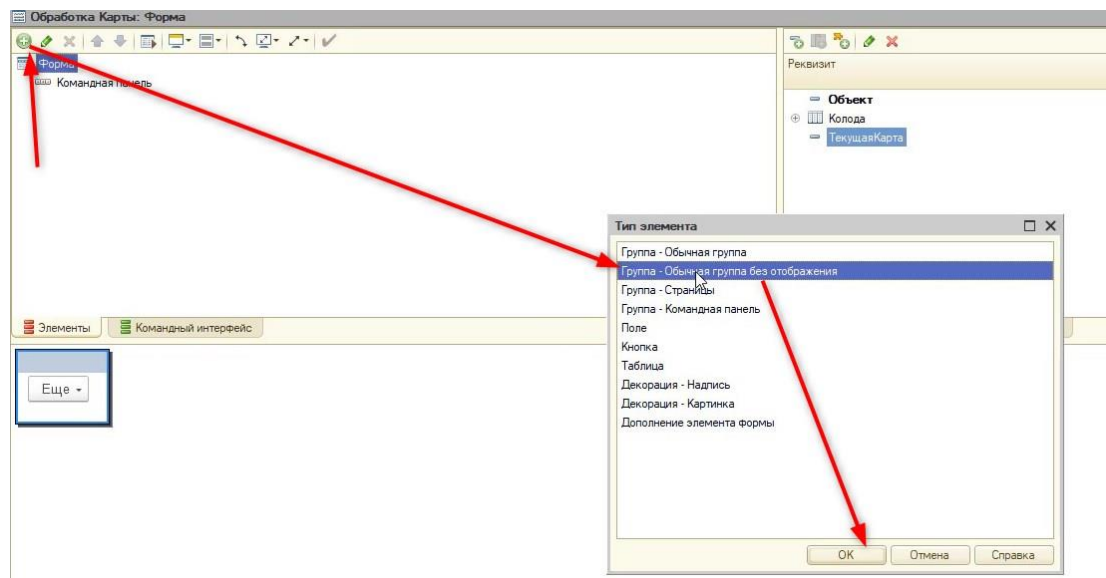


Рис. 1.6.10. Создание групп без отображения

Первая группа будет называться «ГруппаРукаБота» (рис. 1.6.11).

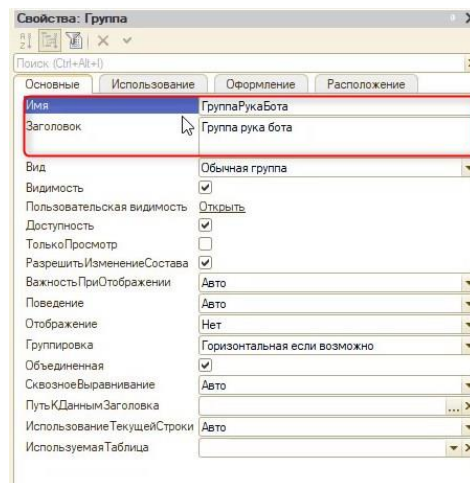


Рис. 1.6.11. Создание группы руки бота

По аналогии создадим группы «ГруппаИгровоеПоле», «ГруппаРукаИгрока» и получим следующую картину (см. рис. 1.6.12). Приставка «Группа» в названии нужна для того, чтобы при написании кода различать группы и элементы.

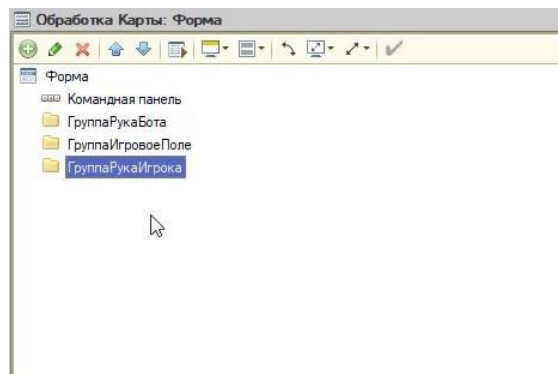


Рис. 1.6.12. Область элементов

Если при создании новой группы в области элементов будет выделена (синим цветом) другая группа, то новая поместится внутрь неё. Для исправления можно изначально перед созданием нажать на саму форму или перетащить, удерживая левую кнопку мыши, уже существующую группу на форму (рис. 1.6.13).

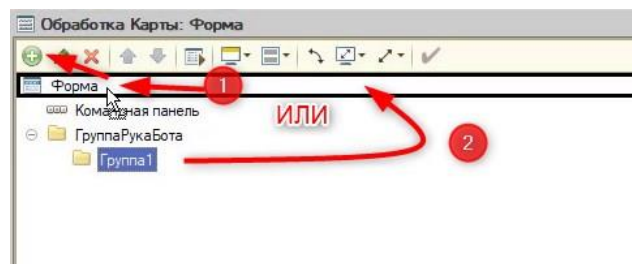


Рис. 1.6.13. Создание группы

Группы с рукой бота и игрока будут формироваться программно, и изначально в них ничего не будет располагаться. А в группе «Игровое поле» будет сразу находиться текущая карта.

Перенесём реквизит «ТекущаяКарта» на форму в группу с игровым полем, удерживая левую кнопку мыши (рис. 1.6.14).

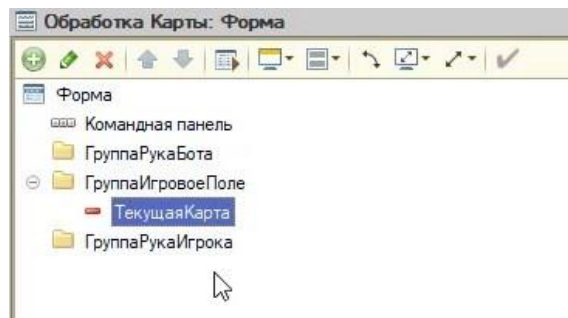


Рис. 1.6.14. Область с группами и элементом

На данный момент текущая карта на форме выглядит как поле ввода. Такой формат нам не подходит, так как необходимо отображать изображения карт. Поменяем его вид на «Поле картинки», как делали ранее (рис. 1.6.15).

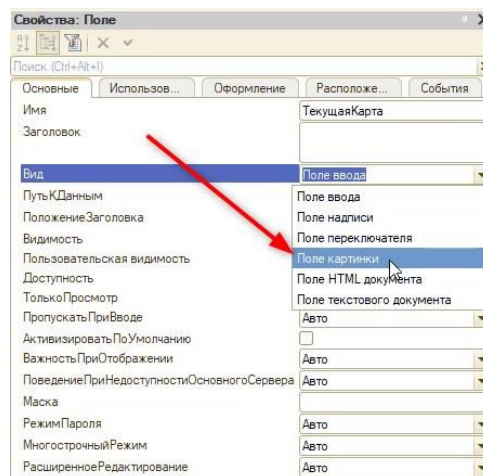


Рис. 1.6.15. Изменение вида поля элемента «Текущая карта»

Помимо этого, изменим на вкладке «Оформление» свойство «РазмерКартинки» на «Автоматический размер» (рис. 1.6.16).

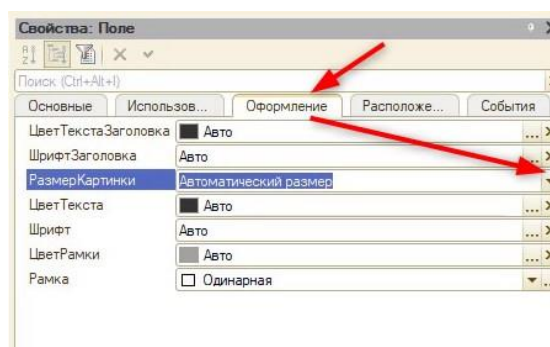


Рис. 1.6.16. Изменение свойства «Размер картинки»

Теперь можно приступить к написанию программного кода для формирования руки игрока и бота. Помимо этого, реализуем первый ход бота — выкладывание первой карты из перемешанной колоды.

Добавим команду «НачатьИгру» и перенесём её на командную панель формы (рис. 1.6.17).

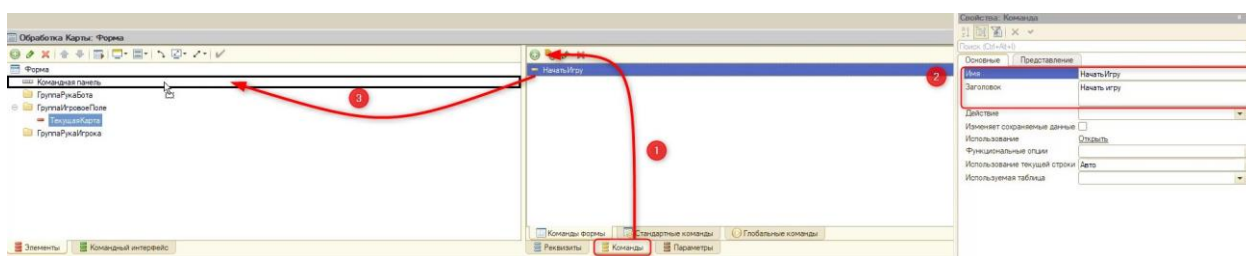


Рис. 1.6.17. Добавление новой кнопки

Определим действие для новой кнопки (рис. 1.6.18).

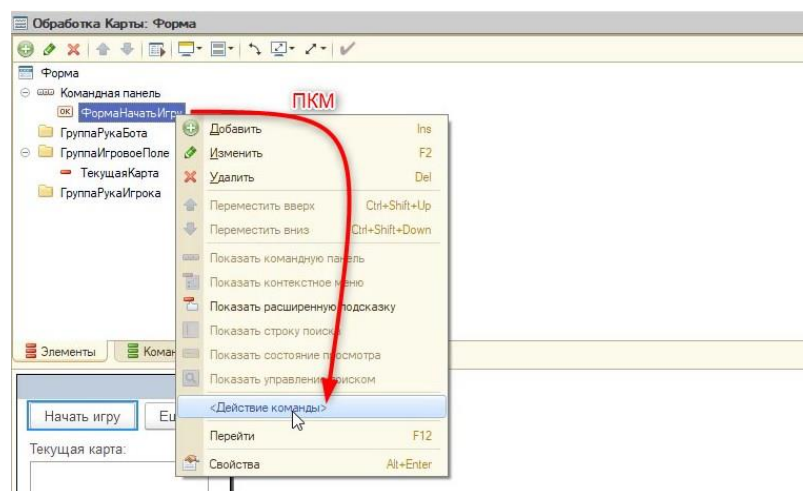


Рис. 1.6.18. Добавление действия команды

Обработчик команды создадим на клиенте.

В результате откроется модуль формы обработки, и можно приступить к созданию программной логики:

&НаКлиенте

Процедура НачатьИгру(Команда)

//Для начала нужно получить все карты из справочника

//Сделать это можно только на сервере, так как для этого нужно обратиться к базе данных

//База данных - это всё что хранится в нашей программе

//Поэтому создадим новую процедуру и вызовем её в обработчике команды:

НачатьИгруНаСервере();

КонецПроцедуры

//Серверная процедура, будет:

//1) Получать все карты из справочника;

//2) Перемешивать все карты;

//3) Заполнять руку бота и игрока.

&НаСервере

Процедура НачатьИгруНаСервере()

КонецПроцедуры

Для того чтобы получить все данные из справочника, понадобится язык запросов. **Язык запросов** — это инструмент, который позволяет получить информацию из базы данных (из справочников, из документов и т. д.) с определёнными условиями.

В данной ситуации нам нужно просто получить всё из справочника с картами. Для этого воспользуемся встроенным конструктором. Открыть его можно, нажав по пустой строке правой кнопкой мыши и выбрав «Конструктор запроса с обработкой результата» (рис. 1.6.19).

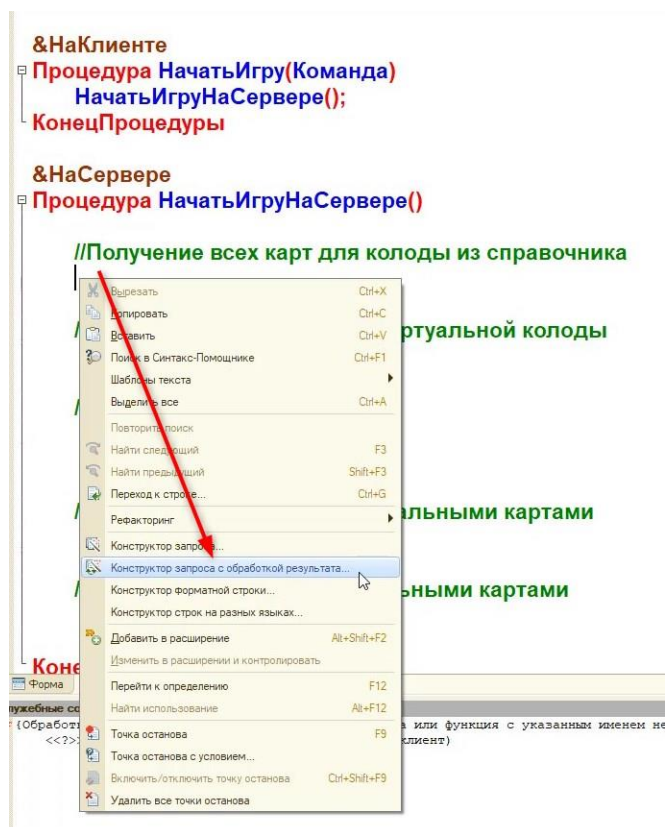


Рис. 1.6.19. Вызов конструктора

В открывшемся окне нужно выбрать «Да» (рис.1.6.20).

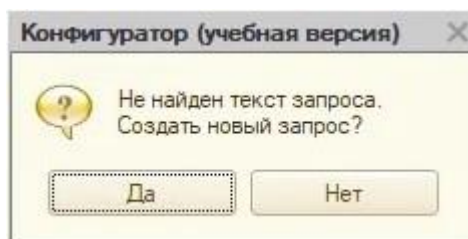


Рис. 1.6.20. Создание нового запроса

Откроется конструктор запроса с обработкой результата. В первой вкладке «Обработка результата» оставим «Обход результата» (рис. 1.6.21).

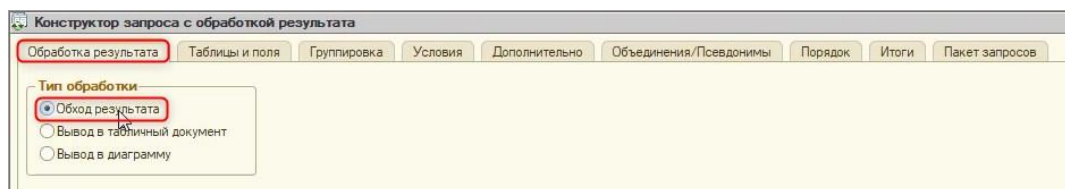


Рис. 1.6.21. Вкладка «Обработка результата»

Далее обратимся на вкладку «Таблицы и поля». На этой вкладке определяется, какие данные и из каких таблиц необходимо получить. В данный момент в базе присутствует один справочник — «Список карт», чтобы его выбрать, нужно нажать по нему двойным щелчком мыши. После этого справочник переместится в раздел таблиц (центральное окно). Далее выберем из справочника реквизиты «Цвет» и «Значение» и нажмём «Ок» (рис. 1.6.22).

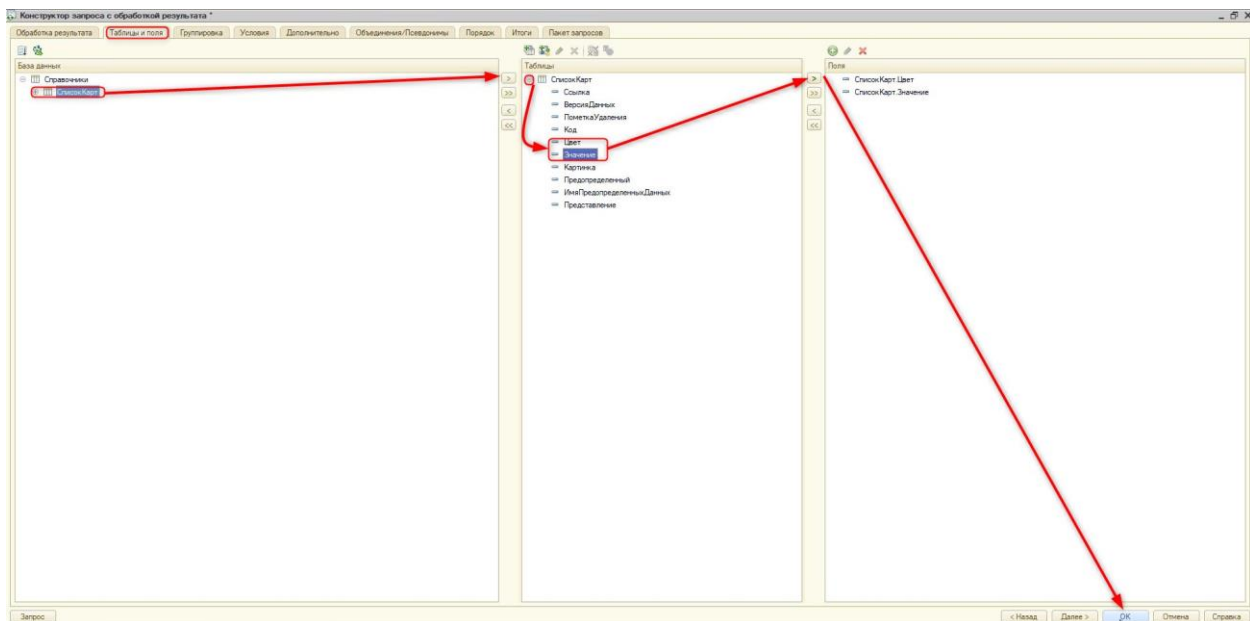


Рис. 1.6.22. Работа с конструктором

После этого конструктор сформирует код запроса. Уберём лишние комментарии конструктора и продолжим писать программную логику:

&НаСервере

Процедура НачатьИгруНаСервере()

//Получение всех карт для колоды из справочника

Запрос = Новый Запрос;

Запрос.Текст =

"ВЫБРАТЬ

| СписокКарт.Цвет КАК Цвет,

| СписокКарт.Значение КАК Значение

| ИЗ

| Справочник.СписокКарт КАК СписокКарт";

РезультатЗапроса = Запрос.Выполнить();

//Заменим РезультатЗапроса.Выбрать() на РезультатЗапроса. Выгрузить()

//И выгрузим сразу в новую переменную КолодаБезПеремешивания

КолодаБезПеремешивания = РезультатЗапроса.Выгрузить();

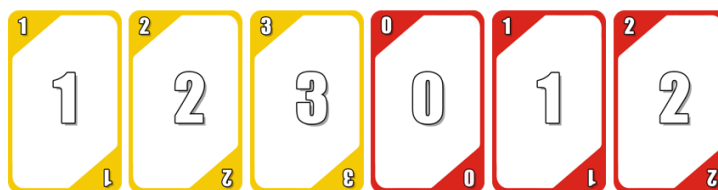
//Поскольку данные выгружены, цикл "Пока", оставленный конструктором следует стереть.

КонецПроцедуры

Далее необходимо в этой же процедуре перемешать карты. Перемешивание будет происходить с помощью генератора случайных чисел. Генератор будет выбирать случайный индекс карты в массиве и затем перемещать её в другую колоду. Индекс — это порядковый номер элемента в массиве, его нумерация начинается с 0 (рис. 1.6.23).

Индекс – это порядковый номер элемента в массиве. Его нумерация начинается с 0.

Таблица значений КолодаБезПеремешивания



Индекс карты

[0] [1] [2] [3] [4] [5]

Рис. 1.6.23. Индексы колоды

Это означает, что если в колоде 20 карт, то их индекс будет в диапазоне от 0 до 19.

Продолжим написание алгоритма:

&НаСервере

Процедура НачатьИгруНаСервере()

Запрос = Новый Запрос;

Запрос.Текст =

"ВЫБРАТЬ

| СписокКарт.Цвет КАК Цвет,

| СписокКарт.Значение КАК Значение

|ИЗ

| Справочник.СписокКарт КАК СписокКарт";

РезультатЗапроса = Запрос.Выполнить();

КолодаБезПеремешивания = РезультатЗапроса.Выгрузить();

//Перемешивание карт для виртуальной колоды

//Для перемешивания воспользуемся генератором случайных чисел

//Он позволит нам выбирать случайное число в диапазоне от первой до последней карты

ГСЧ = Новый ГенераторСлучайныхЧисел;

//Обойдём всю колоду, пока она не закончится:

Пока КолодаБезПеремешивания.Количество() > 0 Цикл

//Так как порядковый номер в колоде начинается с 0

//В рамках цикла создадим переменную ПоследнийНомер

//И сразу же в начале цикла вычтем из нее единицу

//Так как в массиве начинается нумерация с 0

ПоследнийНомер = КолодаБезПеремешивания.Количество() - 1;

//Обрабатываем ещё одну ситуацию - когда будет обрабатываться последняя карта

//Тогда после вычислений выше ПоследнийНомер станет отрицательным числом

```
//И программа выдаст ошибку
```

```
//Поэтому введём условие для такой ситуации и в случае получения последней карты, приравняем последний номер к 0:
```

```
Если ПоследнийНомер < 0 Тогда
```

```
    ПоследнийНомер = 0;
```

```
КонецЕсли;
```

```
//Выберем случайную карту в не перемешанной колоде от 0 до последнего номера:
```

```
НомерКарты =ГСЧ.СлучайноеЧисло(0, ПоследнийНомер);
```

```
//Запишем выбранную карту в новую переменную
```

```
ДанныеОКарте = КолодаБезПеремешивания[НомерКарты];
```

```
//Выделим место в перемешанной колоде для новой карты
```

```
СлучайнаяКарта = ЭтотОбъект.Колода.Добавить();
```

```
//Поскольку в колодах есть, и цвет и значение - скопируем их с помощью метода
```

```
//ЗаполнитьЗначенияСвойств(<Приемник>, <Источник>)
```

```
ЗаполнитьЗначенияСвойств(СлучайнаяКарта, ДанныеОКарте);
```

```
//Удалим карту из колоды без перемешивания:
```

```
КолодаБезПеремешивания.Удалить(ДанныеОКарте);
```

```
КонецЦикла;
```

```
КонецПроцедуры
```

Далее нужно реализовать первый ход бота. Бот будет выкладывать первую карту из колоды на стол (в нашем случае — в группу «ИгровоеПоле», в реквизит «ТекущаяКарта»). При этом нужно будет получить картинку выложенной карты из справочника отдельной функцией на сервере.

Для начала создадим новую функцию «ПолучитьКартинкуКарты()» на сервере (так как мы обращаемся к базе данных), которая будет получать изображение карты из справочника:

```
&НаСервере
```

```
Функция ПолучитьКартинкуКарты()
```

```
//Возьмём первую карту из колоды (индекс такой карты будет равен 0)
```

```
//Соединим Цвет и значение карты в одну переменную ИмяИскомойКарты
```

```
ИмяИскомойКарты = ЭтотОбъект.Колода[0].Цвет + " " +  
ЭтотОбъект.Колода[0].Значение;
```

```
//С помощью метода НайтиПоКоду(<Код>) получим все данные из справочника по карте
```

```
СсылкаНаКартинкуКарты =  
Справочники.СписокКарт.НайтиПоКоду(ИмяИскомойКарты);
```

```
//Вернём из полученных данных ссылку на картинку
```

```
Возврат ПолучитьНавигационнуюСсылку(СсылкаНаКартинкуКарты, "Картинка");
```

```
КонецФункции // ПолучитьКартинкуКарты()
```

После этого допишем код в процедуре «НачатьИгруНаСервере()»:

```
&НаСервере
```

```
Процедура НачатьИгруНаСервере()
```

```
Запрос = Новый Запрос;
```

```
Запрос.Текст =
```

```
"ВЫБРАТЬ
```

```
|      СписокКарт.Цвет КАК Цвет,
|      СписокКарт.Значение КАК Значение
|ИЗ
|      Справочник.СписокКарт КАК СписокКарт";
```

```
РезультатЗапроса = Запрос.Выполнить();
КолодаБезПеремешивания = РезультатЗапроса.Выгрузить();
ГСЧ = Новый ГенераторСлучайныхЧисел;
Пока КолодаБезПеремешивания.Количество() > 0 Цикл
    ПоследнийНомер = КолодаБезПеремешивания.Количество() - 1;
    Если ПоследнийНомер < 0 Тогда
        ПоследнийНомер = 0;
    КонецЕсли;
    НомерКарты = ГСЧ.СлучайноеЧисло(0, ПоследнийНомер);
    ДанныеОКарте = КолодаБезПеремешивания[НомерКарты];
    СлучайнаяКарта = ЭтотОбъект.Колода.Добавить();
    ЗаполнитьЗначенияСвойств(СлучайнаяКарта, ДанныеОКарте);
    КолодаБезПеремешивания.Удалить(ДанныеОКарте);
КонецЦикла;
```

```
//Первый ход бота
```

```
//Вызовем созданную серверную функцию и запишем её результат в реквизит
"ТекущаяКарта"
```

```
ЭтотОбъект.ТекущаяКарта = ПолучитьКартинкуКарты();
```

```
//Запишем в заголовок элемента "ТекущаяКарта" цвет и номинал карты
```

```
//В дальнейшем заголовок понадобится для анализа
```

```
Элементы.ТекущаяКарта.Заголовок = ЭтотОбъект.Колода[0].Цвет + " " +
ЭтотОбъект.Колода[0].Значение;
```

```
//Удалим из виртуальной колоды карту
```

```
ЭтотОбъект.Колода.Удалить(0);
```

```
КонецПроцедуры
```

В результате можно проверить работу обработки. Запустим пользовательский режим.

Если нажать кнопку «Начать игру», то появится текущая карта, которую выложил бот на стол из виртуальной колоды (рис. 1.6.25).

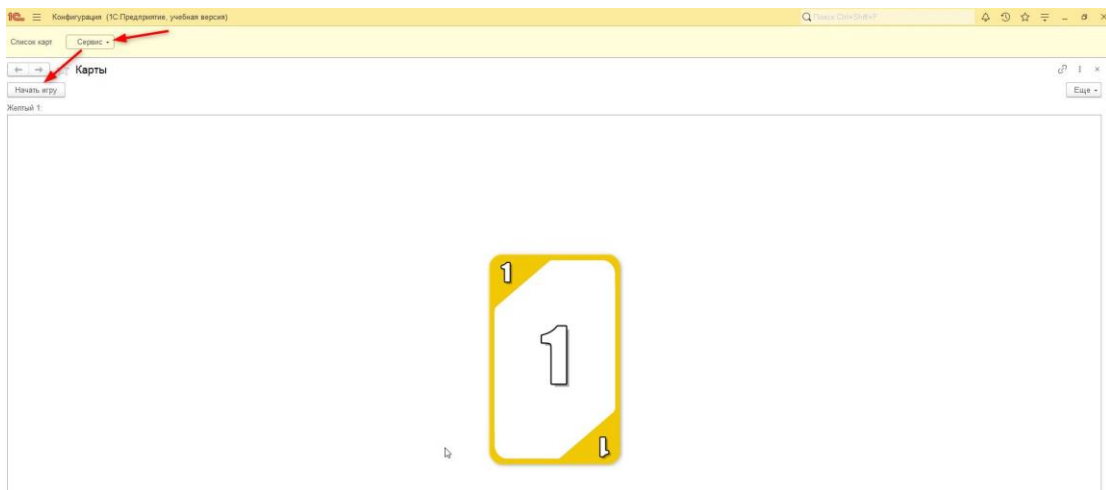


Рис. 1.6.24. Первая карта игры

Если на предыдущем уроке не был заполнен справочник с картами, его следует дополнить, так как в дальнейшем будет сложно тестировать игру с колодой в 2 карты. Для начала можно добавить хотя бы два цвета карт (рис. 1.6.25).

Название	Цвет	Значение
Желтый 0	Желтый	0
Желтый 1	Желтый	1
Желтый 2	Желтый	2
Желтый 3	Желтый	3
Желтый 4	Желтый	4
Желтый 5	Желтый	5
Желтый 6	Желтый	6
Желтый 7	Желтый	7
Желтый 8	Желтый	8
Желтый 9	Желтый	9
Зеленый 0	Зеленый	0
Зеленый 1	Зеленый	1
Зеленый 2	Зеленый	2
Зеленый 3	Зеленый	3
Зеленый 4	Зеленый	4
Зеленый 5	Зеленый	5
Зеленый 6	Зеленый	6
Зеленый 7	Зеленый	7
Зеленый 8	Зеленый	8
Зеленый 9	Зеленый	9

Рис. 1.6.25. Заполненный справочник «Список карт»

Теперь нужно программно раздать 7 карт игроку и боту. Напишем отдельную серверную функцию «ДобавитьКартуНаПоле()». Эта функция будет создавать новый реквизит и элемент программно. В реквизит будет происходить запись навигационной ссылки на картинку карты, а элемент будет ссылаться на этот реквизит. Предварительно нужно предусмотреть передачу номера карты (для названия реквизита и элемента) и название группы (чтобы алгоритм мог заполнять руку бота и игрока):

&НаСервере

//Добавим параметры:

//НомерКарты - нужен для заполнения названия реквизита и элемента

//Группа - название группы, куда нужно добавить карту

Функция ДобавитьКартуНаПоле(НомерКарты, Группа)

//Проверим количество карт в колоде:

Если ЭтотОбъект.Колода.Количество() > 0 Тогда

//Для начала определим имя для нового реквизита

ИмяРеквизитаКарты = "КартаНаРуках" + НомерКарты;

//Зададим тип нового реквизита - "Строка"

```

        РеквизитКарты = Новый РеквизитФормы(ИмяРеквизитаКарты, Новый
ОписаниеТипов("Строка"));

        //Создадим новый массив
        МассивРеквизитов = Новый Массив;

        //Добавим в массив реквизит
        МассивРеквизитов.Добавить(РеквизитКарты);

        //Добавим новый реквизит:
        ИзменитьРеквизиты(МассивРеквизитов);

        //Получим ссылку на изображения карты:
        ЭтаФорма[ИмяРеквизитаКарты] = ПолучитьКартинкуКарты();

        //Создадим новый элемент:
        //Элементы.Добавить(Имя, Тип, Группа)
        ЭлементКарты = Элементы.Добавить(ИмяРеквизитаКарты, Тип("ПолеФормы"),
Группа);

        //Свяжем элемент с реквизитом
        ЭлементКарты.ПутьКДанным = ИмяРеквизитаКарты;

        //Поменяем тип поля на картинку
        ЭлементКарты.Вид = ВидПоляФормы.ПолеКартинки;

        //Для того, чтобы картинка отображалась правильно, включим авторазмер
        ЭлементКарты.РазмерКартинки = РазмерКартинки.АвтоРазмер;

        //Добавим заголовок новому элементу:
        ЭлементКарты.Заголовок = ЭтотОбъект.Колода[0].Цвет + " " +
ЭтотОбъект.Колода[0].Значение;

        //Уберем карту из колоды
        ЭтотОбъект.Колода.Удалить(0);

        //Очистим массив
        МассивРеквизитов.Очистить();

        //Карта успешно добавлена!
        Возврат Истина;

    Иначе
        //Если карты закончились, то устанавливается флаг Ложь, который пригодится позже
        Возврат Ложь;

    КонецЕсли;

КонецФункции // ДобавитьКартуНаПоле()

```

Дополним процедуру «НачатьИгруНаСервере()»:

```

&НаСервере
Процедура НачатьИгруНаСервере()
    Запрос = Новый Запрос;
    Запрос.Текст =
        "ВЫБРАТЬ
        |      СписокКарт.Цвет КАК Цвет,

```



```

|      СписокКарт.Значение КАК Значение
|ИЗ
|      Справочник.СписокКарт КАК СписокКарт";

```

```

РезультатЗапроса = Запрос.Выполнить();
КолодаБезПеремешивания = РезультатЗапроса.Выгрузить();
ГСЧ = Новый ГенераторСлучайныхЧисел;
Пока КолодаБезПеремешивания.Количество() > 0 Цикл
    ПоследнийНомер = КолодаБезПеремешивания.Количество() - 1;
    Если ПоследнийНомер < 0 Тогда
        ПоследнийНомер = 0;
    КонецЕсли;
    НомерКарты = ГСЧ.СлучайноеЧисло(0, ПоследнийНомер);
    ДанныеОКарте = КолодаБезПеремешивания[НомерКарты];
    СлучайнаяКарта = ЭтотОбъект.Колода.Добавить();
    ЗаполнитьЗначенияСвойств(СлучайнаяКарта, ДанныеОКарте);
    КолодаБезПеремешивания.Удалить(ДанныеОКарте);
КонецЦикла;

```

//Первый ход бота

```

ЭтотОбъект.ТекущаяКарта = ПолучитьКартинкуКарты();
Элементы.ТекущаяКарта.Заголовок = ЭтотОбъект.Колода[0].Цвет + " " +
ЭтотОбъект.Колода[0].Значение;
ЭтотОбъект.Колода.Удалить(0);

```

//Заполнение руки игрока начальными картами

//Воспользуемся счетчиком, который 7 раз вызовет новую функцию

```

Для Счетчик = 1 По 7 Цикл
    ДобавитьКартуНаПоле(Счетчик, Элементы.ГруппаРукаИгрока);
КонецЦикла;

```

//Заполнение руки бота начальными картами

//Воспользуемся счетчиком, который 7 раз вызовет новую функцию

//Счетчик начинается с 8, так как название элементов должно быть уникальным

```

Для Счетчик = 8 По 14 Цикл
    ДобавитьКартуНаПоле(Счетчик, Элементы.ГруппаРукаБота);
КонецЦикла;

```

КонецПроцедуры

Готово! По итогу в пользовательском режиме должно получиться следующее (см. рис. 1.6.26).

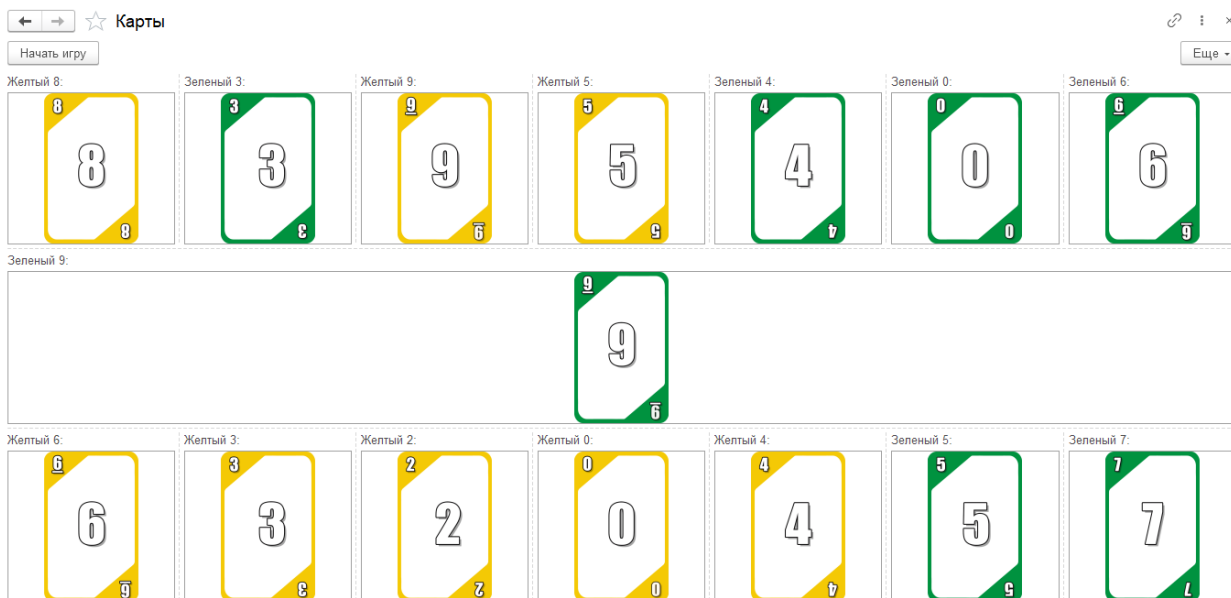


Рис. 1.6.26. Итог занятия

Занятие 7 – Карточная игра. Ход игры

Реализуем непосредственно сам ход игры. Добавим колоду сброса, возможность разыграть карту и контроль выбора игрока.

Для того чтобы добавить возможность нажимать на карту, необходимо вернуться в модуль формы к функции «ДобавитьКартуНаПоле»:

```
&НаСервере
Функция ДобавитьКартуНаПоле(НомерКарты, Группа)

    //Проверка карт в колоде
    Если ЭтотОбъект.Колода.Количество() > 0 Тогда
        //Создание реквизита
        ИмяРеквизитаКарты = "КартаНаРуках" + НомерКарты;
        РеквизитКарты = Новый РеквизитФормы(ИмяРеквизитаКарты, Новый
ОписаниеТипов("Строка"));
        МассивРеквизитов = Новый Массив;
        МассивРеквизитов.Добавить(РеквизитКарты);
        ИзменитьРеквизиты(МассивРеквизитов);
        ЭтаФорма[ИмяРеквизитаКарты] = ПолучитьКартинкуКарты();

        //Создание элемента
        ЭлементКарты = Элементы.Добавить(ИмяРеквизитаКарты, Тип("ПолеФормы"),
Группа);

        ЭлементКарты.ПутьКДанным = ИмяРеквизитаКарты;
        ЭлементКарты.Вид = ВидПоляФормы.ПолеКартинки;
        ЭлементКарты.РазмерКартинки = РазмерКартинки.АвтоРазмер;
        ЭлементКарты.Заголовок = ЭтотОбъект.Колода[0].Цвет + " " +
ЭтотОбъект.Колода[0].Значение;

        //Включим возможность нажатия на карту:
        ЭлементКарты.Гиперссылка = Истина;
        //Добавим обработчик события карте
        //Назовём его "ВыборКарты"
        //В дальнейшем эту процедуру нужно будет создать
        ЭлементКарты.УстановитьДействие("Нажатие", "ВыборКарты");
        //Выключим заголовок у картинок для пользователя:
        ЭлементКарты.ПоложениеЗаголовка =
ПоложениеЗаголовкаЭлементаФормы.Нет;

        ЭтотОбъект.Колода.Удалить(0);
        МассивРеквизитов.Очистить();
        Возврат Истина;
    Иначе
        //Добавим сообщение пользователю, что карты закончились:
```

```

Сообщить("Карты закончились!");
Возврат Ложь;
КонецЕсли;
КонецФункции // ДобавитьКартуНаПоле()

```

Далее нужно создать действие для обработчика события «Нажатие»:

```

&НаКлиенте
//Элемент, СтандартнаяОбработка - это стандартные параметры обработчика события
"Нажатие"
Процедура ВыборКарты(Элемент, СтандартнаяОбработка)
//Выключим сообщение с навигационной ссылкой при нажатии на карточку:
СтандартнаяОбработка = Ложь;
//Получаем по отдельности цвет и значение разыгрываемой карты и текущей карты на
столе
МассивДанныхОКартеВРуке = СтрРазделить(Элемент.Заголовок, " ", Ложь);
МассивДанныхОКартеНаСтоле = СтрРазделить(Элементы.ТекущаяКарта.Заголовок, "
", Ложь);

//Проверка возможности сыграть карту
//Если совпадает цвет или номинал, тогда
Если МассивДанныхОКартеВРуке[0] = МассивДанныхОКартеНаСтоле[0] ИЛИ
МассивДанныхОКартеВРуке[1] = МассивДанныхОКартеНаСтоле[1] Тогда
//Разыгрываем карту
//Для этого создадим новую серверную процедуру "СыгратьКарту" и передадим
название элемента, на который нажал игрок
СыгратьКарту(Элемент.Имя);
//Скроем видимость карты
Элемент.Видимость = Ложь;
Иначе
//Вывод сообщения о том, что карту сыграть нельзя
Сообщить("Нельзя сыграть карту!");
КонецЕсли;
КонецПроцедуры

```

Перед тем как обработать ситуацию с розыгрышем карты, добавим новый реквизит «Сброс» и перенесём его в группу с игровым полем (рис. 1.7.1).

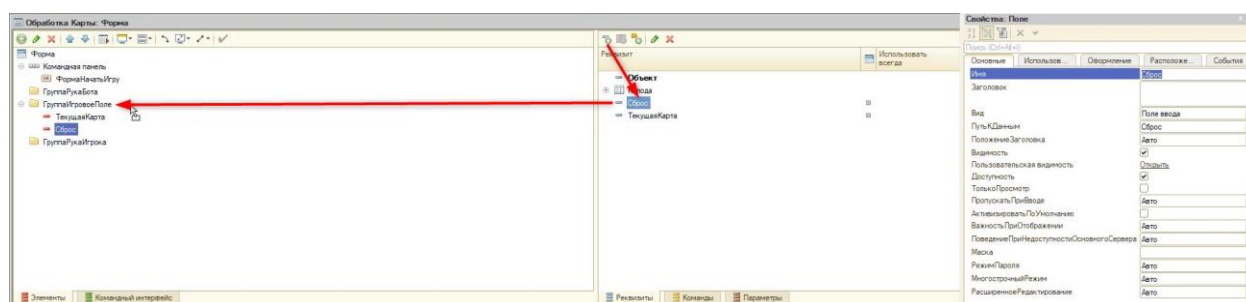


Рис. 1.7.1. Создание разыгранной колоды

Вид у элемента укажем «ПолеКартинки» (рис. 1.7.2).

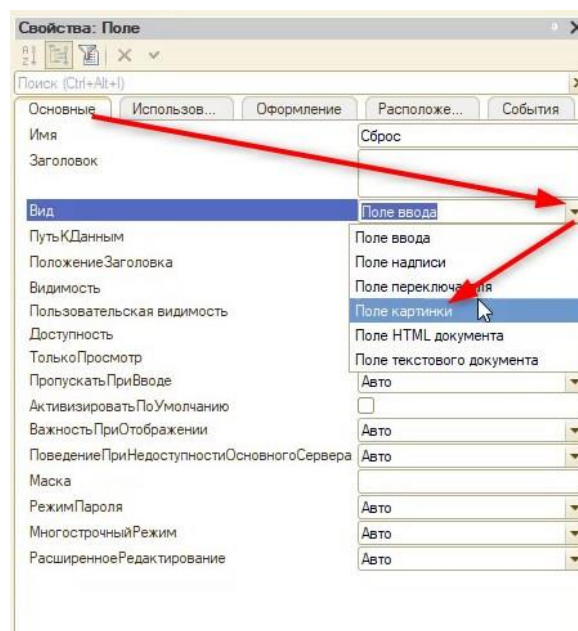


Рис. 1.7.2. Смена вида поля

На вкладке «Оформление» укажем свойство «РазмерКартинки» — «Автоматический размер» (рис. 1.7.3).

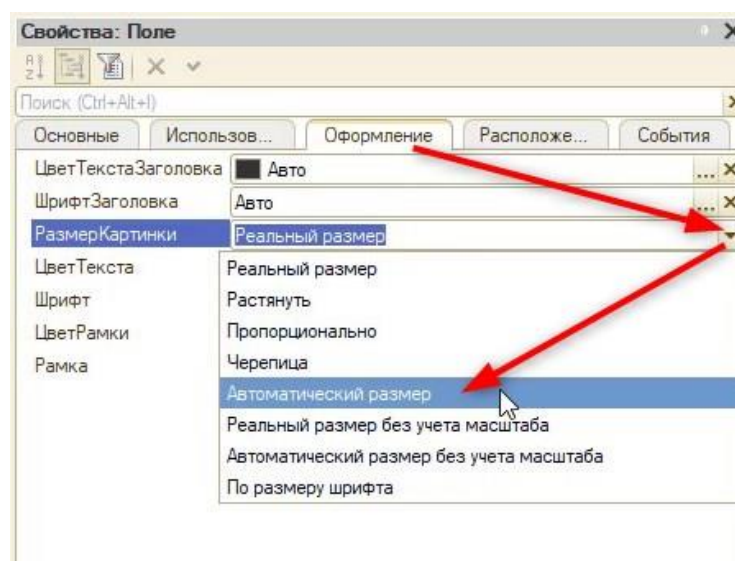


Рис. 1.7.3. Размер картинки

Теперь обработаем ситуацию с розыгрышем карты. В процедуре будет производиться подмена картинки, а её данные будут содержаться в реквизитах. Получить их можно будет только через сервер и «ПутьКДанным». Исходя из этого, новая процедура будет серверной:

&НаСервере

Процедура СыгратьКарту(ИмяЭлемента)

//Найти элемент среди всех элементов формы

ЭлементПоиска = Элементы.Найти(ИмяЭлемента);

//Получить путь к данным, содержащий информацию о картинке

АдресКарты = ЭлементПоиска.ПутьКДанным;

//Предыдущая карта со стола отправляется в сброс

ЭтотОбъект.Сброс = ЭтотОбъект.ТекущаяКарта;

```

//Что было сыграно - текущая карта на столе
ЭтотОбъект.ТекущаяКарта = ЭтотОбъект[АдресКарты];

//Установка нового заголовка для определения цвета и значения
Элементы.ТекущаяКарта.Заголовок = ЭлементПоиска.Заголовок;

КонецПроцедуры

```

Теперь можно протестировать механизм, который получился. На данный момент ход за бота может сделать игрок, но в дальнейшем компьютерный соперник будет производить свои ходы автоматически, а его карты будут скрыты от игрока.

Во время тестирования можно заметить, что в конце игры не всегда есть карты, которые можно разыграть, поэтому необходимо создать специальный механизм добора карт. У игрока он будет осуществляться по кнопке, а у бота — автоматически.

Для начала создадим команду «ВытащитьКарту» и перенесём её на форму, в самый низ (рис. 1.7.4).

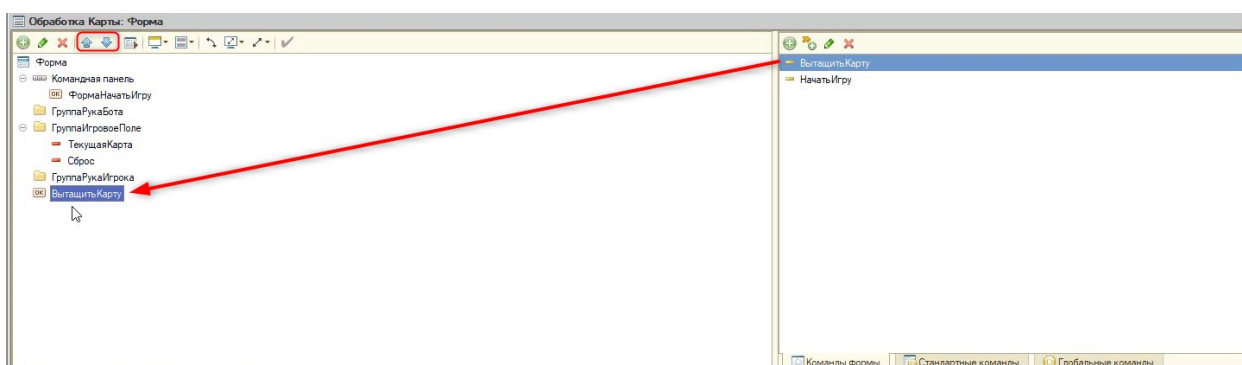


Рис. 1.7.4. Добавление новой кнопки

Далее определим действие для данной команды:

&НаКлиенте

Процедура ВытащитьКарту(Команда)

//Создадим серверную процедуру и передадим в нее название группы

ВытащитьКартуНаСервере(Элементы.ГруппаРукаИгрока.Имя);

КонецПроцедуры

//Обязательно укажем параметр ИмяГруппы

//Чтобы в дальнейшем использовать эту процедуру для руки бота

&НаСервере

Процедура ВытащитьКартуНаСервере(ИмяГруппы)

//Получим группу для возможности редактирования:

Группа = Элементы.Найти(ИмяГруппы);

//Далее нужно проанализировать и найти свободный номер карты для элемента

//(так как названия элементов должны быть уникальными)

//Поставим цикл от 0 до 100

Для Номер = 1 По 100 Цикл

//Если элемент "КартаНаРуках №" отсутствует, значит

Если Элементы.Найти("КартаНаРуках" + Номер) = Неопределено Тогда

//Добавим новую карту с номером, которого нет в системе

ДобавитьКартуНаПоле(Номер, Группа);
 //Прервём цикл:
 Прервать;
 КонецЕсли;
 КонецЦикла;
 КонецПроцедуры

При большом доборе карт в пользовательском режиме может случиться такая ситуация (см. рис. 1.7.5).

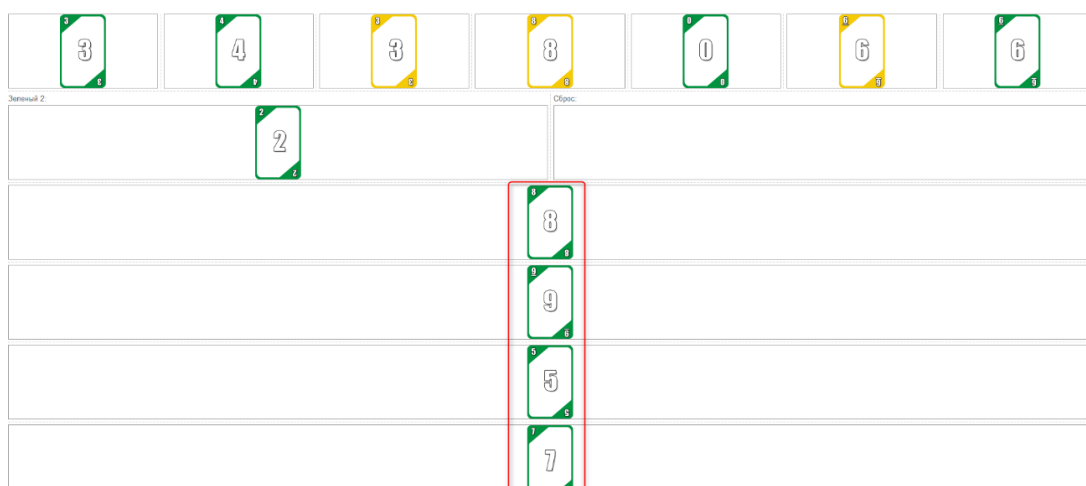


Рис. 1.7.5. Карты в вертикальном положении

Для того чтобы такого не происходило, у группы игрока и бота в редакторе формы нужно поставить свойство «Группировка» — «Горизонтальная всегда» (рис. 1.7.6).

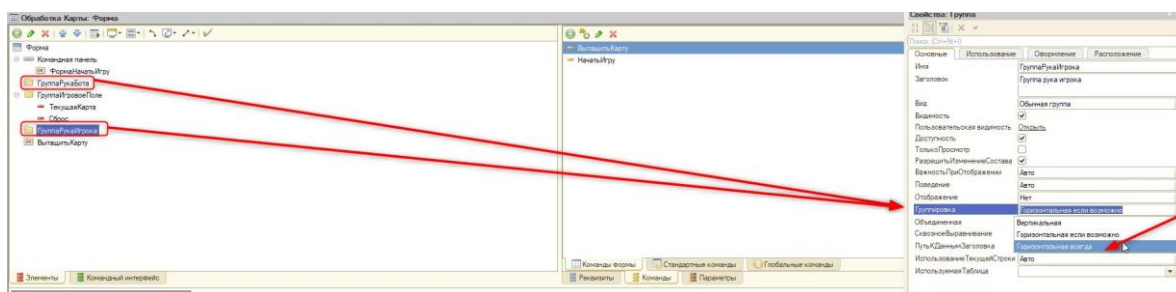


Рис. 1.7.6. Смена группировки

Отлично! Осталось совсем немного до выпуска ещё одной игры.

Занятие 8 – Карточная игра

На данный момент карты, которые были разыграны, никак не удаляются. Реализовать это «удаление» можно с помощью двух процедур, которые были написаны ранее — «ВытащитьКарту» и «ВытащитьКартуНаСервере».

Для начала поменяем логику функции «ДобавитьКартуНаПоле» в модуле формы обработки. Функция могла возвращать два значения — истина, если колода не пустая, и ложь, если карты закончились. Уберём вывод сообщения «Карты закончились», вернём значения функции «ДобавитьКартуНаПоле» в «ВытащитьКарту» и обработаем пустую колоду. Тогда код будет выглядеть следующим образом:

```
&НаСервере
Функция ДобавитьКартуНаПоле(НомерКарты, Группа)
    Если ЭтотОбъект.Колода.Количество() > 0 Тогда
        ИмяРеквизитаКарты = "КартаНаРуках" + НомерКарты;
        РеквизитКарты = Новый РеквизитФормы(ИмяРеквизитаКарты, Новый
ОписаниеТипов("Строка"));
        МассивРеквизитов = Новый Массив;
        МассивРеквизитов.Добавить(РеквизитКарты);
        ИзменитьРеквизиты(МассивРеквизитов);
        ЭтаФорма[ИмяРеквизитаКарты] = ПолучитьКартинкуКарты();
        ЭлементКарты = Элементы.Добавить(ИмяРеквизитаКарты, Тип("ПолеФормы"),
Группа);
        ЭлементКарты.ПутьКДанным = ИмяРеквизитаКарты;
        ЭлементКарты.Вид = ВидПоляФормы.ПолеКартинки;
        ЭлементКарты.РазмерКартинки = РазмерКартинки.АвтоРазмер;
        ЭлементКарты.Заголовок = ЭтотОбъект.Колода[0].Цвет + " " +
ЭтотОбъект.Колода[0].Значение;
        ЭлементКарты.Гиперссылка = Истина;
        ЭлементКарты.УстановитьДействие("Нажатие", "ВыборКарты");
        ЭлементКарты.ПоложениеЗаголовка =
ПоложениеЗаголовкаЭлементаФормы.Нет;
        ЭтотОбъект.Колода.Удалить(0);
        МассивРеквизитов.Очистить();
        Возврат Истина;
    Иначе
        //Уберём вывод сообщения "Карты закончились" и возложим на другую процедуру
        Возврат Ложь;
    КонецЕсли;
КонецФункции

&НаСервере
//Так как в дальнейшем нужно передать значение на клиент, превратим процедуру в функцию:
Функция ВытащитьКартуНаСервере(ИмяГруппы)
    Группа = Элементы.Найти(ИмяГруппы);
    Для Номер = 1 По 100 Цикл
```

```

        Если Элементы.Найти("КартаНаРуках" + Номер) = Неопределено Тогда
            //Вернём значение функции "ДобавитьКартуНаПоле" на клиент:
            Возврат ДобавитьКартуНаПоле(Номер, Группа);
            Прервать;
        КонецЕсли;
    КонецЦикла;
    //Не забудем поменять в конце "КонецПроцедуры" на:
КонецФункции

&НаКлиенте
Процедура ВытащитьКарту(Команда)
    //Создадим переменную для возвращаемого значения, к примеру - "Статус"
    Статус = ВытащитьКартуНаСервере(Элементы.ГруппаРукаИгрока.Имя);
    //Далее проанализируем эту переменную:
    Если Статус = Ложь Тогда
        //Карты закончились. Игрок не может добрать карту и он проигрывает.
        //Выключим доступность группы (чтобы игрок не мог по ней нажать):
        Элементы.ГруппаРукаИгрока.Доступность = Ложь;
        ПредупреждениеАсинх("Карты закончились! Проигрыш!");
    КонецЕсли;
КонецПроцедуры

```

Помимо этого, в момент добора карт имеет смысл очистить невидимые и неиспользуемые реквизиты и элементы. Для этого на стороне сервера в функции «ВытащитьКартуНаСервере» добавим программную логику:

```

&НаСервере
Функция ВытащитьКартуНаСервере(ИмяГруппы)
    //Добавим два массива, так как есть вероятность, что ненужных реквизитов и
элементов будет несколько:
    МассивУдаляемыхРеквизитов = Новый Массив;
    МассивУдаляемыхЭлементов = Новый Массив;
    Группа = Элементы.Найти(ИмяГруппы);
    //Подготавливаем наборы элементов и реквизитов к удалению
    //Обойдём все элементы в группе:
    Для Каждого Элемент Из Группа.ПодчиненныеЭлементы Цикл
        //Если элемент невидимый, значит, поместим сыгранную карту в массивы для
удаления:
        Если Элемент.Видимость = Ложь Тогда
            МассивУдаляемыхРеквизитов.Добавить(Элемент.ПутьКДанным);
            МассивУдаляемыхЭлементов.Добавить(Элемент);
        КонецЕсли;
    КонецЦикла;
    //Удаление реквизитов и элементов
    ИзменитьРеквизиты(, МассивУдаляемыхРеквизитов);

```

```

Для Каждого Элемент Из МассивУдаляемыхЭлементов Цикл
    Элементы.Удалить(Элемент);
КонецЦикла;
Для Номер = 1 По 100 Цикл
    Если Элементы.Найти("КартаНаРуках" + Номер) = Неопределено Тогда
        Возврат ДобавитьКартуНаПоле(Номер, Группа);
    Прервать;
КонецЕсли;
КонецЦикла;
КонецФункции

```

Теперь все ненужные элементы и реквизиты будут удаляться и перестанут нагружать систему.

Далее реализуем возможность перезапуска игры. Он должен происходить, когда игрок нажимает на кнопку «Начать игру». Перед тем как запустится «цепочка» создания руки игрока и бота, произведём очистку с помощью отдельной процедуры:

```

&НаКлиенте
Процедура НачатьИгру(Команда)
    ОчиститьФорму();
    //Тут очистим сброс:
    ЭтотОбъект.Сброс = "";
    НачатьИгруНаСервере();
КонецПроцедуры

//Создадим новую серверную процедуру:
&НаСервере
Процедура ОчиститьФорму()
    //Создадим массив под удаление:
    МассивРеквизитовПодУдаление = Новый Массив;
    //Собираем реквизиты, связанные с элементами руки бота
    Для Каждого Элемент Из Элементы.ГруппаРукаБота.ПодчиненныеЭлементы Цикл
        МассивРеквизитовПодУдаление.Добавить(Элемент.ПутьКДанным);
    КонецЦикла;
    //Собираем реквизиты, связанные с элементами руки игрока
    Для Каждого Элемент Из Элементы.ГруппаРукаИгрока.ПодчиненныеЭлементы Цикл
        МассивРеквизитовПодУдаление.Добавить(Элемент.ПутьКДанным);
    КонецЦикла;
    //Удаляем реквизиты, если они есть
    Если МассивРеквизитовПодУдаление.Количество() > 0 Тогда
        ИзменитьРеквизиты( , МассивРеквизитовПодУдаление);
    КонецЕсли;
КонецПроцедуры

```

Запустим игру в пользовательском режиме и проверим работоспособность новых механизмов. Если повторно нажать кнопку «Начать игру», сброс очистится, и текущая карта станет новой.

Улучшим интерфейс игры, заменим заголовки текущей карты и карты сброса.

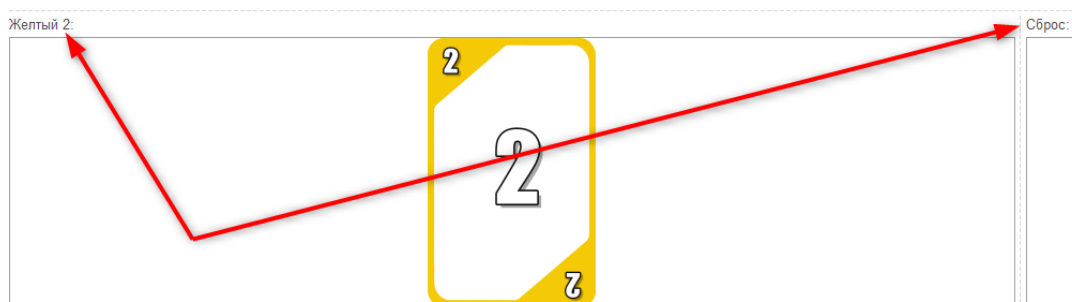


Рис. 1.8.1. Заголовки карт

Для этого вернёмся в режим configurator и откроем соответствующую форму обработки. На области элементов с помощью клавиши «Ctrl» выделим «ТекущаяКарта» и «Сброс» и откроем их свойства (рис. 1.8.2).

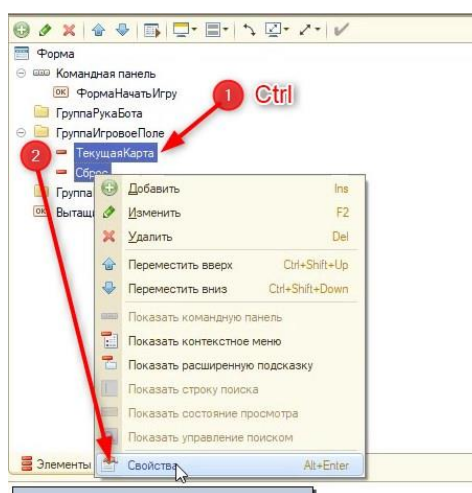


Рис. 1.8.2. Заголовки карт

После этого укажем «ПоложениеЗаголовка» — «Нет» (рис. 1.8.3).

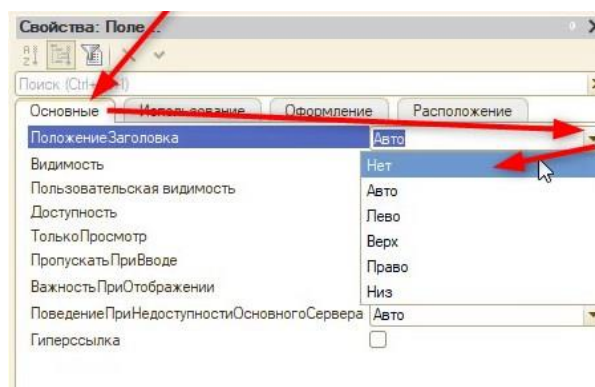


Рис. 1.8.3. Свойство «Положение заголовка»

Создадим две обычные группы (рис. 1.8.4):

- 1) «ГруппаТекущаяКарта» с заголовком «Текущая карта» (рис. 1.8.5).
- 2) «ГруппаСброс» с заголовком «Сброс».

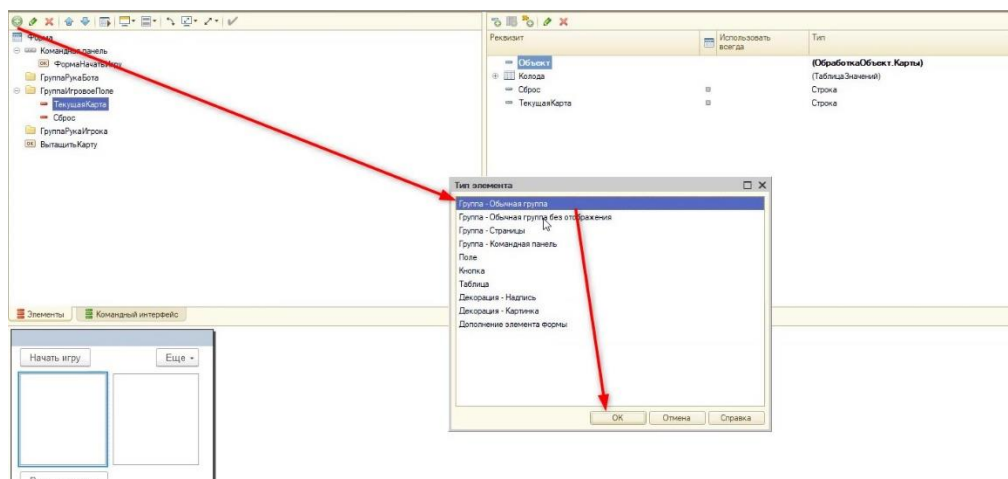


Рис. 1.8.4. Создание группы

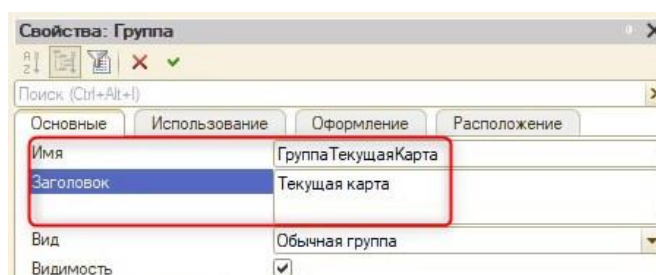


Рис. 1.8.5. Группа с текущей картой

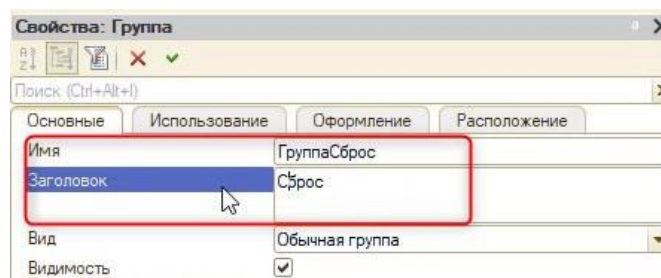


Рис. 1.8.6. Группа сброса

После этого расположим на форме новые группы следующим образом (см. рис. 1.8.7).

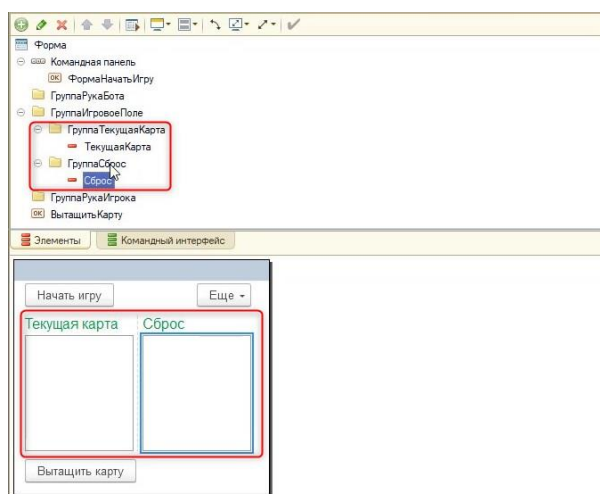


Рис. 1.8.7. Расположение новых групп

Теперь в режиме пользователя будут новые заголовки (рис. 1.8.8).

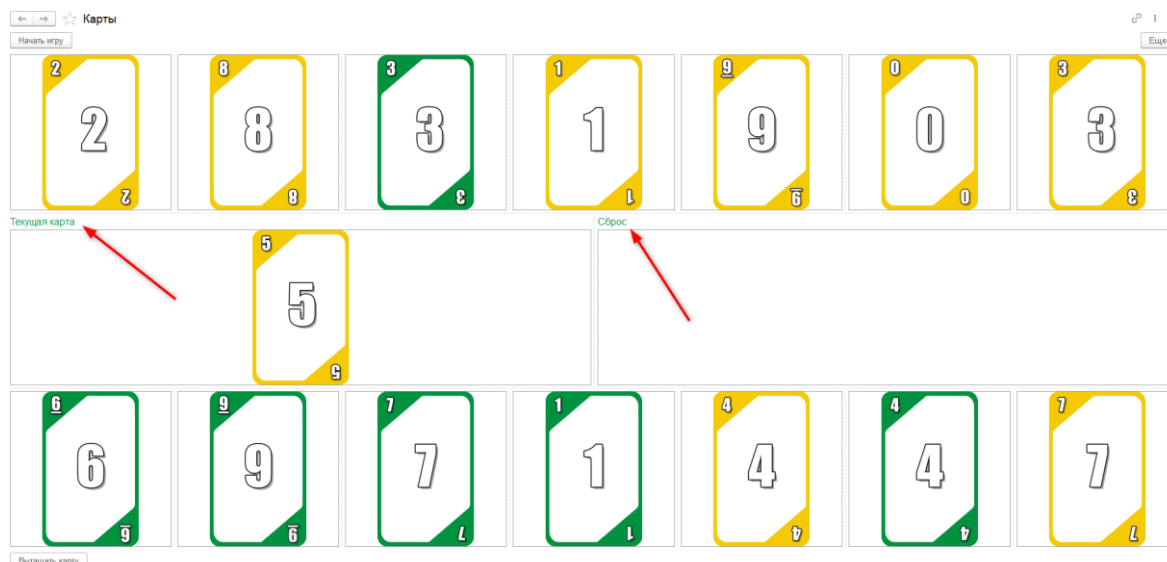


Рис. 1.8.8. Новые заголовки в режиме пользователя

Реализуем ход компьютерного игрока (бота). Ход бота будет происходить после того, как игрок выберет карту. После своего хода он должен передать игроку возможность продолжить игру.

Для начала дополним процедуру, отвечающую за ход игрока — «ВыборКарты»:

Процедура ВыборКарты(Элемент, СтандартнаяОбработка)

//Создадим счетчик карт, для последующей проверки и обнулим его

СчетчикКарт = 0;

ОчиститьСообщения();

СтандартнаяОбработка = Ложь;

МассивДанныхОКартеВРуке = СтрРазделить(Элемент.Заголовок, " ", Ложь);

МассивДанныхОКартеНаСтоле = СтрРазделить(Элементы.ТекущаяКарта.Заголовок, " ", Ложь);

Если МассивДанныхОКартеВРуке[0] = МассивДанныхОКартеНаСтоле[0] ИЛИ МассивДанныхОКартеВРуке[1] = МассивДанныхОКартеНаСтоле[1] Тогда

СыгратьКарту(Элемент.Имя);

Элемент.Видимость = Ложь;

//Проверка что выкладываем последнюю карту с руки

//Считаем количество карт в руке

Для Каждого Карта Из Элементы.ГруппаРукаИгрока.ПодчиненныеЭлементы Цикл

Если Карта.Видимость = Истина Тогда

СчетчикКарт = СчетчикКарт + 1;

КонецЕсли;

КонецЦикла;

//Если карт не осталось, то победа

Если СчетчикКарт = 0 Тогда

Элементы.ГруппаИгровоеПоле.Доступность = Ложь;

Элементы.ГруппаРукаИгрока.Доступность = Ложь;

ПредупреждениеАсинх("Ура, победа!");

```

        Возврат;
    КонечЕсли;
    //Далее должен произойти ход бота
Иначе
    Сообщить("Нельзя сыграть карту!");
    КонечЕсли;
КонечПроцедуры

```

Протестируем механизм в пользовательском режиме. Действительно, после того как все карты в руке у игрока заканчиваются, выводится сообщение «Ура, победа!» (рис. 1.8.9). Но если повторно начать игру, карты будут заблокированы для выбора (рис. 1.8.10).

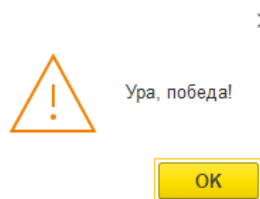


Рис. 1.8.9. Победа

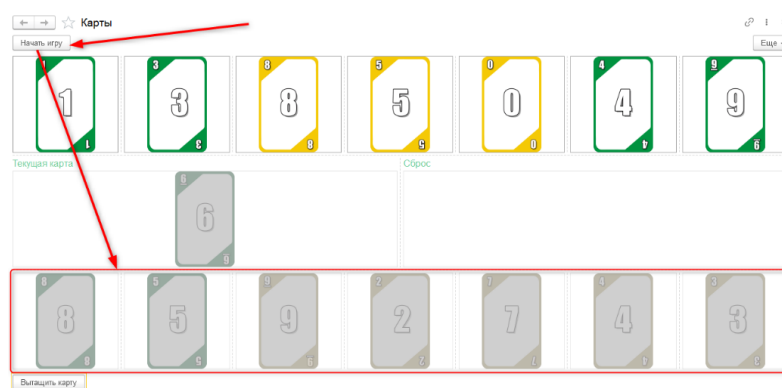


Рис. 1.8.10. Блокировка карт

Необходимо вмешаться в программную логику кнопки «Начать игру» и вернуть доступ к картам.

&НаКлиенте

Процедура НачатьИгру(Команда)

```

    ОчиститьФорму();
    ЭтотОбъект.Сброс = "";
    //Включим доступность:
    Элементы.ГруппаРукаИгрока.Доступность = Истина;
    Элементы.ГруппаИгровоеПоле.Доступность = Истина;
    //Включим кнопку "Вытащить карту"
    Элементы.ВытащитьКарту.Видимость = Истина;
    //Включим видимость игрового поля:
    Элементы.ГруппаИгровоеПоле.Видимость = Истина;
    НачатьИгруНаСервере();

```

КонечПроцедуры

Выключим видимость у кнопки «Вытащить карту» (рис. 1.8.11) и группы «Игровое поле» (рис. 1.8.12), для того чтобы форма была изначально для пользователя пустой.

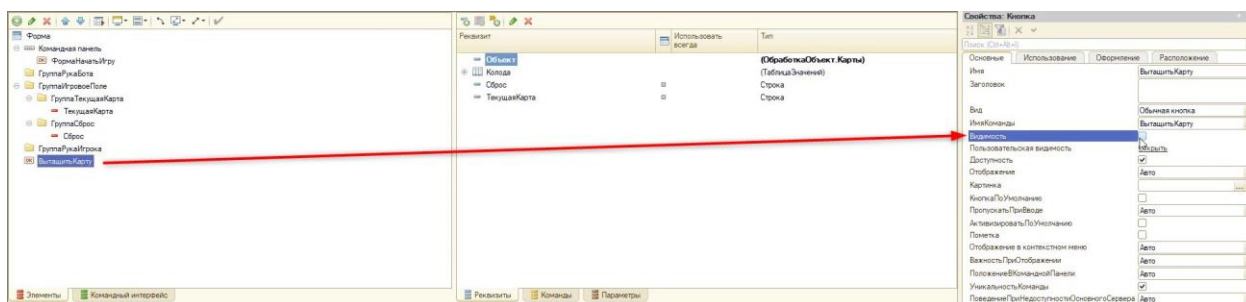


Рис. 1.8.11. Выключение видимости у кнопки

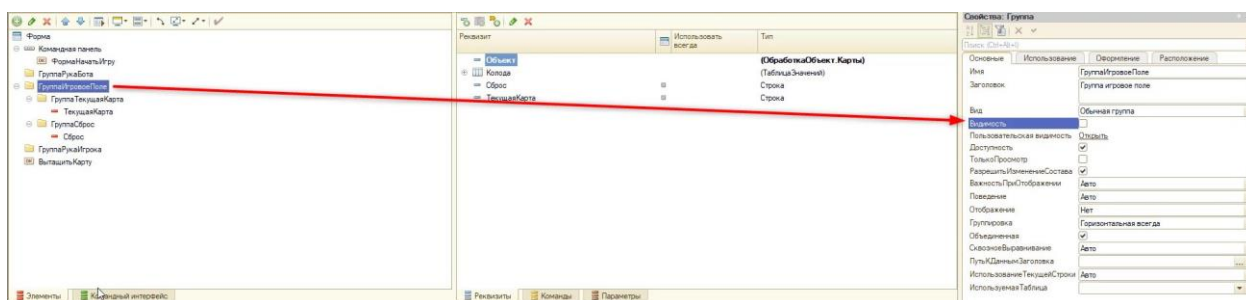


Рис. 1.8.12. Выключение видимости у группы

Проверим в пользовательском режиме. Сыграем до победы и попробуем заново начать игру. Если всё сделано правильно, карты снова будут активны для игры (рис. 1.8.13).

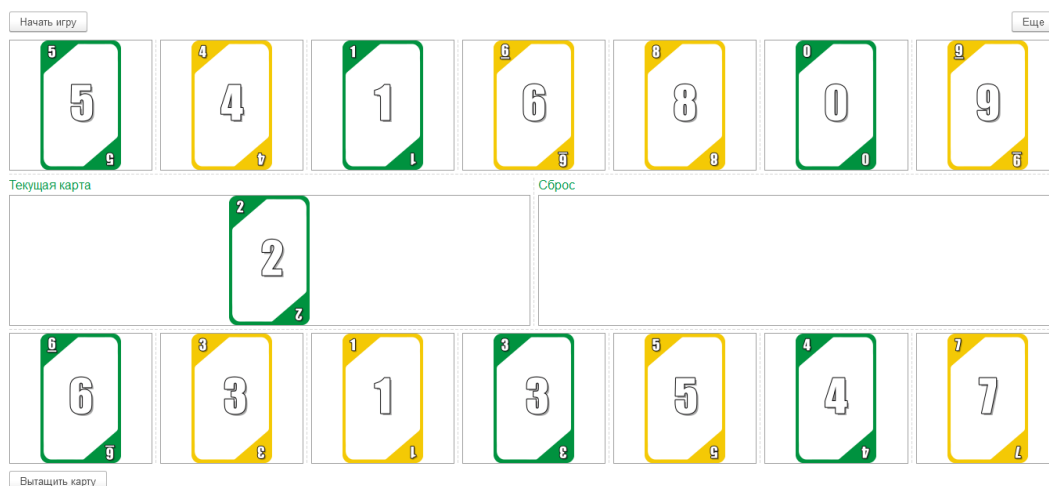


Рис. 1.8.13. Новая игра

Вернёмся к процедуре «ВыборКарты», которую редактировали ранее:

&НаКлиенте

Процедура ВыборКарты(Элемент, СтандартнаяОбработка)

СчетчикКарт = 0;

ОчиститьСообщения();

СтандартнаяОбработка = Ложь;

МассивДанныхОКартеВРуке = СтрРазделить(Элемент.Заголовок, " ", Ложь);

МассивДанныхОКартеНаСтоле = СтрРазделить(Элементы.ТекущаяКарта.Заголовок, " ", Ложь);

Цикл	Если МассивДанныхОКартеВРуке[0] = МассивДанныхОКартеНаСтоле[0] ИЛИ МассивДанныхОКартеВРуке[1] = МассивДанныхОКартеНаСтоле[1] Тогда СыгратьКарту(Элемент.Имя); Элемент.Видимость = Ложь; Для Каждого Карта Из Элементы.ГруппаРукаИгрока.ПодчиненныеЭлементы
	Если Карта.Видимость = Истина Тогда СчетчикКарт = СчетчикКарт + 1; КонецЕсли; КонецЦикла; Если СчетчикКарт = 0 Тогда Элементы.ГруппаИгровоеПоле.Доступность = Ложь; Элементы.ГруппаРукаИгрока.Доступность = Ложь; ПредупреждениеАсинх("Ура, победа!"); Возврат; КонецЕсли; //Передача хода боту //Проинформируем игрока, что бот начал свой ход: ЭтотОбъект.Заголовок = "Бот: Дай подумать..."; //Смоделируем ситуацию, что для бота нужно некоторое время, чтобы сделать
ход	//С помощью метода ПодключитьОбработчикОжидания(<ИмяПроцедуры>, <ИнтервалВСекундах>, <Однократно>) //Вызовем новую процедуру ХодБота, которую в дальнейшем опишем ПодключитьОбработчикОжидания("ХодБота", 1, Истина); Иначе Сообщить("Нельзя сыграть карту!"); КонецЕсли; КонецПроцедуры

После этого опишем новую процедуру «ХодБота»:

", Ложь);	&НаКлиенте Процедура ХодБота() ХодБыл = Ложь; МассивДанныхОКартеНаСтоле = СтрРазделить(Элементы.ТекущаяКарта.Заголовок, " //Считаем количество карт в руке бота КартУБота = 0; Для Каждого Элемент Из Элементы.ГруппаРукаБота.ПодчиненныеЭлементы Цикл //Если карта не разыграна (т.е. её видимость не отключена), то посчитаем карту Если Элемент.Видимость = Истина Тогда КартУБота = КартУБота + 1; КонецЕсли; КонецЦикла; //Перебор карт из руки
-----------	--

```

Для Каждого Карта Из Элементы.ГруппаРукаБота.ПодчиненныеЭлементы Цикл
    Если Карта.Видимость = Истина Тогда
        МассивДанныхОКартеБота = СтрРазделить(Карта.Заголовок, " ", Ложь);
        //Если может сыграть карту
        Если МассивДанныхОКартеБота[0] = МассивДанныхОКартеНаСтоле[0] ИЛИ
МассивДанныхОКартеБота[1] = МассивДанныхОКартеНаСтоле[1] Тогда
            СыгратьКарту(Карта.Имя);
            Карта.Видимость = Ложь;
            КартУБота = КартУБота - 1;
            ЭтотОбъект.Заголовок = "Бот: Готово!";

            //Если выложил последнюю карту из руки
            Если КартУБота = 0 Тогда
                Элементы.ГруппаИгровоеПоле.Доступность = Ложь;
                Элементы.ГруппаРукаИгрока.Доступность = Ложь;
                ПредупреждениеАсинх("Бот: Я выиграл!");
            КонецЕсли;
            ХодБыл = Истина;
            Прервать;
        Иначе
            //Если карта перебора не подошла
            ЭтотОбъект.Заголовок = "Бот: Думаю...";
            КонецЕсли;
        КонецЕсли;
    КонецЦикла;
    //Если ни одна карта не подошла
    Если ХодБыл = Ложь Тогда
        //Если бот не ходил, то пытается взять карту из колоды
        Если ВытащитьКартуНаСервере(Элементы.ГруппаРукаБота.Имя) = Истина Тогда
            Сообщить("Бот: Возьму еще карту!");
            ХодБота();
        Иначе //Если не получается - проигрыш бота
            Элементы.ГруппаИгровоеПоле.Доступность = Ложь;
            Элементы.ГруппаРукаИгрока.Доступность = Ложь;
            ПредупреждениеАсинх("Бот: Карты в колоде закончились! Я проиграл!");
            КонецЕсли;
        КонецЕсли;
    КонецПроцедуры

```

Проверим механизм хода бота в режиме пользователя. Далее, если несколько игр прошли успешно, можно выключить отображение руки компьютера. Сделать это можно с помощью свойства «Видимость» (рис. 1.8.14).

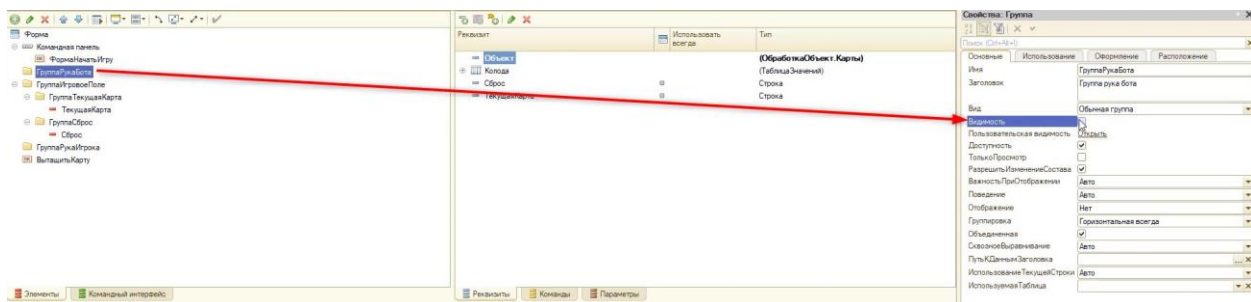


Рис. 1.8.14. Отключение видимости руки бота

Отлично! Осталось совсем немного, и игра будет готова.